Smart Campus Parking and Traffic Management System

CSA0925-Programming in Java

K.Dhilip (192365013)

Lithish Kumar R (192424169)

Mohammed Subahaan(192572165)

Pseudo code:

```
// SYSTEM ARCHITECTURE & UNIFIED PSEUDOCODE
// Project: Smart Campus Parking & Traffic Management System
// Framework: Java AWT (Cross-Platform GUI) + JDBC + MySQL Database
// Database Schema: smart_campus_parking
// MODULE 1: DATABASE CONNECTION & PERSISTENCE LAYER
CLASS DBConnection:
    CONSTANT DB_URL = "jdbc:mysql://localhost:3306/smart_campus_parking"
    CONSTANT DB_USER = "root"
    CONSTANT DB_PASS = "2601"
    STATIC isConnected = NULL
    FUNCTION getConnection():
        TRY:
            Load Driver "com.mysql.cj.jdbc.Driver"
            connection = DriverManager.getConnection(DB_URL, DB_USER, DB_PASS)
            isConnected = TRUE
            RETURN connection
        CATCH Exception:
            isConnected = FALSE
            Log "Database offline. Operating in Resilient In-Memory Mode."
            RETURN NULL

    FUNCTION isDatabaseAvailable():
        con = getConnection()
```

RETURN (con != NULL AND NOT con.isClosed())

// MODULE 2: DATA ENTITY MODELS

CLASS User:

ATTRIBUTES: userId, name, email, phone, userType, password, status

CLASS Vehicle:

ATTRIBUTES: vehicleId, userId, vehicleNumber, vehicleType, model, color

CLASS ParkingZone:

ATTRIBUTES: zoneId, zoneName, location, zoneType, totalSlots, availableSlots, status

CLASS ParkingSlot:

ATTRIBUTES: slotId, zoneId, slotNumber, vehicleType, slotStatus

CLASS Reservation:

ATTRIBUTES: reservationId, userId, vehicleId, slotId, reservationDate, startTime, endTime, reservationStatus

CLASS ParkingSession:

ATTRIBUTES: sessionId, vehicleId, slotId, entryTime, exitTime, durationMinutes, parkingFee, sessionStatus

CLASS ParkingPass:

ATTRIBUTES: passId, userId, vehicleId, passType, startDate, endDate, passStatus, passAmount

CLASS Payment:

ATTRIBUTES: paymentId, sessionId, passId, userId, amount, paymentMethod, paymentDate, paymentStatus, transactionReference

CLASS Violation:

ATTRIBUTES: violationId, userId, vehicleId, violationType, description, violationDate, fineAmount, violationStatus

// MODULE 3: DATA ACCESS OBJECTS (DAO) & AUTO-GENERATION ENGINE

CLASS GenericDAO:

STATIC In-Memory Repositories:

usersList, vehiclesList, zonesList, slotsList,

```
    reservationsList, sessionsList, passesList, paymentsList, violationsList
FUNCTION getNextId(tableName, idColumn, inMemoryList):
    IF DBConnection.isDatabaseAvailable():
        sql = "SELECT COALESCE(MAX(" + idColumn + "), 0) + 1 FROM " + tableName
        EXECUTE SQL query AND RETURN result
    ELSE:
        maxId = Find Maximum ID in inMemoryList
        RETURN maxId + 1
FUNCTION executeInsert(record, sqlQuery, inMemoryList):
    IF DBConnection.isDatabaseAvailable():
        EXECUTE PreparedStatement with SQL parameters
    ELSE:
        IF record.id == 0:
            record.id = getNextId(...)
        inMemoryList.ADD(record)
        RETURN TRUE
FUNCTION executeSearch(id, sqlQuery, inMemoryList):
    IF DBConnection.isDatabaseAvailable():
        EXECUTE SQL SELECT WHERE id = ? AND RETURN Record
    ELSE:
        RETURN Record from inMemoryList WHERE record.id == id
FUNCTION executeUpdate(record, sqlQuery, inMemoryList):
    IF DBConnection.isDatabaseAvailable():
        EXECUTE SQL UPDATE WHERE id = ?
    ELSE:
        REPLACE matching record in inMemoryList WITH record
        RETURN TRUE
FUNCTION executeDelete(id, sqlQuery, inMemoryList):
    IF DBConnection.isDatabaseAvailable():
```

```
        EXECUTE SQL DELETE WHERE id = ?
    ELSE:
        REMOVE matching record from inMemoryList WHERE record.id == id
        RETURN TRUE

FUNCTION getAllRecords(tableName, inMemoryList):
    IF DBConnection.isDatabaseAvailable():
        EXECUTE SQL "SELECT * FROM " + tableName
        RETURN List of fetched records
    ELSE:
        RETURN Copy of inMemoryList
// MODULE 4: CUSTOM CROSS-PLATFORM AWT BUTTON (AwtButton)
CLASS AwtButton EXTENDS java.awt.Panel:
    ATTRIBUTES: text, bgColor, hoverColor, pressedColor, fgColor, isHovered, isPressed, actionListeners
    CONSTRUCTOR(text, bgColor, fgColor, font):
        Set custom text and background color
        Attach MouseAdapter (hover -> true, exit -> false, press -> true, release -> fireActionEvent)
    FUNCTION paint(Graphics g):
        Enable Antialiasing for Text and Shapes
        Compute active color (if pressed -> pressedColor, if hovered -> hoverColor, else -> bgColor)
        Draw Rounded Shadow Box
        Draw Rounded Button Shape with active color
        Draw Top Gloss Border Highlight
        Draw Horizontally and Vertically Centered Bold Text and Icons
    FUNCTION getPreferredSize():
        Calculate Width = FontMetrics.stringWidth(text) + 24 Padding (Min 105px)
        Calculate Height = 38px
        RETURN Dimension(Width, Height)
```

// MODULE 5: LIVE PARKING AVAILABILITY MAP & BLINKING VISUAL GRID

CLASS ParkingAvailabilityDashboard EXTENDS Frame:

ATTRIBUTES: blinkThread, isBlinkActive = TRUE, blinkToggle = FALSE, slotDAO, renderedCardsList

FUNCTION initializeUI():

Set Title "Campus Real-Time Parking Availability Map"

Create Metrics Header: [Total Slots], [Available Slots], [Occupied Slots], [Reserved Slots], [Occupancy %]

Create Filter Toolbar: [Zone Filter], [Vehicle Filter], [Status Filter], [Refresh], [Toggle Blink], [Close]

Create Scrollable Visual Grid Container (GridLayout 4 Columns)

Create Color Legend Footer

Attach Event Listeners

Load and Render Slots Grid

Start Background Blink Animation Thread

FUNCTION startBlinkThread():

SPAWN Background Daemon Thread:

WHILE isBlinkActive IS TRUE:

SLEEP 550 milliseconds

blinkToggle = NOT blinkToggle

FOR EACH card IN renderedCardsList:

IF card.slot.status EQUALS "Available":

IF blinkToggle IS TRUE:

Set Background to Glowing Luminous Lime Green (Phase 1)

Set Header Text "□ " + slotNumber + " □ "

Set Badge "□ OPEN / VACANT"

ELSE:

Set Background to Deep Forest Green (Phase 2)

Set Header Text slotNumber

Set Badge "AVAILABLE"

Repaint Slot Card Component

FUNCTION renderGrid(slotsList):

Clear Grid Container

FOR EACH slot IN slotsList:

card = Create SlotVisualCard(slot)

IF slot.status EQUALS "Occupied": Set Card Background Crimson Red

ELSE IF slot.status EQUALS "Reserved": Set Card Background Amber Orange

ELSE IF slot.status EQUALS "Maintenance": Set Card Background Slate Gray

ELSE: Add card to renderedCardsList for Blinking Animation

Attach Click Listener -> openSlotActionDialog(slot)

Add card to Grid Container

FUNCTION openSlotActionDialog(slot):

Open Modal Dialog

Display Slot ID, Zone, Code, Vehicle Type

Provide Status Selection: [Available | Occupied | Reserved | Maintenance]

WHEN "Save Status" Clicked:

slotDAO.updateSlotStatus(slot.id, selectedStatus)

Refresh Dashboard Grid & Metrics

// MODULE 6: MANAGEMENT MODULES (User, Vehicle, Zone, Slot, Reservation, Session, Pass, Payment)

CLASS BaseManagementWindow EXTENDS Frame:

ATTRIBUTES: idField, formFields, tableArea, statusBanner, actionButtons

FUNCTION initializeWindow(moduleTitle, nextId, fieldsList):

Build Top Header with Module Title & Subtitle

Build Left Form Card (Top-aligned GridBagLayout with vertical filler):

Add ID Field pre-filled with nextId (Auto-Generated)

FOR EACH field IN fieldsList:

Add Label + Input Field (TextField / Choice)

Build Right Records Card:

Add Monospaced Scrollable Table Area

```
    Build Bottom Toolbar with AwtButtons:
        [ADD], [SEARCH], [UPDATE], [DELETE], [VIEW ALL], [CLEAR], [CLOSE]

    Load and Display all existing records in Table Area immediately
FUNCTION handleAction(actionType):
    SWITCH actionType:
        CASE "ADD":
            Validate required fields
            record = Extract values from form inputs
            DAO.insert(record)
            Display Success Notification
            clearForm()
            refreshRecordsTable()
        CASE "SEARCH":
            id = Parse integer from ID Field
            record = DAO.search(id)
            IF record exists:
                Populate form inputs with record details
                statusBanner.setText("Found record ID " + id)
            ELSE:
                Show Popup "Record not found"
        CASE "UPDATE":
            id = Parse integer from ID Field
            record = Extract updated values from form inputs
            DAO.update(record)
            Display "Record updated successfully"
            clearForm()
            refreshRecordsTable()
```

```
        CASE "DELETE":
            id = Parse integer from ID Field
            DAO.delete(id)
            Display "Record deleted"
            clearForm()
            refreshRecordsTable()

        CASE "VIEW ALL":
            refreshRecordsTable()
        CASE "CLEAR":
            clearForm()
        CASE "CLOSE":
            Close Current Window
    FUNCTION clearForm():
        idField.setText(DAO.getNextId())
        Reset all input fields to defaults
        statusBanner.setText("Ready | Next ID auto-assigned: " + idField.getText())
    FUNCTION refreshRecordsTable():
        records = DAO.getAll()
        Format records into tabulated monospace text
        tableArea.setText(formattedText)
// MODULE 7: MAIN SYSTEM DASHBOARD & APPLICATION ENTRY POINT
CLASS SmartCampusParkingSystem EXTENDS Frame:

    FUNCTION startMainDashboard():
        Set Window Size (1040 x 750), Centered on Screen
        Set Background Color Light Slate (245, 247, 250)

        Create Header: "SMART CAMPUS PARKING & TRAFFIC MANAGEMENT SYSTEM"
```

Create Navigation Tile Grid (6 Rows x 2 Columns) with AwtButtons:

Button 1: "□ LIVE PARKING AVAILABILITY MAP" -> OPEN ParkingAvailabilityDashboard

Button 2: "□ PARKING SLOTS" -> OPEN ParkingSlotManagement

Button 3: "□ USER MANAGEMENT" -> OPEN UserManagement

Button 4: "□ VEHICLE MANAGEMENT" -> OPEN VehicleManagement

Button 5: "□ PARKING ZONES" -> OPEN ParkingZoneManagement

Button 6: "□ RESERVATION" -> OPEN ReservationManagement

Button 7: "□ PARKING SESSION" -> OPEN ParkingSessionManagement

Button 8: "□ PARKING PASS" -> OPEN ParkingPassManagement

Button 9: "□ PAYMENT" -> OPEN PaymentManagement

Button 10: "□ VIOLATIONS" -> OPEN ViolationManagement

Button 11: "□ EXIT SYSTEM" -> System.exit(0)

Create Footer: "Java AWT | JDBC | MySQL Database (smart_campus_parking)"

Show Window

// MAIN EXECUTION LIFECYCLE

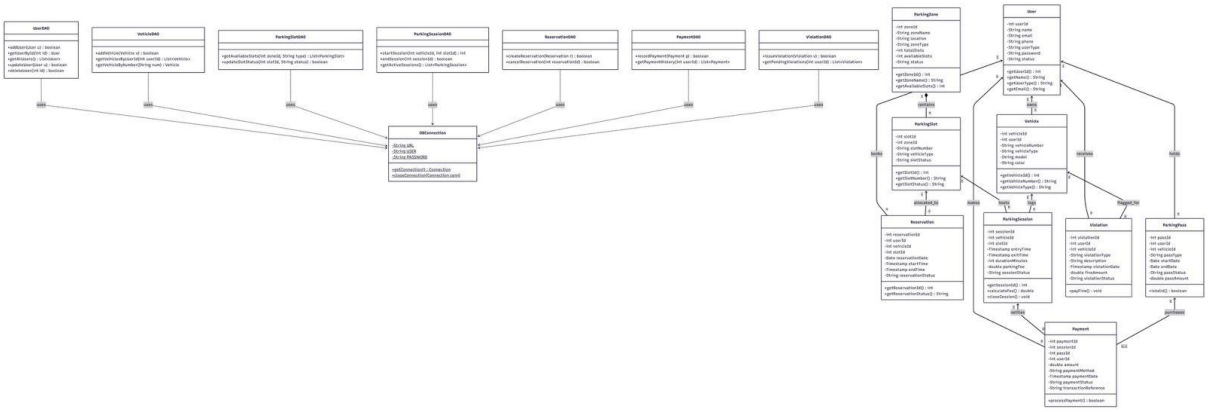
FUNCTION main():

PRINT "Starting Smart Campus Parking & Traffic Management System..."

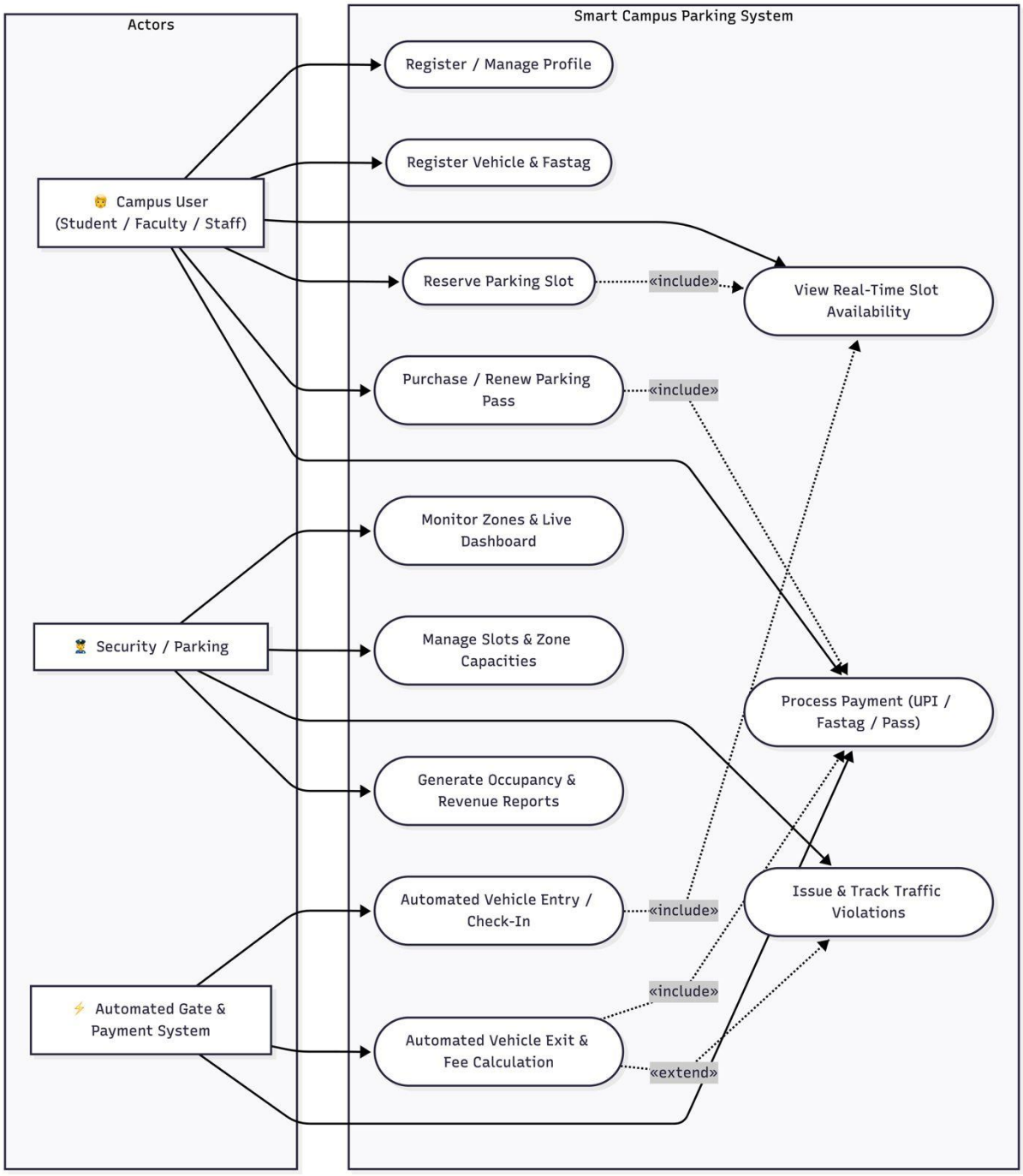
Initialize DBConnection

LAUNCH SmartCampusParkingSystem()

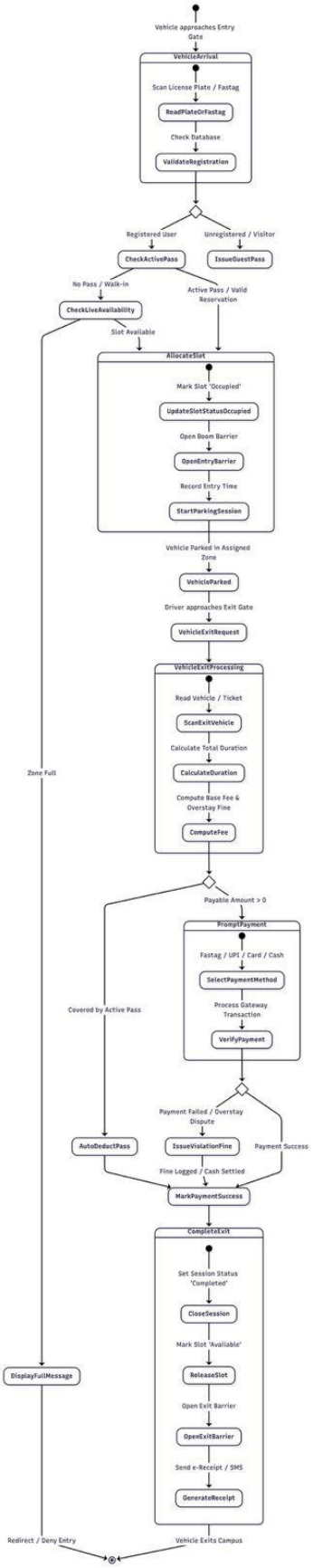
Class Diagram:



Use Case Diagram:



State Diagram:



Source Code:

Table Creation:

```
CREATE DATABASE IF NOT EXISTS smart_campus_parking;
USE smart_campus_parking;

-- 1. USERS TABLE
CREATE TABLE IF NOT EXISTS users (
    user_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    phone VARCHAR(20),
    user_type ENUM('Student', 'Faculty', 'Staff', 'Admin') DEFAULT 'Student',
    password VARCHAR(255) DEFAULT 'pass123',
    status ENUM('Active', 'Inactive') DEFAULT 'Active',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 2. VEHICLES TABLE
CREATE TABLE IF NOT EXISTS vehicles (
    vehicle_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT NOT NULL,
    vehicle_number VARCHAR(30) UNIQUE NOT NULL,
    vehicle_type ENUM('Two Wheeler', 'Four Wheeler', 'EV') NOT NULL,
    model VARCHAR(100),
    color VARCHAR(50),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
);

-- 3. PARKING ZONES TABLE
CREATE TABLE IF NOT EXISTS parking_zones (
    zone_id INT PRIMARY KEY AUTO_INCREMENT,
    zone_name VARCHAR(100) NOT NULL,
    location VARCHAR(255),
    zone_type VARCHAR(100) DEFAULT 'Two & Four Wheeler',
    total_slots INT NOT NULL DEFAULT 10,
    available_slots INT NOT NULL DEFAULT 10,
    status ENUM('Active', 'Inactive', 'Under Maintenance') DEFAULT 'Active'
);

-- 4. PARKING SLOTS TABLE
CREATE TABLE IF NOT EXISTS parking_slots (
    slot_id INT PRIMARY KEY AUTO_INCREMENT,
    zone_id INT NOT NULL,
    slot_number VARCHAR(20) NOT NULL,
    vehicle_type ENUM('Two Wheeler', 'Four Wheeler', 'EV') NOT NULL,
        slot_status ENUM('Available', 'Occupied', 'Reserved', 'Maintenance') DEFAULT
'Available',
```

```
    FOREIGN KEY (zone_id) REFERENCES parking_zones(zone_id) ON DELETE CASCADE
);

-- 5. RESERVATIONS TABLE
CREATE TABLE IF NOT EXISTS reservations (
    reservation_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT NOT NULL,
    vehicle_id INT NOT NULL,
    slot_id INT NOT NULL,
    reservation_date DATE NOT NULL,
    start_time TIMESTAMP NOT NULL,
    end_time TIMESTAMP NOT NULL,
    reservation_status ENUM('Active', 'Confirmed', 'Cancelled', 'Completed') DEFAULT 'Active',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE,
    FOREIGN KEY (vehicle_id) REFERENCES vehicles(vehicle_id) ON DELETE CASCADE,
    FOREIGN KEY (slot_id) REFERENCES parking_slots(slot_id) ON DELETE CASCADE
);

-- 6. PARKING SESSIONS TABLE
CREATE TABLE IF NOT EXISTS parking_sessions (
    session_id INT PRIMARY KEY AUTO_INCREMENT,
    vehicle_id INT NOT NULL,
    slot_id INT NOT NULL,
    entry_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    exit_time TIMESTAMP NULL,
    duration_minutes INT DEFAULT 0,
    parking_fee DECIMAL(10, 2) DEFAULT 0.00,
    session_status ENUM('Active', 'Completed', 'Overstay') DEFAULT 'Active',
    FOREIGN KEY (vehicle_id) REFERENCES vehicles(vehicle_id) ON DELETE CASCADE,
    FOREIGN KEY (slot_id) REFERENCES parking_slots(slot_id) ON DELETE CASCADE
);

-- 7. PARKING PASSES TABLE
CREATE TABLE IF NOT EXISTS parking_passes (
    pass_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT NOT NULL,
    vehicle_id INT NOT NULL,
    pass_type ENUM('Daily', 'Weekly', 'Monthly', 'Semester', 'Annual') DEFAULT 'Monthly',
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    pass_status ENUM('Active', 'Expired', 'Cancelled') DEFAULT 'Active',
    pass_amount DECIMAL(10, 2) NOT NULL DEFAULT 500.00,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE,
```

```
    FOREIGN KEY (vehicle_id) REFERENCES vehicles(vehicle_id) ON DELETE CASCADE
);

-- 8. PAYMENTS TABLE
CREATE TABLE IF NOT EXISTS payments (
    payment_id INT PRIMARY KEY AUTO_INCREMENT,
    session_id INT NULL,
    pass_id INT NULL,
    user_id INT NOT NULL,
    amount DECIMAL(10, 2) NOT NULL,
     payment_method ENUM('UPI', 'Fastag', 'Credit Card', 'Debit Card', 'Cash') DEFAULT 'UPI',
    payment_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    payment_status ENUM('Success', 'Pending', 'Failed') DEFAULT 'Success',
    transaction_reference VARCHAR(100),
    FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
);

-- 9. VIOLATIONS TABLE
CREATE TABLE IF NOT EXISTS violations (
    violation_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT NOT NULL,
    vehicle_id INT NOT NULL,
    violation_type VARCHAR(100) NOT NULL,
    description TEXT,
    violation_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    fine_amount DECIMAL(10, 2) DEFAULT 100.00,
    violation_status ENUM('Pending', 'Paid', 'Waived') DEFAULT 'Pending',
    FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE,
     FOREIGN KEY (vehicle_id) REFERENCES vehicles(vehicle_id) ON DELETE CASCADE
);

-- Insert Users
INSERT INTO users (user_id, name, email, phone, user_type, password, status) VALUES
(101, 'Lithish Kumar', 'lithish@campus.edu', '9876543210', 'Student', 'pass123', 'Active'),
(102, 'Dr. Rajesh Sharma', 'rajesh.s@campus.edu', '9845012345', 'Faculty', 'pass123', 'Active'),
(103, 'Priya Nair', 'priya.n@campus.edu', '9740123456', 'Staff', 'pass123', 'Active'),
(104, 'Rahul Verma', 'rahul.v@campus.edu', '9632145870', 'Student', 'pass123', 'Active'),
(105, 'Campus Security Admin', 'security@campus.edu', '9123456780', 'Admin', 'pass123', 'Active')
ON DUPLICATE KEY UPDATE name=name;

-- Insert Vehicles
INSERT INTO vehicles (vehicle_id, user_id, vehicle_number, vehicle_type, model, color) VALUES
(1, 101, 'KA-01-AB-1234', 'Four Wheeler', 'Honda City', 'White'),
(2, 102, 'KA-04-CD-5678', 'Two Wheeler', 'Yamaha FZ', 'Black'),
```

```
(3, 103, 'KA-05-EF-9012', 'EV', 'Tata Nexon EV', 'Blue'),
(4, 104, 'KA-03-GH-3456', 'Four Wheeler', 'Hyundai Creta', 'Silver'),
(5, 105, 'KA-02-JK-7890', 'Two Wheeler', 'Royal Enfield', 'Red')
ON DUPLICATE KEY UPDATE vehicle_number=vehicle_number;

-- Insert Parking Zones
INSERT INTO parking_zones (zone_id, zone_name, location, zone_type, total_slots, available_slots, status) VALUES
(1, 'Main Gate Parking', 'Near Gate 1 - Campus Entrance', 'Two & Four Wheeler', 8, 4, 'Active'),
(2, 'Science Block Parking', 'Behind Science & Research Center', 'Faculty & Student', 6, 3, 'Active'),
(3, 'Admin Block Parking', 'Opposite Administrative Building', 'VIP & Staff', 6, 3, 'Active'),
(4, 'Academic Block Parking', 'Next to Main Library & Lecture Halls', 'Student Only', 6, 4, 'Active')
ON DUPLICATE KEY UPDATE zone_name=zone_name;

-- Insert Parking Slots
INSERT INTO parking_slots (slot_id, zone_id, slot_number, vehicle_type, slot_status) VALUES
-- Zone 1: Main Gate
(101, 1, 'A-01', 'Four Wheeler', 'Available'),
(102, 1, 'A-02', 'Four Wheeler', 'Occupied'),
(103, 1, 'A-03', 'Four Wheeler', 'Available'),
(104, 1, 'A-04', 'EV', 'Occupied'),
(105, 1, 'A-05', 'Two Wheeler', 'Available'),
(106, 1, 'A-06', 'Two Wheeler', 'Occupied'),
(107, 1, 'A-07', 'Two Wheeler', 'Available'),
(108, 1, 'A-08', 'Four Wheeler', 'Reserved'),

-- Zone 2: Science Block
(201, 2, 'B-01', 'Four Wheeler', 'Occupied'),
(202, 2, 'B-02', 'Four Wheeler', 'Available'),
(203, 2, 'B-03', 'EV', 'Available'),
(204, 2, 'B-04', 'Two Wheeler', 'Occupied'),
(205, 2, 'B-05', 'Two Wheeler', 'Available'),
(206, 2, 'B-06', 'Four Wheeler', 'Maintenance'),

-- Zone 3: Admin Block
(301, 3, 'C-01', 'Four Wheeler', 'Reserved'),
(302, 3, 'C-02', 'Four Wheeler', 'Available'),
(303, 3, 'C-03', 'EV', 'Occupied'),
(304, 3, 'C-04', 'Two Wheeler', 'Available'),
(305, 3, 'C-05', 'Four Wheeler', 'Available'),
(306, 3, 'C-06', 'Four Wheeler', 'Occupied'),

-- Zone 4: Academic Block
(401, 4, 'D-01', 'Two Wheeler', 'Available'),
(402, 4, 'D-02', 'Two Wheeler', 'Occupied'),
(403, 4, 'D-03', 'Two Wheeler', 'Available'),
```

```
(404, 4, 'D-04', 'Four Wheeler', 'Available'),
(405, 4, 'D-05', 'Four Wheeler', 'Occupied'),
(406, 4, 'D-06', 'EV', 'Available')
ON DUPLICATE KEY UPDATE slot_number=slot_number;
```

AwtButton.java

```
package parking;

import java.awt.*;
import java.awt.event.*;
import java.util.ArrayList;
import java.util.List;

public class AwtButton extends Panel {

    private String text;
    private Color bgColor;
    private Color hoverColor;
    private Color pressedColor;
    private Color fgColor;
    private Font buttonFont;
    private int cornerRadius = 8;
    private boolean isHovered = false;
    private boolean isPressed = false;
    private final List<ActionListener> listeners = new ArrayList<>();

    public AwtButton(String text, Color bgColor, Color fgColor) {
        this(text, bgColor, fgColor, new Font("Arial", Font.BOLD, 13));
    }

    public AwtButton(String text, Color bgColor, Color fgColor, Font font) {
        this.text = text;
        this.bgColor = bgColor;
        this.fgColor = fgColor;
        this.buttonFont = font != null ? font : new Font("Arial", Font.BOLD, 13);

        this.hoverColor = brighten(bgColor, 25);
        this.pressedColor = darken(bgColor, 30);

        setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));

        MouseAdapter ma = new MouseAdapter() {
            @Override
            public void mouseEntered(MouseEvent e) {
                isHovered = true;
                repaint();
            }
```

```
        @Override
        public void mouseExited(MouseEvent e) {
            isHovered = false;
            isPressed = false;
            repaint();
        }

        @Override
        public void mousePressed(MouseEvent e) {
            isPressed = true;
            repaint();
        }

        @Override
        public void mouseReleased(MouseEvent e) {
            if (isPressed && isHovered) {
                fireActionEvent();
            }
            isPressed = false;
            repaint();
        }
    };

    addMouseListener(ma);
    addMouseMotionListener(ma);
}

public void addActionListener(ActionListener listener) {
    if (listener != null && !listeners.contains(listener)) {
        listeners.add(listener);
    }
}

private void fireActionEvent() {
    ActionEvent evt = new ActionEvent(this, ActionEvent.ACTION_PERFORMED, text);
    for (ActionListener l : listeners) {
        l.actionPerformed(evt);
    }
}

public void setButtonText(String text) {
    this.text = text;
    invalidate();
    repaint();
}

public String getButtonText() {
    return text;
}
```

```
    public void setButtonBg(Color color) {
        this.bgColor = color;
        this.hoverColor = brighten(color, 25);
        this.pressedColor = darken(color, 30);
        repaint();
    }

    public void setCornerRadius(int r) {
        this.cornerRadius = r;
        repaint();
    }

    @Override
    public void update(Graphics g) {
        paint(g);
    }

    @Override
    public void paint(Graphics g) {
        Graphics2D g2 = (Graphics2D) g;
                                        g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);
                                g2.setRenderingHint(RenderingHints.KEY_TEXT_ANTIALIASING,
RenderingHints.VALUE_TEXT_ANTIALIAS_ON);

        int w = getWidth();
        int h = getHeight();
        if (w <= 0 || h <= 0) return;

        // Determine active background color
        Color currentBg;
        if (isPressed) {
            currentBg = pressedColor;
        } else if (isHovered) {
            currentBg = hoverColor;
        } else {
            currentBg = bgColor;
        }

        // Draw shadow
        g2.setColor(new Color(0, 0, 0, 35));
        g2.fillRoundRect(1, 2, w - 2, h - 3, cornerRadius, cornerRadius);

        // Draw button shape
        g2.setColor(currentBg);
        g2.fillRoundRect(0, 0, w - 1, h - 3, cornerRadius, cornerRadius);

        // Top border highlight
        g2.setColor(new Color(255, 255, 255, 60));
        g2.drawRoundRect(1, 1, w - 3, (h / 2) - 1, cornerRadius, cornerRadius);
```

```
    // Subtle dark outline
    g2.setColor(new Color(0, 0, 0, 45));
    g2.drawRoundRect(0, 0, w - 1, h - 3, cornerRadius, cornerRadius);

    // Draw centered text
    g2.setColor(fgColor);
    g2.setFont(buttonFont);
    FontMetrics fm = g2.getFontMetrics(buttonFont);
    int strWidth = fm.stringWidth(text);
    int strHeight = fm.getAscent();
    int x = (w - strWidth) / 2;
    int y = ((h - 3) + strHeight) / 2 - 2;

    if (isPressed) {
        y += 1;
    }

    g2.drawString(text, x, y);
}

@Override
public Dimension getPreferredSize() {
    int textWidth = text != null ? text.length() * 9 : 60;
    try {
        FontMetrics fm = getFontMetrics(buttonFont);
        if (fm != null && text != null) {
            textWidth = fm.stringWidth(text);
        }
    } catch (Exception ignored) {}

    int w = Math.max(105, textWidth + 24);
    return new Dimension(w, 38);
}

@Override
public Dimension getMinimumSize() {
    return getPreferredSize();
}

private Color brighten(Color c, int amount) {
    int r = Math.min(255, c.getRed() + amount);
    int g = Math.min(255, c.getGreen() + amount);
    int b = Math.min(255, c.getBlue() + amount);
    return new Color(r, g, b);
}

private Color darken(Color c, int amount) {
    int r = Math.max(0, c.getRed() - amount);
    int g = Math.max(0, c.getGreen() - amount);
```

```
        int b = Math.max(0, c.getBlue() - amount);
        return new Color(r, g, b);
    }
}
```

DBConnection.java

```
package parking;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DBConnection {

    private static final String URL = "jdbc:mysql://localhost:3306/smart_campus_parking";

    private static final String USER = "root";

    private static final String PASSWORD = "2601";

    private static Boolean isConnected = null;

    public static Connection getConnection() {

        Connection con = null;

        try {

            Class.forName("com.mysql.cj.jdbc.Driver");

            con = DriverManager.getConnection(
                    URL,
                    USER,
                    PASSWORD);

            isConnected = true;

        } catch (ClassNotFoundException e) {

             System.out.println("MySQL JDBC Driver not found. Operating in Demo/In-Memory
Mode.");
            isConnected = false;

        } catch (SQLException e) {

            if (isConnected == null || isConnected) {
                System.out.println("Database Connection Failed (Offline). Operating in Demo/In-
Memory Mode.");
```

```
            }
            isConnected = false;
        }

        return con;
    }

    public static boolean isDatabaseAvailable() {
        if (isConnected != null) {
            return isConnected;
        }
        try (Connection con = getConnection()) {
            return con != null && !con.isClosed();
        } catch (Exception e) {
            return false;
        }
    }
}
```

ParkingAvailabilityDashboard.java

```
package parking;

import java.awt.*;
import java.awt.event.*;
import java.util.*;
import java.util.List;

public class ParkingAvailabilityDashboard extends Frame implements ActionListener {

    // ============================================================
    // COLOR THEME
    // ============================================================
    private final Color DARK_BLUE   = new Color(25, 55, 95);
    private final Color BLUE        = new Color(45, 95, 150);
    private final Color LIGHT_BG    = new Color(245, 247, 250);
    private final Color WHITE       = Color.WHITE;
    private final Color TEXT_DARK   = new Color(30, 41, 59);
    private final Color TEXT_MUTED  = new Color(100, 116, 139);

    // Status Colors
     private final Color COLOR_AVAILABLE_1 = new Color(34, 139, 34);    // Forest Green
(Phase 1)
     private final Color COLOR_AVAILABLE_2 = new Color(76, 175, 80);    // Bright Lime
Glow (Phase 2 - Blink)
    private final Color COLOR_OCCUPIED    = new Color(198, 40, 40);    // Crimson Red
    private final Color COLOR_RESERVED    = new Color(239, 108, 0);    // Amber Orange
    private final Color COLOR_MAINTENANCE = new Color(84, 110, 122);   // Slate Gray
```

```
// ====================================================
// UI COMPONENTS
// ====================================================
private Label totalSlotsVal;
private Label availableSlotsVal;
private Label occupiedSlotsVal;
private Label reservedSlotsVal;
private Label occupancyRateVal;
private Label statusNotificationLabel;

private Choice zoneFilterChoice;
private Choice vehicleFilterChoice;
private Choice statusFilterChoice;

private AwtButton btnRefresh;
private AwtButton btnToggleBlink;
private AwtButton btnClose;

private Panel gridContainer;
private ScrollPane scrollPane;

private ParkingSlotDAO slotDAO;
private List<ParkingSlot> currentSlots = new ArrayList<>();
private final Random random = new Random();

// Blinking Animation Engine
private volatile boolean isBlinkActive = true;
private volatile boolean blinkToggle = false;
private Thread blinkThread;
private final List<SlotVisualCard> renderedCards = new ArrayList<>();

// ====================================================
// CONSTRUCTOR
// ====================================================
public ParkingAvailabilityDashboard() {

    slotDAO = new ParkingSlotDAO();

    setTitle("Smart Campus Parking - Live Parking Space Availability & Visualizer");
    setSize(1120, 800);
    setLayout(new BorderLayout(0, 0));
    setBackground(LIGHT_BG);

    // ================================================
    // 1. HEADER PANEL
    // ================================================
    Panel headerPanel = new Panel(new BorderLayout(5, 5));
    headerPanel.setBackground(DARK_BLUE);

    Panel titleBox = new Panel(new GridLayout(2, 1));
```

```
        titleBox.setBackground(DARK_BLUE);

        Label title = new Label("LIVE CAMPUS PARKING AVAILABILITY & MONITOR",
Label.CENTER);
        title.setFont(new Font("Arial", Font.BOLD, 22));
        title.setForeground(WHITE);
        titleBox.add(title);

         Label subtitle = new Label("□ Real-Time Color-Coded Map | Pulsing Unoccupied
Slots | Zone Overview", Label.CENTER);
        subtitle.setFont(new Font("Arial", Font.PLAIN, 13));
        subtitle.setForeground(new Color(200, 220, 245));
        titleBox.add(subtitle);

        headerPanel.add(titleBox, BorderLayout.CENTER);
        add(headerPanel, BorderLayout.NORTH);

        // ================================================
        // 2. TOP METRICS BANNER
        // ================================================
        Panel topSection = new Panel(new BorderLayout(5, 5));
        topSection.setBackground(LIGHT_BG);

        Panel metricsPanel = new Panel(new GridLayout(1, 5, 10, 10));
        metricsPanel.setBackground(LIGHT_BG);

        totalSlotsVal = new Label("0", Label.CENTER);
        availableSlotsVal = new Label("0", Label.CENTER);
        occupiedSlotsVal = new Label("0", Label.CENTER);
        reservedSlotsVal = new Label("0", Label.CENTER);
        occupancyRateVal = new Label("0%", Label.CENTER);

              metricsPanel.add(createMetricCard("TOTAL CAPACITY", totalSlotsVal, new
Color(51, 65, 85), WHITE));
            metricsPanel.add(createMetricCard("□ AVAILABLE (FREE)", availableSlotsVal,
COLOR_AVAILABLE_1, WHITE));
                    metricsPanel.add(createMetricCard("□ OCCUPIED", occupiedSlotsVal,
COLOR_OCCUPIED, WHITE));
                    metricsPanel.add(createMetricCard("□ RESERVED", reservedSlotsVal,
COLOR_RESERVED, WHITE));
          metricsPanel.add(createMetricCard("□ OCCUPANCY RATE", occupancyRateVal,
BLUE, WHITE));

        topSection.add(metricsPanel, BorderLayout.NORTH);

        // ================================================
        // 3. FILTER & CONTROLS BAR
        // ================================================
        Panel controlBar = new Panel(new FlowLayout(FlowLayout.LEFT, 10, 8));
        controlBar.setBackground(new Color(230, 238, 248));
```

```
        Label lblZone = new Label("Zone:");
        lblZone.setFont(new Font("Arial", Font.BOLD, 13));
        controlBar.add(lblZone);

        zoneFilterChoice = new Choice();
        zoneFilterChoice.add("All Zones");
        zoneFilterChoice.add("Zone 1 - Main Gate");
        zoneFilterChoice.add("Zone 2 - Science Block");
        zoneFilterChoice.add("Zone 3 - Admin Block");
        zoneFilterChoice.add("Zone 4 - Academic Block");
        zoneFilterChoice.setFont(new Font("Arial", Font.PLAIN, 12));
        controlBar.add(zoneFilterChoice);

        Label lblType = new Label("Vehicle:");
        lblType.setFont(new Font("Arial", Font.BOLD, 13));
        controlBar.add(lblType);

        vehicleFilterChoice = new Choice();
        vehicleFilterChoice.add("All Vehicle Types");
        vehicleFilterChoice.add("Four Wheeler");
        vehicleFilterChoice.add("Two Wheeler");
        vehicleFilterChoice.add("EV");
        vehicleFilterChoice.setFont(new Font("Arial", Font.PLAIN, 12));
        controlBar.add(vehicleFilterChoice);

        Label lblStatus = new Label("Status:");
        lblStatus.setFont(new Font("Arial", Font.BOLD, 13));
        controlBar.add(lblStatus);

        statusFilterChoice = new Choice();
        statusFilterChoice.add("All Statuses");
        statusFilterChoice.add("Available");
        statusFilterChoice.add("Occupied");
        statusFilterChoice.add("Reserved");
        statusFilterChoice.add("Maintenance");
        statusFilterChoice.setFont(new Font("Arial", Font.PLAIN, 12));
        controlBar.add(statusFilterChoice);

        btnRefresh = new AwtButton("□ Refresh", BLUE, WHITE);
        btnToggleBlink = new AwtButton("□  Blink: ON", new Color(40, 140, 85), WHITE);
        btnClose = new AwtButton("□  Close", new Color(75, 85, 99), WHITE);

        controlBar.add(btnRefresh);
        controlBar.add(btnToggleBlink);
        controlBar.add(btnClose);

        topSection.add(controlBar, BorderLayout.CENTER);

        // Notification bar
```

```
        statusNotificationLabel = new Label("  Unoccupied / Available parking slots are flashing green for high visibility. Click any slot to manage.", Label.CENTER);
        statusNotificationLabel.setFont(new Font("Arial", Font.ITALIC, 12));
        statusNotificationLabel.setForeground(TEXT_DARK);
        statusNotificationLabel.setBackground(new Color(241, 245, 249));
        topSection.add(statusNotificationLabel, BorderLayout.SOUTH);

        add(topSection, BorderLayout.NORTH);

        // ================================================
        // 4. MAIN PARKING GRID AREA
        // ================================================
        scrollPane = new ScrollPane(ScrollPane.SCROLLBARS_AS_NEEDED);
        gridContainer = new Panel();
        gridContainer.setBackground(LIGHT_BG);
        scrollPane.add(gridContainer);
        add(scrollPane, BorderLayout.CENTER);

        // ================================================
        // 5. FOOTER LEGEND
        // ================================================
        Panel footerPanel = new Panel(new FlowLayout(FlowLayout.CENTER, 20, 8));
        footerPanel.setBackground(new Color(30, 41, 59));

                                footerPanel.add(createLegendItem("Available       (Blinking/Pulsing)", COLOR_AVAILABLE_1));
        footerPanel.add(createLegendItem("Occupied", COLOR_OCCUPIED));
        footerPanel.add(createLegendItem("Reserved", COLOR_RESERVED));
        footerPanel.add(createLegendItem("Maintenance", COLOR_MAINTENANCE));

         Label dbStatus = new Label(DBConnection.isDatabaseAvailable() ? " [DB Online] " : " [Demo / Local Mode] ");
        dbStatus.setForeground(Color.CYAN);
        dbStatus.setFont(new Font("Arial", Font.BOLD, 11));
        footerPanel.add(dbStatus);

        add(footerPanel, BorderLayout.SOUTH);

        // ================================================
        // EVENT LISTENERS
        // ================================================
        btnRefresh.addActionListener(this);
        btnToggleBlink.addActionListener(this);
        btnClose.addActionListener(this);

        zoneFilterChoice.addItemListener(e -> refreshDashboard());
        vehicleFilterChoice.addItemListener(e -> refreshDashboard());
        statusFilterChoice.addItemListener(e -> refreshDashboard());

        addWindowListener(new WindowAdapter() {
```

```
        @Override
        public void windowClosing(WindowEvent e) {
            stopBlinkThread();
            dispose();
        }
    });

    // Initialize & render
    refreshDashboard();
    startBlinkThread();

    setLocationRelativeTo(null);
    setVisible(true);
}

// ========================================================
// BLINK ANIMATION THREAD
// ========================================================
private void startBlinkThread() {
    if (blinkThread != null && blinkThread.isAlive()) return;

    blinkThread = new Thread(() -> {
        while (isBlinkActive) {
            try {
                Thread.sleep(550);
                blinkToggle = !blinkToggle;

                // Repaint all available visual cards
                synchronized (renderedCards) {
                    for (SlotVisualCard card : renderedCards) {
                        if ("Available".equalsIgnoreCase(card.getSlot().getSlotStatus())) {
                            card.setBlinkState(blinkToggle);
                        }
                    }
                }
            } catch (InterruptedException ex) {
                break;
            }
        }
    });
    blinkThread.setDaemon(true);
    blinkThread.start();
}

private void stopBlinkThread() {
    isBlinkActive = false;
    if (blinkThread != null) {
        blinkThread.interrupt();
    }
}
```

```
    // ========================================================
    // HELPER: CREATE METRIC CARD
    // ========================================================
     private Panel createMetricCard(String labelText, Label valueLabel, Color bgColor, Color
fgColor) {
        Panel card = new Panel(new BorderLayout(2, 2));
        card.setBackground(bgColor);

        Label lbl = new Label(labelText, Label.CENTER);
        lbl.setFont(new Font("Arial", Font.BOLD, 11));
        lbl.setForeground(new Color(240, 240, 240));

        valueLabel.setFont(new Font("Arial", Font.BOLD, 22));
        valueLabel.setForeground(fgColor);

        card.add(lbl, BorderLayout.NORTH);
        card.add(valueLabel, BorderLayout.CENTER);
        return card;
    }

    // ========================================================
    // HELPER: CREATE LEGEND ITEM
    // ========================================================
    private Panel createLegendItem(String text, Color color) {
        Panel item = new Panel(new FlowLayout(FlowLayout.CENTER, 5, 0));
        item.setBackground(new Color(30, 41, 59));

        Canvas badge = new Canvas() {
            @Override
            public void paint(Graphics g) {
                g.setColor(color);
                g.fillRoundRect(0, 2, 14, 14, 4, 4);
            }
        };
        badge.setSize(14, 18);
        item.add(badge);

        Label lbl = new Label(text);
        lbl.setForeground(WHITE);
        lbl.setFont(new Font("Arial", Font.PLAIN, 12));
        item.add(lbl);

        return item;
    }

    // ========================================================
    // REFRESH DASHBOARD & REBUILD GRID
    // ========================================================
    public void refreshDashboard() {
```

```
        currentSlots = slotDAO.getAllSlots();

        // Calculate metrics
        int total = currentSlots.size();
        int available = 0;
        int occupied = 0;
        int reserved = 0;
        int maintenance = 0;

        for (ParkingSlot s : currentSlots) {
            String status = s.getSlotStatus();
            if ("Available".equalsIgnoreCase(status)) {
                available++;
            } else if ("Occupied".equalsIgnoreCase(status)) {
                occupied++;
            } else if ("Reserved".equalsIgnoreCase(status)) {
                reserved++;
            } else if ("Maintenance".equalsIgnoreCase(status)) {
                maintenance++;
            }
        }

        int occupancyRate = total > 0 ? (int) Math.round(((double) occupied / total) * 100.0) : 0;

        totalSlotsVal.setText(String.valueOf(total));
        availableSlotsVal.setText(String.valueOf(available));
        occupiedSlotsVal.setText(String.valueOf(occupied));
        reservedSlotsVal.setText(String.valueOf(reserved));
        occupancyRateVal.setText(occupancyRate + "%");

        // Filter slots
        String selectedZone = zoneFilterChoice.getSelectedItem();
        String selectedVehicle = vehicleFilterChoice.getSelectedItem();
        String selectedStatus = statusFilterChoice.getSelectedItem();

        List<ParkingSlot> filtered = new ArrayList<>();
        for (ParkingSlot s : currentSlots) {
            boolean matchZone = true;
            if (selectedZone != null && !selectedZone.startsWith("All")) {
                if (selectedZone.contains("Zone 1") && s.getZoneId() != 1) matchZone = false;
                    else if (selectedZone.contains("Zone 2") && s.getZoneId() != 2) matchZone =
false;
                    else if (selectedZone.contains("Zone 3") && s.getZoneId() != 3) matchZone =
false;
                    else if (selectedZone.contains("Zone 4") && s.getZoneId() != 4) matchZone =
false;
            }

            boolean matchVehicle = true;
            if (selectedVehicle != null && !selectedVehicle.startsWith("All")) {
```

```
            if (!s.getVehicleType().equalsIgnoreCase(selectedVehicle)) matchVehicle = false;
        }

        boolean matchStatus = true;
        if (selectedStatus != null && !selectedStatus.startsWith("All")) {
            if (!s.getSlotStatus().equalsIgnoreCase(selectedStatus)) matchStatus = false;
        }

        if (matchZone && matchVehicle && matchStatus) {
            filtered.add(s);
        }
    }

    renderGrid(filtered);
}

// ====================================================
// RENDER PARKING SLOT CARDS
// ====================================================
private void renderGrid(List<ParkingSlot> slots) {
    gridContainer.removeAll();

    synchronized (renderedCards) {
        renderedCards.clear();
    }

    int count = slots.size();
    int cols = 4;
    int rows = (int) Math.ceil((double) count / cols);
    if (rows < 2) rows = 2;

    gridContainer.setLayout(new GridLayout(rows, cols, 15, 15));

    for (ParkingSlot slot : slots) {
        SlotVisualCard card = new SlotVisualCard(slot);
        synchronized (renderedCards) {
            renderedCards.add(card);
        }
        gridContainer.add(card);
    }

    // Fill blanks
    int totalCells = rows * cols;
    for (int i = count; i < totalCells; i++) {
        Panel blank = new Panel();
        blank.setBackground(LIGHT_BG);
        gridContainer.add(blank);
    }

    gridContainer.validate();
```

```
        gridContainer.repaint();
        scrollPane.validate();
    }

    // ================================================
    // CUSTOM VISUAL CARD WITH BLINKING SUPPORT
    // ================================================
    private class SlotVisualCard extends Panel {
        private final ParkingSlot slot;
        private final Panel cardHeader;
        private final Label lblSlotNum;
        private final Label lblStatus;
        private boolean isAvailable;

        public SlotVisualCard(ParkingSlot slot) {
            super(new BorderLayout(4, 4));
            this.slot = slot;
            this.isAvailable = "Available".equalsIgnoreCase(slot.getSlotStatus());

            setBackground(WHITE);

            Color headerColor = getStatusColor(slot.getSlotStatus());

            // Header
            cardHeader = new Panel(new BorderLayout());
            cardHeader.setBackground(headerColor);

            lblSlotNum = new Label(" SLOT " + slot.getSlotNumber() + " ", Label.CENTER);
            lblSlotNum.setFont(new Font("Arial", Font.BOLD, 15));
            lblSlotNum.setForeground(WHITE);
            cardHeader.add(lblSlotNum, BorderLayout.CENTER);
            add(cardHeader, BorderLayout.NORTH);

            // Body
            Panel cardBody = new Panel(new GridLayout(3, 1));
            cardBody.setBackground(WHITE);

            String zoneName = "Zone " + slot.getZoneId();
            if (slot.getZoneId() == 1) zoneName = "Main Gate (Z1)";
            else if (slot.getZoneId() == 2) zoneName = "Science Blk (Z2)";
            else if (slot.getZoneId() == 3) zoneName = "Admin Blk (Z3)";
            else if (slot.getZoneId() == 4) zoneName = "Academic (Z4)";

            Label lblZone = new Label("□ " + zoneName, Label.CENTER);
            lblZone.setFont(new Font("Arial", Font.PLAIN, 12));
            lblZone.setForeground(TEXT_DARK);

            String vehicleIcon = "□";
            if ("Two Wheeler".equalsIgnoreCase(slot.getVehicleType())) vehicleIcon = "□";
            else if ("EV".equalsIgnoreCase(slot.getVehicleType())) vehicleIcon = "□ ";
```

```
            Label lblType = new Label(vehicleIcon + " " + slot.getVehicleType(),
Label.CENTER);
        lblType.setFont(new Font("Arial", Font.BOLD, 12));
        lblType.setForeground(BLUE);

        lblStatus = new Label("● " + slot.getSlotStatus().toUpperCase(), Label.CENTER);
        lblStatus.setFont(new Font("Arial", Font.BOLD, 12));
        lblStatus.setForeground(headerColor);

        cardBody.add(lblZone);
        cardBody.add(lblType);
        cardBody.add(lblStatus);
        add(cardBody, BorderLayout.CENTER);

        // Action Button
         AwtButton btnAction = new AwtButton("Manage Slot", new Color(241, 245, 249),
TEXT_DARK, new Font("Arial", Font.BOLD, 11));
        btnAction.addActionListener(e -> openSlotActionDialog(slot));
        add(btnAction, BorderLayout.SOUTH);
    }

    public ParkingSlot getSlot() {
        return slot;
    }

    public void setBlinkState(boolean state) {
        if (!isAvailable) return;
        if (state) {
            cardHeader.setBackground(COLOR_AVAILABLE_2);
            lblSlotNum.setText(" □  SLOT "+ slot.getSlotNumber() + " □  ");
            lblStatus.setForeground(COLOR_AVAILABLE_2);
            lblStatus.setText("□ OPEN / VACANT");
        } else {
            cardHeader.setBackground(COLOR_AVAILABLE_1);
            lblSlotNum.setText(" SLOT " + slot.getSlotNumber() + " ");
            lblStatus.setForeground(COLOR_AVAILABLE_1);
            lblStatus.setText("● AVAILABLE");
        }
        cardHeader.repaint();
        lblStatus.repaint();
    }

    private Color getStatusColor(String status) {
        if ("Available".equalsIgnoreCase(status)) return COLOR_AVAILABLE_1;
        if ("Occupied".equalsIgnoreCase(status)) return COLOR_OCCUPIED;
        if ("Reserved".equalsIgnoreCase(status)) return COLOR_RESERVED;
        return COLOR_MAINTENANCE;
    }
  }
```

```
    // ========================================================
    // SLOT ACTION DIALOG (Pure AWT)
    // ========================================================
    private void openSlotActionDialog(ParkingSlot slot) {
        Dialog dialog = new Dialog(this, "Manage Parking Slot: " + slot.getSlotNumber(), true);
        dialog.setSize(440, 340);
        dialog.setLayout(new BorderLayout(10, 10));
        dialog.setBackground(LIGHT_BG);

        // Header
        Panel dHeader = new Panel(new BorderLayout());
        dHeader.setBackground(DARK_BLUE);
            Label dTitle = new Label("SLOT " + slot.getSlotNumber() + " DETAILS &
ACTIONS", Label.CENTER);
        dTitle.setFont(new Font("Arial", Font.BOLD, 14));
        dTitle.setForeground(WHITE);
        dHeader.add(dTitle, BorderLayout.CENTER);
        dialog.add(dHeader, BorderLayout.NORTH);

        // Info Body
        Panel infoPanel = new Panel(new GridLayout(4, 2, 5, 8));
        infoPanel.setBackground(LIGHT_BG);

        infoPanel.add(new Label("Slot ID:"));
        infoPanel.add(new Label(String.valueOf(slot.getSlotId())));

        infoPanel.add(new Label("Zone:"));
        infoPanel.add(new Label("Zone " + slot.getZoneId()));

        infoPanel.add(new Label("Vehicle Allowed:"));
        infoPanel.add(new Label(slot.getVehicleType()));

        infoPanel.add(new Label("Current Status:"));
        Label curStatus = new Label(slot.getSlotStatus());
        curStatus.setFont(new Font("Arial", Font.BOLD, 13));
                                    if          ("Available".equalsIgnoreCase(slot.getSlotStatus()))
curStatus.setForeground(COLOR_AVAILABLE_1);
                                else        if        ("Occupied".equalsIgnoreCase(slot.getSlotStatus()))
curStatus.setForeground(COLOR_OCCUPIED);
        else curStatus.setForeground(COLOR_RESERVED);
        infoPanel.add(curStatus);

        dialog.add(infoPanel, BorderLayout.CENTER);

        // Action Buttons
        Panel btnGroup = new Panel(new GridLayout(2, 2, 8, 8));
        btnGroup.setBackground(LIGHT_BG);
```

```
                AwtButton btnAvailable = new AwtButton("□ Mark Available",
COLOR_AVAILABLE_1, WHITE);
                AwtButton btnOccupied = new AwtButton("□ Park (Occupied)",
COLOR_OCCUPIED, WHITE);
        AwtButton btnReserved = new AwtButton("□ Reserve Slot", COLOR_RESERVED,
WHITE);
        AwtButton btnCloseDialog = new AwtButton("Cancel", new Color(100, 116, 139),
WHITE);

        btnAvailable.addActionListener(e -> {
            slotDAO.updateSlotStatus(slot.getSlotId(), "Available");
                statusNotificationLabel.setText("□  Slot " + slot.getSlotNumber() + " marked as
AVAILABLE (Blinking).");
            dialog.dispose();
            refreshDashboard();
        });

        btnOccupied.addActionListener(e -> {
            slotDAO.updateSlotStatus(slot.getSlotId(), "Occupied");
                statusNotificationLabel.setText("□ Slot " + slot.getSlotNumber() + " marked as
OCCUPIED (Vehicle Parked).");
            dialog.dispose();
            refreshDashboard();
        });

        btnReserved.addActionListener(e -> {
            slotDAO.updateSlotStatus(slot.getSlotId(), "Reserved");
                statusNotificationLabel.setText("□ Slot " + slot.getSlotNumber() + " marked as
RESERVED.");
            dialog.dispose();
            refreshDashboard();
        });

        btnCloseDialog.addActionListener(e -> dialog.dispose());

        btnGroup.add(btnAvailable);
        btnGroup.add(btnOccupied);
        btnGroup.add(btnReserved);
        btnGroup.add(btnCloseDialog);

        dialog.add(btnGroup, BorderLayout.SOUTH);

        dialog.addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                dialog.dispose();
            }
        });

        dialog.setLocationRelativeTo(this);
```

```
        dialog.setVisible(true);
    }

    // ====================================================
    // ACTION PERFORMED
    // ====================================================
    @Override
    public void actionPerformed(ActionEvent e) {
        Object src = e.getSource();
        if (src == btnRefresh) {
            statusNotificationLabel.setText("□ Slot map refreshed successfully.");
            refreshDashboard();
        } else if (src == btnToggleBlink) {
            isBlinkActive = !isBlinkActive;
            if (isBlinkActive) {
                btnToggleBlink.setButtonText("□  Blink: ON");
                btnToggleBlink.setButtonBg(new Color(40, 140, 85));
                startBlinkThread();
                statusNotificationLabel.setText("□  Available slots blinking enabled.");
            } else {
                btnToggleBlink.setButtonText("□  Blink: OFF");
                btnToggleBlink.setButtonBg(new Color(100, 116, 139));
                statusNotificationLabel.setText("Available slots blinking paused.");
            }
        } else if (src == btnClose) {
            stopBlinkThread();
            dispose();
        }
    }

    public static void main(String[] args) {
        new ParkingAvailabilityDashboard();
    }
}
```

ParkingPass.java

```
package parking;

import java.sql.Date;

public class ParkingPass {

    private int passId;
    private int userId;
    private int vehicleId;
    private String passType;
    private Date startDate;
    private Date endDate;
```

```
    private String passStatus;
    private double passAmount;

    public ParkingPass() {
    }

    public ParkingPass(int passId, int userId, int vehicleId,
                       String passType, Date startDate, Date endDate,
                       String passStatus, double passAmount) {

        this.passId = passId;
        this.userId = userId;
        this.vehicleId = vehicleId;
        this.passType = passType;
        this.startDate = startDate;
        this.endDate = endDate;
        this.passStatus = passStatus;
        this.passAmount = passAmount;
    }

    public int getPassId() {
        return passId;
    }

    public void setPassId(int passId) {
        this.passId = passId;
    }

    public int getUserId() {
        return userId;
    }

    public void setUserId(int userId) {
        this.userId = userId;
    }

    public int getVehicleId() {
        return vehicleId;
    }

    public void setVehicleId(int vehicleId) {
        this.vehicleId = vehicleId;
    }

    public String getPassType() {
        return passType;
    }

    public void setPassType(String passType) {
        this.passType = passType;
```

```
    }

    public Date getStartDate() {
        return startDate;
    }

    public void setStartDate(Date startDate) {
        this.startDate = startDate;
    }

    public Date getEndDate() {
        return endDate;
    }

    public void setEndDate(Date endDate) {
        this.endDate = endDate;
    }

    public String getPassStatus() {
        return passStatus;
    }

    public void setPassStatus(String passStatus) {
        this.passStatus = passStatus;
    }

    public double getPassAmount() {
        return passAmount;
    }

    public void setPassAmount(double passAmount) {
        this.passAmount = passAmount;
    }
}
```

ParkingPassDAO.java

```
package parking;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Date;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.CopyOnWriteArrayList;

public class ParkingPassDAO {
```

```
    private static final List<ParkingPass> demoPasses = new CopyOnWriteArrayList<>();

    static {
        initDemoPasses();
    }

    private static void initDemoPasses() {
        if (!demoPasses.isEmpty()) return;
        long now = System.currentTimeMillis();
        long thirtyDays = 30L * 24 * 60 * 60 * 1000;
        demoPasses.add(new ParkingPass(1, 101, 1, "Monthly", new Date(now), new Date(now
+ thirtyDays), "Active", 500.0));
        demoPasses.add(new ParkingPass(2, 102, 2, "Semester", new Date(now), new Date(now
+ (thirtyDays * 4)), "Active", 1800.0));
    }

    // ADD PARKING PASS
    public boolean addPass(ParkingPass pass) {

        String sql = "INSERT INTO parking_passes "
                   + "(user_id, vehicle_id, pass_type, start_date, "
                   + "end_date, pass_status, pass_amount) "
                   + "VALUES (?, ?, ?, ?, ?, ?, ?)";

        Connection con = DBConnection.getConnection();
        if (con != null) {
            try (con; PreparedStatement ps = con.prepareStatement(sql)) {

                ps.setInt(1, pass.getUserId());
                ps.setInt(2, pass.getVehicleId());
                ps.setString(3, pass.getPassType());
                ps.setDate(4, pass.getStartDate());
                ps.setDate(5, pass.getEndDate());
                ps.setString(6, pass.getPassStatus());
                ps.setDouble(7, pass.getPassAmount());

                int rows = ps.executeUpdate();
                return rows > 0;

            } catch (SQLException e) {
                System.out.println("Error adding parking pass in DB: " + e.getMessage());
            }
        }

        if (pass.getPassId() == 0) {
            pass.setPassId(demoPasses.size() + 1);
        }
        demoPasses.add(pass);
        return true;
    }
```

```java
// SEARCH PARKING PASS BY ID
public ParkingPass searchPass(int passId) {

    String sql = "SELECT * FROM parking_passes WHERE pass_id = ?";

    Connection con = DBConnection.getConnection();
    if (con != null) {
        try (con; PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, passId);
            ResultSet rs = ps.executeQuery();

            if (rs.next()) {
                ParkingPass pass = new ParkingPass();
                pass.setPassId(rs.getInt("pass_id"));
                pass.setUserId(rs.getInt("user_id"));
                pass.setVehicleId(rs.getInt("vehicle_id"));
                pass.setPassType(rs.getString("pass_type"));
                pass.setStartDate(rs.getDate("start_date"));
                pass.setEndDate(rs.getDate("end_date"));
                pass.setPassStatus(rs.getString("pass_status"));
                pass.setPassAmount(rs.getDouble("pass_amount"));
                return pass;
            }

        } catch (SQLException e) {
            System.out.println("Error searching parking pass in DB: " + e.getMessage());
        }
    }

    for (ParkingPass p : demoPasses) {
        if (p.getPassId() == passId) return p;
    }

    return null;
}

// GET ALL PASSES
public List<ParkingPass> getAllPasses() {

    List<ParkingPass> list = new ArrayList<>();
    String sql = "SELECT * FROM parking_passes ORDER BY pass_id";

    Connection con = DBConnection.getConnection();
    if (con != null) {
        try (con;
             PreparedStatement ps = con.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {
```

```
            while (rs.next()) {
                ParkingPass pass = new ParkingPass();
                pass.setPassId(rs.getInt("pass_id"));
                pass.setUserId(rs.getInt("user_id"));
                pass.setVehicleId(rs.getInt("vehicle_id"));
                pass.setPassType(rs.getString("pass_type"));
                pass.setStartDate(rs.getDate("start_date"));
                pass.setEndDate(rs.getDate("end_date"));
                pass.setPassStatus(rs.getString("pass_status"));
                pass.setPassAmount(rs.getDouble("pass_amount"));
                list.add(pass);
            }

            if (!list.isEmpty()) {
                return list;
            }

        } catch (SQLException e) {
            System.out.println("Error retrieving parking passes from DB: " + e.getMessage());
        }
    }

    return new ArrayList<>(demoPasses);
  }
}
```

ParkingPassManagement.java

```
package parking;

import java.awt.*;
import java.awt.event.*;
import java.sql.Date;

public class ParkingPassManagement extends Frame implements ActionListener {

  private TextField passIdField;
  private TextField userIdField;
  private TextField vehicleIdField;
  private TextField passTypeField;
  private TextField startDateField;
  private TextField endDateField;
  private TextField passAmountField;

  private Choice passStatusChoice;

  private Button addButton;
  private Button searchButton;
  private Button clearButton;
```

```
private Button closeButton;

private TextArea resultArea;

// Colors
private final Color HEADER_COLOR = new Color(35, 55, 85);
private final Color PANEL_COLOR = new Color(245, 247, 250);
private final Color BUTTON_COLOR = new Color(55, 90, 130);
private final Color SEARCH_COLOR = new Color(45, 120, 90);
private final Color CLEAR_COLOR = new Color(210, 145, 45);
private final Color CLOSE_COLOR = new Color(180, 65, 65);

public ParkingPassManagement() {

    setTitle("Smart Campus Parking - Parking Pass Management");

    setSize(900, 650);

    setLayout(new BorderLayout(15, 15));

    setBackground(Color.WHITE);

    // ========================================================
    // HEADER
    // ========================================================

    Panel headerPanel = new Panel(new BorderLayout());

    headerPanel.setBackground(HEADER_COLOR);

    headerPanel.setPreferredSize(new Dimension(900, 80));

    Label title = new Label(
            "PARKING PASS MANAGEMENT",
            Label.CENTER
    );

    title.setFont(
            new Font("Arial", Font.BOLD, 24)
    );

    title.setForeground(Color.WHITE);

    headerPanel.add(
            title,
            BorderLayout.CENTER
    );

    Label subtitle = new Label(
            "Manage vehicle parking passes",
```

```
        Label.CENTER
    );

    subtitle.setFont(
        new Font("Arial", Font.PLAIN, 13)
    );

    subtitle.setForeground(Color.WHITE);

    headerPanel.add(
        subtitle,
        BorderLayout.SOUTH
    );

    add(
        headerPanel,
        BorderLayout.NORTH
    );

    // ========================================================
    // MAIN CONTENT
    // ========================================================

    Panel mainPanel = new Panel(
        new GridLayout(1, 2, 15, 0)
    );

    mainPanel.setBackground(Color.WHITE);

    // ========================================================
    // LEFT - FORM PANEL
    // ========================================================

    Panel formContainer = new Panel(
        new BorderLayout(10, 10)
    );

    formContainer.setBackground(PANEL_COLOR);

    Label formTitle = new Label(
        "PASS DETAILS",
        Label.CENTER
    );

    formTitle.setFont(
        new Font("Arial", Font.BOLD, 16)
    );

    formContainer.add(
        formTitle,
```

```
        BorderLayout.NORTH
);

Panel formPanel = new Panel();

formPanel.setLayout(
        new GridLayout(8, 2, 10, 12)
);

formPanel.setBackground(PANEL_COLOR);

// ---------------------------------------------------
// PASS ID
// ---------------------------------------------------

formPanel.add(
        createLabel("Pass ID:")
);

passIdField = new TextField();

passIdField.setFont(
        new Font("Arial", Font.PLAIN, 14)
);

// Pass ID is mainly used for SEARCH
passIdField.setEditable(true);

formPanel.add(passIdField);

// ---------------------------------------------------
// USER ID
// ---------------------------------------------------

formPanel.add(
        createLabel("User ID:")
);

userIdField = createTextField();

formPanel.add(userIdField);

// ---------------------------------------------------
// VEHICLE ID
// ---------------------------------------------------

formPanel.add(
        createLabel("Vehicle ID:")
);
```

```
        vehicleIdField = createTextField();

        formPanel.add(vehicleIdField);

        // ------------------------------------------------
        // PASS TYPE
        // ------------------------------------------------

        formPanel.add(
                createLabel("Pass Type:")
        );

        passTypeField = createTextField();

        formPanel.add(passTypeField);

        // ------------------------------------------------
        // START DATE
        // ------------------------------------------------

        formPanel.add(
                createLabel("Start Date:")
        );

        startDateField = createTextField();

        startDateField.setText("YYYY-MM-DD");

        formPanel.add(startDateField);

        // ------------------------------------------------
        // END DATE
        // ------------------------------------------------

        formPanel.add(
                createLabel("End Date:")
        );

        endDateField = createTextField();

        endDateField.setText("YYYY-MM-DD");

        formPanel.add(endDateField);

        // ------------------------------------------------
        // STATUS
        // ------------------------------------------------

        formPanel.add(
                createLabel("Pass Status:")
```

```
);

passStatusChoice = new Choice();

passStatusChoice.add("Active");
passStatusChoice.add("Expired");
passStatusChoice.add("Cancelled");

passStatusChoice.setFont(
        new Font("Arial", Font.PLAIN, 14)
);

formPanel.add(passStatusChoice);

// ------------------------------------------------
// PASS AMOUNT
// ------------------------------------------------

formPanel.add(
        createLabel("Pass Amount:")
);

passAmountField = createTextField();

passAmountField.setText("0.00");

formPanel.add(passAmountField);

formContainer.add(
        formPanel,
        BorderLayout.CENTER
);

mainPanel.add(formContainer);

// ================================================
// RIGHT - RESULT PANEL
// ================================================

Panel resultContainer = new Panel(
        new BorderLayout(10, 10)
);

resultContainer.setBackground(PANEL_COLOR);

Label resultTitle = new Label(
        "RESULT / INFORMATION",
        Label.CENTER
);
```

```
        resultTitle.setFont(
                new Font("Arial", Font.BOLD, 16)
        );

        resultContainer.add(
                resultTitle,
                BorderLayout.NORTH
        );

        resultArea = new TextArea();

        resultArea.setFont(
                new Font("Monospaced", Font.PLAIN, 13)
        );

        resultArea.setEditable(false);

        resultArea.setBackground(Color.WHITE);

        resultContainer.add(
                resultArea,
                BorderLayout.CENTER
        );

        mainPanel.add(resultContainer);

        add(
                mainPanel,
                BorderLayout.CENTER
        );

        // ========================================================
        // BOTTOM PANEL
        // ========================================================

        Panel bottomPanel = new Panel(
                new BorderLayout(10, 10)
        );

        bottomPanel.setBackground(Color.WHITE);

        // --------------------------------------------------
        // BUTTON TITLE
        // --------------------------------------------------

        Label actionLabel = new Label(
                "ACTIONS",
                Label.CENTER
        );
```

```
        actionLabel.setFont(
                new Font("Arial", Font.BOLD, 14)
        );

        bottomPanel.add(
                actionLabel,
                BorderLayout.NORTH
        );

        // ---------------------------------------------------
        // BUTTON PANEL
        // ---------------------------------------------------

        Panel buttonPanel = new Panel(
                new GridLayout(1, 4, 15, 10)
        );

        addButton = new Button("ADD PASS");

        searchButton = new Button("SEARCH");

        clearButton = new Button("CLEAR");

        closeButton = new Button("CLOSE");

        styleButton(
                addButton,
                BUTTON_COLOR
        );

        styleButton(
                searchButton,
                SEARCH_COLOR
        );

        styleButton(
                clearButton,
                CLEAR_COLOR
        );

        styleButton(
                closeButton,
                CLOSE_COLOR
        );

        buttonPanel.add(addButton);
        buttonPanel.add(searchButton);
        buttonPanel.add(clearButton);
        buttonPanel.add(closeButton);
```

```
        bottomPanel.add(
                buttonPanel,
                BorderLayout.CENTER
        );

        add(
                bottomPanel,
                BorderLayout.SOUTH
        );

        // ========================================================
        // EVENT LISTENERS
        // ========================================================

        addButton.addActionListener(this);

        searchButton.addActionListener(this);

        clearButton.addActionListener(this);

        closeButton.addActionListener(this);

        // ========================================================
        // WINDOW CLOSE
        // ========================================================

        addWindowListener(
                new WindowAdapter() {

                    @Override
                    public void windowClosing(WindowEvent e) {

                        confirmClose();

                    }
                }
        );

        // ========================================================
        // WINDOW SETTINGS
        // ========================================================

        setLocationRelativeTo(null);

        setResizable(false);

        setVisible(true);
    }

    // ============================================================
```

```java
// CREATE LABEL
// ============================================================

private Label createLabel(String text) {

    Label label = new Label(text);

    label.setFont(
            new Font("Arial", Font.BOLD, 13)
    );

    return label;
}

// ============================================================
// CREATE TEXT FIELD
// ============================================================

private TextField createTextField() {

    TextField field = new TextField();

    field.setFont(
            new Font("Arial", Font.PLAIN, 14)
    );

    return field;
}

// ============================================================
// STYLE BUTTON
// ============================================================

private void styleButton(
        Button button,
        Color color
) {

    button.setFont(
            new Font("Arial", Font.BOLD, 13)
    );

    button.setBackground(color);

    button.setForeground(Color.WHITE);
}

// ============================================================
// BUTTON EVENTS
// ============================================================
```

```java
@Override
public void actionPerformed(ActionEvent e) {

    if (e.getSource() == addButton) {

        addPass();

    } else if (e.getSource() == searchButton) {

        searchPass();

    } else if (e.getSource() == clearButton) {

        clearFields();

    } else if (e.getSource() == closeButton) {

        confirmClose();
    }
}

// ============================================================
// ADD PASS
// ============================================================

private void addPass() {

    try {

        String userText =
                userIdField.getText().trim();

        String vehicleText =
                vehicleIdField.getText().trim();

        String passType =
                passTypeField.getText().trim();

        String startText =
                startDateField.getText().trim();

        String endText =
                endDateField.getText().trim();

        String amountText =
                passAmountField.getText().trim();

        if (userText.isEmpty()
                || vehicleText.isEmpty()
```

```
            || passType.isEmpty()
            || startText.isEmpty()
            || endText.isEmpty()
            || amountText.isEmpty()) {

        showMessage(
                "Please fill all mandatory fields."
        );

        return;
    }

    int userId =
            Integer.parseInt(userText);

    int vehicleId =
            Integer.parseInt(vehicleText);

    Date startDate =
            Date.valueOf(startText);

    Date endDate =
            Date.valueOf(endText);

    double passAmount =
            Double.parseDouble(amountText);

    if (passAmount < 0) {

        showMessage(
                "Pass amount cannot be negative."
        );

        return;
    }

    if (endDate.before(startDate)) {

        showMessage(
                "End date cannot be before start date."
        );

        return;
    }

    String passStatus =
            passStatusChoice.getSelectedItem();

    ParkingPass pass =
            new ParkingPass();
```

```
            pass.setUserId(userId);

            pass.setVehicleId(vehicleId);

            pass.setPassType(passType);

            pass.setStartDate(startDate);

            pass.setEndDate(endDate);

            pass.setPassStatus(passStatus);

            pass.setPassAmount(passAmount);

            ParkingPassDAO dao =
                    new ParkingPassDAO();

            boolean result =
                    dao.addPass(pass);

            if (result) {

                showMessage(
                        "Parking Pass Added Successfully!"
                );

                clearFields();

            } else {

                showMessage(
                        "Parking Pass Addition Failed!"
                );
            }

        } catch (NumberFormatException ex) {

            showMessage(
                    "User ID, Vehicle ID and Pass Amount\n"
                    + "must contain valid numbers."
            );

        } catch (IllegalArgumentException ex) {

            showMessage(
                    "Invalid date format.\n\n"
                    + "Please use: YYYY-MM-DD"
            );
```

```
    } catch (Exception ex) {

        showMessage(
                "Error adding parking pass:\n"
                + ex.getMessage()
        );

        ex.printStackTrace();
    }
}

// ========================================================
// SEARCH PASS
// ========================================================

private void searchPass() {

    try {

        String passText =
                passIdField.getText().trim();

        if (passText.isEmpty()) {

            showMessage(
                    "Please enter Pass ID."
            );

            return;
        }

        int passId =
                Integer.parseInt(passText);

        ParkingPassDAO dao =
                new ParkingPassDAO();

        ParkingPass pass =
                dao.searchPass(passId);

        if (pass != null) {

            userIdField.setText(
                    String.valueOf(
                            pass.getUserId()
                    )
            );

            vehicleIdField.setText(
                    String.valueOf(
```

```
                pass.getVehicleId()
        )
);

passTypeField.setText(
        pass.getPassType()
);

startDateField.setText(
        String.valueOf(
                pass.getStartDate()
        )
);

endDateField.setText(
        String.valueOf(
                pass.getEndDate()
        )
);

passStatusChoice.select(
        pass.getPassStatus()
);

passAmountField.setText(
        String.valueOf(
                pass.getPassAmount()
        )
);

// Show details on right side
resultArea.setText(
        "================================\n"
        + "      PARKING PASS FOUND\n"
        + "================================\n\n"
        + "Pass ID      : "
        + pass.getPassId()
        + "\n"
        + "User ID      : "
        + pass.getUserId()
        + "\n"
        + "Vehicle ID   : "
        + pass.getVehicleId()
        + "\n"
        + "Pass Type    : "
        + pass.getPassType()
        + "\n"
        + "Start Date   : "
        + pass.getStartDate()
        + "\n"
```

```
                + "End Date     : "
                + pass.getEndDate()
                + "\n"
                + "Status       : "
                + pass.getPassStatus()
                + "\n"
                + "Pass Amount  : "
                + pass.getPassAmount()
                + "\n\n"
                + "================================="
            );

        } else {

            resultArea.setText(
                    "No parking pass found\n"
                    + "for Pass ID: "
                    + passId
            );

            showMessage(
                    "Parking Pass Not Found!"
            );
        }

    } catch (NumberFormatException ex) {

        showMessage(
                "Pass ID must be a valid number."
        );

    } catch (Exception ex) {

        showMessage(
                "Error searching parking pass:\n"
                + ex.getMessage()
        );

        ex.printStackTrace();
    }
}

// ============================================================
// CLEAR FIELDS
// ============================================================

private void clearFields() {

    passIdField.setText("");
```

```
    userIdField.setText("");

    vehicleIdField.setText("");

    passTypeField.setText("");

    startDateField.setText("YYYY-MM-DD");

    endDateField.setText("YYYY-MM-DD");

    passStatusChoice.select(0);

    passAmountField.setText("0.00");

    resultArea.setText(
            "Enter Pass ID and click SEARCH\n"
            + "to view an existing parking pass."
    );
}

// ============================================================
// CONFIRM CLOSE
// ============================================================

private void confirmClose() {

    Dialog dialog =
            new Dialog(
                    this,
                    "Confirm Exit",
                    true
            );

    dialog.setSize(400, 170);

    dialog.setLayout(
            new BorderLayout(10, 10)
    );

    Label message =
            new Label(
                    "Are you sure you want to close?",
                    Label.CENTER
            );

    message.setFont(
            new Font("Arial", Font.BOLD, 14)
    );

    dialog.add(
```

```
        message,
        BorderLayout.CENTER
    );

    Panel panel =
        new Panel();

    Button yesButton =
        new Button("YES");

    Button noButton =
        new Button("NO");

    yesButton.setBackground(CLOSE_COLOR);

    yesButton.setForeground(Color.WHITE);

    noButton.setBackground(SEARCH_COLOR);

    noButton.setForeground(Color.WHITE);

    yesButton.addActionListener(
        new ActionListener() {

            @Override
            public void actionPerformed(
                    ActionEvent e) {

                dialog.dispose();

                dispose();
            }
        }
    );

    noButton.addActionListener(
        new ActionListener() {

            @Override
            public void actionPerformed(
                    ActionEvent e) {

                dialog.dispose();
            }
        }
    );

    panel.add(yesButton);

    panel.add(noButton);
```

```
    dialog.add(
            panel,
            BorderLayout.SOUTH
    );

    dialog.setLocationRelativeTo(this);

    dialog.setVisible(true);
}

// ========================================================
// MESSAGE DIALOG
// ========================================================

private void showMessage(String message) {

    Dialog dialog =
            new Dialog(
                    this,
                    "Parking Pass Management",
                    true
            );

    dialog.setSize(450, 190);

    dialog.setLayout(
            new BorderLayout(10, 10)
    );

    TextArea messageArea =
            new TextArea();

    messageArea.setText(message);

    messageArea.setEditable(false);

    messageArea.setFont(
            new Font("Arial", Font.PLAIN, 14)
    );

    dialog.add(
            messageArea,
            BorderLayout.CENTER
    );

    Panel buttonPanel =
            new Panel();

    Button okButton =
```

```
            new Button("OK");

        okButton.setFont(
            new Font("Arial", Font.BOLD, 13)
        );

        okButton.setBackground(BUTTON_COLOR);

        okButton.setForeground(Color.WHITE);

        okButton.addActionListener(
            new ActionListener() {

                @Override
                public void actionPerformed(
                        ActionEvent e) {

                    dialog.dispose();
                }
            }
        );

        buttonPanel.add(okButton);

        dialog.add(
            buttonPanel,
            BorderLayout.SOUTH
        );

        dialog.setLocationRelativeTo(this);

        dialog.setVisible(true);
    }

    // ============================================================
    // MAIN METHOD
    // ============================================================

    public static void main(String[] args) {

        new ParkingPassManagement();
    }
}
```

ParkingSession.java

```
package parking;

import java.sql.Timestamp;
```

```java
public class ParkingSession {

    private int sessionId;
    private int vehicleId;
    private int slotId;
    private Timestamp entryTime;
    private Timestamp exitTime;
    private int durationMinutes;
    private double parkingFee;
    private String sessionStatus;

    public ParkingSession() {
    }

    public ParkingSession(int sessionId, int vehicleId, int slotId,
                          Timestamp entryTime, Timestamp exitTime,
                          int durationMinutes, double parkingFee,
                          String sessionStatus) {

        this.sessionId = sessionId;
        this.vehicleId = vehicleId;
        this.slotId = slotId;
        this.entryTime = entryTime;
        this.exitTime = exitTime;
        this.durationMinutes = durationMinutes;
        this.parkingFee = parkingFee;
        this.sessionStatus = sessionStatus;
    }

    public int getSessionId() {
        return sessionId;
    }

    public void setSessionId(int sessionId) {
        this.sessionId = sessionId;
    }

    public int getVehicleId() {
        return vehicleId;
    }

    public void setVehicleId(int vehicleId) {
        this.vehicleId = vehicleId;
    }

    public int getSlotId() {
        return slotId;
    }

    public void setSlotId(int slotId) {
```

```
        this.slotId = slotId;
    }

    public Timestamp getEntryTime() {
        return entryTime;
    }

    public void setEntryTime(Timestamp entryTime) {
        this.entryTime = entryTime;
    }

    public Timestamp getExitTime() {
        return exitTime;
    }

    public void setExitTime(Timestamp exitTime) {
        this.exitTime = exitTime;
    }

    public int getDurationMinutes() {
        return durationMinutes;
    }

    public void setDurationMinutes(int durationMinutes) {
        this.durationMinutes = durationMinutes;
    }

    public double getParkingFee() {
        return parkingFee;
    }

    public void setParkingFee(double parkingFee) {
        this.parkingFee = parkingFee;
    }

    public String getSessionStatus() {
        return sessionStatus;
    }

    public void setSessionStatus(String sessionStatus) {
        this.sessionStatus = sessionStatus;
    }
}
```

ParkingSessionDAO.java

```
package parking;

import java.sql.Connection;
import java.sql.PreparedStatement;
```

```
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Timestamp;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.CopyOnWriteArrayList;

public class ParkingSessionDAO {

            private static final List<ParkingSession> demoSessions = new
CopyOnWriteArrayList<>();

    static {
        initDemoSessions();
    }

    private static void initDemoSessions() {
        if (!demoSessions.isEmpty()) return;
        long now = System.currentTimeMillis();
        demoSessions.add(new ParkingSession(1, 1, 102, new Timestamp(now - 7200000), null,
120, 60.0, "Active"));
        demoSessions.add(new ParkingSession(2, 2, 106, new Timestamp(now - 3600000), null,
60, 30.0, "Active"));
          demoSessions.add(new ParkingSession(3, 3, 104, new Timestamp(now - 14400000),
new Timestamp(now - 3600000), 180, 90.0, "Completed"));
    }

    // GET NEXT AUTO-GENERATED SESSION ID
    public int getNextSessionId() {
              String sql = "SELECT COALESCE(MAX(session_id), 0) + 1 FROM
parking_sessions";
        Connection con = DBConnection.getConnection();
        if (con != null) {
            try (con; PreparedStatement ps = con.prepareStatement(sql);
                 ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    return rs.getInt(1);
                }
            } catch (SQLException ignored) {}
        }
        int max = 0;
        for (ParkingSession s : demoSessions) {
            if (s.getSessionId() > max) max = s.getSessionId();
        }
        return max + 1;
    }

    // ADD PARKING SESSION
    public boolean addSession(ParkingSession session) {
```

```
    String sql = "INSERT INTO parking_sessions "
               + "(vehicle_id, slot_id, entry_time, exit_time, "
               + "duration_minutes, parking_fee, session_status) "
               + "VALUES (?, ?, ?, ?, ?, ?, ?)";

    Connection con = DBConnection.getConnection();
    if (con != null) {
        try (con; PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, session.getVehicleId());
            ps.setInt(2, session.getSlotId());
            ps.setTimestamp(3, session.getEntryTime());
            ps.setTimestamp(4, session.getExitTime());
            ps.setInt(5, session.getDurationMinutes());
            ps.setDouble(6, session.getParkingFee());
            ps.setString(7, session.getSessionStatus());

            int rows = ps.executeUpdate();
            return rows > 0;

        } catch (SQLException e) {
            System.out.println("Error adding parking session in DB: " + e.getMessage());
        }
    }

    if (session.getSessionId() == 0) {
        session.setSessionId(demoSessions.size() + 1);
    }
    demoSessions.add(session);
    return true;
}


// SEARCH PARKING SESSION BY ID
public ParkingSession searchSession(int sessionId) {

    String sql = "SELECT * FROM parking_sessions WHERE session_id = ?";

    Connection con = DBConnection.getConnection();
    if (con != null) {
        try (con; PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, sessionId);
            ResultSet rs = ps.executeQuery();

            if (rs.next()) {
                ParkingSession session = new ParkingSession();
                session.setSessionId(rs.getInt("session_id"));
                session.setVehicleId(rs.getInt("vehicle_id"));
                session.setSlotId(rs.getInt("slot_id"));
```

```
                session.setEntryTime(rs.getTimestamp("entry_time"));
                session.setExitTime(rs.getTimestamp("exit_time"));
                session.setDurationMinutes(rs.getInt("duration_minutes"));
                session.setParkingFee(rs.getDouble("parking_fee"));
                session.setSessionStatus(rs.getString("session_status"));
                return session;
            }

        } catch (SQLException e) {
            System.out.println("Error searching parking session in DB: " + e.getMessage());
        }
    }

    for (ParkingSession s : demoSessions) {
        if (s.getSessionId() == sessionId) return s;
    }

    return null;
  }

  // UPDATE PARKING SESSION
  public boolean updateSession(ParkingSession session) {

      String sql = "UPDATE parking_sessions SET vehicle_id = ?, slot_id = ?, entry_time = ?, exit_time = ?, duration_minutes = ?, parking_fee = ?, session_status = ? WHERE session_id = ?";

    Connection con = DBConnection.getConnection();
    if (con != null) {
        try (con; PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, session.getVehicleId());
            ps.setInt(2, session.getSlotId());
            ps.setTimestamp(3, session.getEntryTime());
            ps.setTimestamp(4, session.getExitTime());
            ps.setInt(5, session.getDurationMinutes());
            ps.setDouble(6, session.getParkingFee());
            ps.setString(7, session.getSessionStatus());
            ps.setInt(8, session.getSessionId());

            int rows = ps.executeUpdate();
            return rows > 0;

        } catch (SQLException e) {
            System.out.println("Error updating parking session in DB: " + e.getMessage());
        }
    }

    for (int i = 0; i < demoSessions.size(); i++) {
        if (demoSessions.get(i).getSessionId() == session.getSessionId()) {
```

```
            demoSessions.set(i, session);
            return true;
        }
    }

    return false;
}

// DELETE PARKING SESSION
public boolean deleteSession(int sessionId) {

    String sql = "DELETE FROM parking_sessions WHERE session_id = ?";

    Connection con = DBConnection.getConnection();
    if (con != null) {
        try (con; PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, sessionId);
            int rows = ps.executeUpdate();
            return rows > 0;

        } catch (SQLException e) {
            System.out.println("Error deleting parking session in DB: " + e.getMessage());
        }
    }

    return demoSessions.removeIf(s -> s.getSessionId() == sessionId);
}

// GET ALL SESSIONS
public List<ParkingSession> getAllSessions() {

    List<ParkingSession> list = new ArrayList<>();
    String sql = "SELECT * FROM parking_sessions ORDER BY session_id";

    Connection con = DBConnection.getConnection();
    if (con != null) {
        try (con;
             PreparedStatement ps = con.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {

            while (rs.next()) {
                ParkingSession s = new ParkingSession();
                s.setSessionId(rs.getInt("session_id"));
                s.setVehicleId(rs.getInt("vehicle_id"));
                s.setSlotId(rs.getInt("slot_id"));
                s.setEntryTime(rs.getTimestamp("entry_time"));
                s.setExitTime(rs.getTimestamp("exit_time"));
                s.setDurationMinutes(rs.getInt("duration_minutes"));
                s.setParkingFee(rs.getDouble("parking_fee"));
```

```
                s.setSessionStatus(rs.getString("session_status"));
                list.add(s);
            }

            if (!list.isEmpty()) {
                return list;
            }

        } catch (SQLException e) {
                    System.out.println("Error retrieving parking sessions from DB: " +
e.getMessage());
        }
    }

    return new ArrayList<>(demoSessions);
  }
}
```

ParkingSessionManagement.java

```
package parking;

import java.awt.*;
import java.awt.event.*;
import java.sql.Timestamp;
import java.util.List;

public class ParkingSessionManagement extends Frame implements ActionListener {

  // COLORS
  private final Color DARK_BLUE  = new Color(25, 55, 95);
  private final Color BLUE       = new Color(45, 95, 150);
  private final Color LIGHT_BG   = new Color(245, 247, 250);
  private final Color DARK_TEXT  = new Color(30, 41, 59);
  private final Color WHITE      = Color.WHITE;

  private final Color GREEN      = new Color(40, 140, 85);
  private final Color ORANGE     = new Color(230, 145, 40);
  private final Color RED        = new Color(190, 65, 65);
  private final Color GRAY       = new Color(100, 116, 139);
  private final Color CHARCOAL   = new Color(51, 65, 85);

  // FORM INPUTS
  private TextField txtSessionId;
  private TextField txtVehicleId;
  private TextField txtSlotId;
  private TextField txtEntryTime;
  private TextField txtExitTime;
  private TextField txtDuration;
  private TextField txtParkingFee;
```

```
    private Choice statusChoice;

    // CUSTOM AWT BUTTONS
    private AwtButton addButton;
    private AwtButton searchButton;
    private AwtButton updateButton;
    private AwtButton deleteButton;
    private AwtButton viewButton;
    private AwtButton clearButton;
    private AwtButton closeButton;

    private TextArea output;
    private Label statusLabel;
    private ParkingSessionDAO sessionDAO;

    public ParkingSessionManagement() {

        sessionDAO = new ParkingSessionDAO();

        setTitle("Smart Campus Parking - Parking Session Management");
        setSize(1040, 740);
        setLayout(new BorderLayout(0, 0));
        setBackground(LIGHT_BG);

        // HEADER
        Panel headerPanel = new Panel(new BorderLayout(5, 5));
        headerPanel.setBackground(DARK_BLUE);

        Panel titleBox = new Panel(new GridLayout(2, 1));
        titleBox.setBackground(DARK_BLUE);

        Label title = new Label("PARKING SESSION MANAGEMENT", Label.CENTER);
        title.setFont(new Font("Arial", Font.BOLD, 22));
        title.setForeground(WHITE);
        titleBox.add(title);

        Label subtitle = new Label("Track Real-Time Vehicle Check-in, Check-out, Duration &
Billing", Label.CENTER);
        subtitle.setFont(new Font("Arial", Font.PLAIN, 13));
        subtitle.setForeground(new Color(210, 225, 245));
        titleBox.add(subtitle);

        headerPanel.add(titleBox, BorderLayout.CENTER);
        add(headerPanel, BorderLayout.NORTH);

        // MAIN CONTENT
        Panel mainContent = new Panel(new GridLayout(1, 2, 16, 0));
        mainContent.setBackground(LIGHT_BG);

        // LEFT CARD: FORM
```

```
Panel formCard = new Panel(new BorderLayout(0, 0));
formCard.setBackground(WHITE);

Panel formCardHeader = new Panel(new BorderLayout());
formCardHeader.setBackground(BLUE);
Label formTitle = new Label("  PARKING SESSION DETAILS", Label.LEFT);
formTitle.setFont(new Font("Arial", Font.BOLD, 14));
formTitle.setForeground(WHITE);
formCardHeader.add(formTitle, BorderLayout.CENTER);
formCard.add(formCardHeader, BorderLayout.NORTH);

Panel formBody = new Panel(new GridBagLayout());
formBody.setBackground(WHITE);
GridBagConstraints gbc = new GridBagConstraints();
gbc.insets = new Insets(6, 16, 6, 16);
gbc.fill = GridBagConstraints.HORIZONTAL;
gbc.anchor = GridBagConstraints.NORTHWEST;

gbc.gridx = 0; gbc.gridy = 0; gbc.weightx = 0.35;
formBody.add(createLabel("Session ID (Auto):"), gbc);
gbc.gridx = 1; gbc.weightx = 0.65;
txtSessionId = new TextField(String.valueOf(sessionDAO.getNextSessionId()));
txtSessionId.setFont(new Font("Arial", Font.BOLD, 13));
formBody.add(txtSessionId, gbc);

gbc.gridx = 0; gbc.gridy = 1; gbc.weightx = 0.35;
formBody.add(createLabel("Vehicle ID:"), gbc);
gbc.gridx = 1; gbc.weightx = 0.65;
txtVehicleId = new TextField("1");
formBody.add(txtVehicleId, gbc);

gbc.gridx = 0; gbc.gridy = 2; gbc.weightx = 0.35;
formBody.add(createLabel("Slot ID:"), gbc);
gbc.gridx = 1; gbc.weightx = 0.65;
txtSlotId = new TextField("101");
formBody.add(txtSlotId, gbc);

gbc.gridx = 0; gbc.gridy = 3; gbc.weightx = 0.35;
formBody.add(createLabel("Entry Time:"), gbc);
gbc.gridx = 1; gbc.weightx = 0.65;
txtEntryTime = new TextField("2026-09-01 10:00:00");
formBody.add(txtEntryTime, gbc);

gbc.gridx = 0; gbc.gridy = 4; gbc.weightx = 0.35;
formBody.add(createLabel("Exit Time:"), gbc);
gbc.gridx = 1; gbc.weightx = 0.65;
txtExitTime = new TextField("");
formBody.add(txtExitTime, gbc);

gbc.gridx = 0; gbc.gridy = 5; gbc.weightx = 0.35;
```

```
        formBody.add(createLabel("Duration (Mins):"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.65;
        txtDuration = new TextField("60");
        formBody.add(txtDuration, gbc);

        gbc.gridx = 0; gbc.gridy = 6; gbc.weightx = 0.35;
        formBody.add(createLabel("Parking Fee (₹):"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.65;
        txtParkingFee = new TextField("30.0");
        formBody.add(txtParkingFee, gbc);

        gbc.gridx = 0; gbc.gridy = 7; gbc.weightx = 0.35;
        formBody.add(createLabel("Status:"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.65;
        statusChoice = new Choice();
        statusChoice.add("Active");
        statusChoice.add("Completed");
        statusChoice.add("Overstay");
        formBody.add(statusChoice, gbc);

        gbc.gridx = 0; gbc.gridy = 8; gbc.gridwidth = 2; gbc.weighty = 1.0;
        formBody.add(new Panel(), gbc);

        formCard.add(formBody, BorderLayout.CENTER);
        mainContent.add(formCard);

        // RIGHT CARD: RECORDS
        Panel tableCard = new Panel(new BorderLayout(0, 0));
        tableCard.setBackground(WHITE);

        Panel tableCardHeader = new Panel(new BorderLayout());
        tableCardHeader.setBackground(DARK_BLUE);
        Label tableTitle = new Label("  CAMPUS PARKING SESSIONS", Label.LEFT);
        tableTitle.setFont(new Font("Arial", Font.BOLD, 14));
        tableTitle.setForeground(WHITE);
        tableCardHeader.add(tableTitle, BorderLayout.CENTER);
        tableCard.add(tableCardHeader, BorderLayout.NORTH);

        output = new TextArea("", 20, 50, TextArea.SCROLLBARS_VERTICAL_ONLY);
        output.setFont(new Font("Monospaced", Font.PLAIN, 12));
        output.setEditable(false);
        output.setBackground(new Color(250, 252, 255));
        tableCard.add(output, BorderLayout.CENTER);

        mainContent.add(tableCard);
        add(mainContent, BorderLayout.CENTER);

        // BOTTOM BAR
        Panel bottomContainer = new Panel(new BorderLayout(0, 0));
        bottomContainer.setBackground(LIGHT_BG);
```

```
    Panel buttonPanel = new Panel(new FlowLayout(FlowLayout.CENTER, 10, 10));
    buttonPanel.setBackground(new Color(230, 238, 248));

    addButton    = new AwtButton("□  CHECK-IN", GREEN, WHITE);
    searchButton = new AwtButton("□ SEARCH", BLUE, WHITE);
    updateButton = new AwtButton("✏□ UPDATE", ORANGE, WHITE);
    deleteButton = new AwtButton("□ DELETE", RED, WHITE);
    viewButton   = new AwtButton("□ VIEW ALL", DARK_BLUE, WHITE);
    clearButton  = new AwtButton("□ CLEAR", GRAY, WHITE);
    closeButton  = new AwtButton("□  CLOSE", CHARCOAL, WHITE);

    buttonPanel.add(addButton);
    buttonPanel.add(searchButton);
    buttonPanel.add(updateButton);
    buttonPanel.add(deleteButton);
    buttonPanel.add(viewButton);
    buttonPanel.add(clearButton);
    buttonPanel.add(closeButton);

    bottomContainer.add(buttonPanel, BorderLayout.CENTER);

    statusLabel = new Label(" Ready | Parking Session Management.", Label.LEFT);
    statusLabel.setFont(new Font("Arial", Font.PLAIN, 12));
    statusLabel.setForeground(DARK_TEXT);
    statusLabel.setBackground(new Color(220, 230, 242));
    bottomContainer.add(statusLabel, BorderLayout.SOUTH);

    add(bottomContainer, BorderLayout.SOUTH);

    addButton.addActionListener(this);
    searchButton.addActionListener(this);
    updateButton.addActionListener(this);
    deleteButton.addActionListener(this);
    viewButton.addActionListener(this);
    clearButton.addActionListener(this);
    closeButton.addActionListener(this);

    addWindowListener(new WindowAdapter() {
        @Override
        public void windowClosing(WindowEvent e) {
            dispose();
        }
    });

    viewSessions();

    setLocationRelativeTo(null);
    setVisible(true);
}
```

```
    private Label createLabel(String text) {
        Label lbl = new Label(text);
        lbl.setFont(new Font("Arial", Font.BOLD, 12));
        lbl.setForeground(DARK_TEXT);
        return lbl;
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        Object src = e.getSource();
        if (src == addButton) {
            addSession();
        } else if (src == searchButton) {
            searchSession();
        } else if (src == updateButton) {
            updateSession();
        } else if (src == deleteButton) {
            deleteSession();
        } else if (src == viewButton) {
            viewSessions();
        } else if (src == clearButton) {
            clearForm();
        } else if (src == closeButton) {
            dispose();
        }
    }

    private void addSession() {
        try {
            int vid = Integer.parseInt(txtVehicleId.getText().trim());
            int sid = Integer.parseInt(txtSlotId.getText().trim());
            Timestamp entry = Timestamp.valueOf(txtEntryTime.getText().trim());
                        Timestamp exit = txtExitTime.getText().trim().isEmpty() ? null :
Timestamp.valueOf(txtExitTime.getText().trim());
            int dur = Integer.parseInt(txtDuration.getText().trim());
            double fee = Double.parseDouble(txtParkingFee.getText().trim());
            String status = statusChoice.getSelectedItem();

            ParkingSession s = new ParkingSession();
            String idStr = txtSessionId.getText().trim();
            if (!idStr.isEmpty()) {
                try {
                    s.setSessionId(Integer.parseInt(idStr));
                } catch (NumberFormatException ignored) {}
            }
            s.setVehicleId(vid);
            s.setSlotId(sid);
            s.setEntryTime(entry);
            s.setExitTime(exit);
```

```
            s.setDurationMinutes(dur);
            s.setParkingFee(fee);
            s.setSessionStatus(status);

            boolean ok = sessionDAO.addSession(s);
            if (ok) {
                statusLabel.setText("✅ Parking Session (Check-in) Created!");
                showMessage("Parking Session Created Successfully!");
                clearForm();
                viewSessions();
            } else {
                showMessage("Failed to create parking session.");
            }
        } catch (Exception ex) {
            showMessage("Invalid input. Check numbers and timestamp format (YYYY-MM-DD HH:MM:SS).");
        }
    }

    private void searchSession() {
        try {
            int id = Integer.parseInt(txtSessionId.getText().trim());
            ParkingSession s = sessionDAO.searchSession(id);
            if (s != null) {
                txtVehicleId.setText(String.valueOf(s.getVehicleId()));
                txtSlotId.setText(String.valueOf(s.getSlotId()));
                if (s.getEntryTime() != null) txtEntryTime.setText(s.getEntryTime().toString());
                txtExitTime.setText(s.getExitTime() != null ? s.getExitTime().toString() : "");
                txtDuration.setText(String.valueOf(s.getDurationMinutes()));
                txtParkingFee.setText(String.valueOf(s.getParkingFee()));
                statusChoice.select(s.getSessionStatus());
                statusLabel.setText("✅ Found Session ID " + id);
            } else {
                showMessage("No parking session found with ID: " + id);
            }
        } catch (Exception ex) {
            showMessage("Please enter a valid numeric Session ID.");
        }
    }

    private void updateSession() {
        try {
            int id = Integer.parseInt(txtSessionId.getText().trim());
            int vid = Integer.parseInt(txtVehicleId.getText().trim());
            int sid = Integer.parseInt(txtSlotId.getText().trim());
            Timestamp entry = Timestamp.valueOf(txtEntryTime.getText().trim());
                    Timestamp exit = txtExitTime.getText().trim().isEmpty() ? null : Timestamp.valueOf(txtExitTime.getText().trim());
            int dur = Integer.parseInt(txtDuration.getText().trim());
            double fee = Double.parseDouble(txtParkingFee.getText().trim());
```

```
            String status = statusChoice.getSelectedItem();

            ParkingSession s = new ParkingSession();
            s.setSessionId(id);
            s.setVehicleId(vid);
            s.setSlotId(sid);
            s.setEntryTime(entry);
            s.setExitTime(exit);
            s.setDurationMinutes(dur);
            s.setParkingFee(fee);
            s.setSessionStatus(status);

            boolean ok = sessionDAO.updateSession(s);
            if (ok) {
                statusLabel.setText("✏ Session ID " + id + " updated successfully.");
                showMessage("Parking Session Updated Successfully!");
                clearForm();
                viewSessions();
            } else {
                showMessage("Failed to update parking session.");
            }
        } catch (Exception ex) {
            showMessage("Invalid input. Check numbers and timestamps.");
        }
    }

    private void deleteSession() {
        try {
            int id = Integer.parseInt(txtSessionId.getText().trim());
            boolean ok = sessionDAO.deleteSession(id);
            if (ok) {
                statusLabel.setText(" Session ID " + id + " deleted.");
                showMessage("Parking Session Deleted Successfully.");
                clearForm();
                viewSessions();
            } else {
                showMessage("Could not delete session ID: " + id);
            }
        } catch (Exception ex) {
            showMessage("Please enter a valid numeric Session ID.");
        }
    }

    private void viewSessions() {
        List<ParkingSession> list = sessionDAO.getAllSessions();
        output.setText("");

         output.append("==========================================================
==============================\n");
        output.append("                        ACTIVE & COMPLETED SESSIONS\n");
```

```
        output.append("================================================================================\n");
        output.append(String.format("%-6s %-8s %-8s %-12s %-10s %-10s%n", "SES ID", "VEHICLE", "SLOT", "DURATION", "FEE (₹)", "STATUS"));
        output.append("--------------------------------------------------------------------------------\n");

        if (list == null || list.isEmpty()) {
            output.append("\nNo parking session records found.\n");
            return;
        }

        for (ParkingSession s : list) {
            output.append(String.format("%-6d %-8d %-8d %-12s %-10.2f %-10s%n",
                    s.getSessionId(),
                    s.getVehicleId(),
                    s.getSlotId(),
                    s.getDurationMinutes() + " mins",
                    s.getParkingFee(),
                    s.getSessionStatus()
            ));
        }
        output.append("--------------------------------------------------------------------------------\n");
        output.append("Total Sessions: " + list.size() + "\n");
        statusLabel.setText(" Loaded " + list.size() + " parking sessions.");
    }

    private void clearForm() {
        txtSessionId.setText(String.valueOf(sessionDAO.getNextSessionId()));
        txtVehicleId.setText("1");
        txtSlotId.setText("101");
        txtEntryTime.setText("2026-09-01 10:00:00");
        txtExitTime.setText("");
        txtDuration.setText("60");
        txtParkingFee.setText("30.0");
        statusChoice.select(0);
        statusLabel.setText(" Ready | Next Session ID auto-assigned: " + sessionDAO.getNextSessionId());
    }

    private void showMessage(String msg) {
        Dialog d = new Dialog(this, "Notification", true);
        d.setSize(380, 160);
        d.setLayout(new BorderLayout(10, 10));
        d.setBackground(LIGHT_BG);

        Label lbl = new Label(msg, Label.CENTER);
        lbl.setFont(new Font("Arial", Font.PLAIN, 12));
        d.add(lbl, BorderLayout.CENTER);
```

```
        AwtButton btnOk = new AwtButton("OK", BLUE, WHITE);
        btnOk.addActionListener(e -> d.dispose());
        Panel p = new Panel(new FlowLayout());
        p.add(btnOk);
        d.add(p, BorderLayout.SOUTH);

        d.addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                d.dispose();
            }
        });

        d.setLocationRelativeTo(this);
        d.setVisible(true);
    }

    public static void main(String[] args) {
        new ParkingSessionManagement();
    }
}
```

ParkingSlot.java

```
package parking;

public class ParkingSlot {

    private int slotId;
    private int zoneId;
    private String slotNumber;
    private String vehicleType;
    private String slotStatus;

    public ParkingSlot() {
    }

    public ParkingSlot(int slotId, int zoneId, String slotNumber,
                       String vehicleType, String slotStatus) {

        this.slotId = slotId;
        this.zoneId = zoneId;
        this.slotNumber = slotNumber;
        this.vehicleType = vehicleType;
        this.slotStatus = slotStatus;
    }

    public int getSlotId() {
        return slotId;
```

```
    }

    public void setSlotId(int slotId) {
        this.slotId = slotId;
    }

    public int getZoneId() {
        return zoneId;
    }

    public void setZoneId(int zoneId) {
        this.zoneId = zoneId;
    }

    public String getSlotNumber() {
        return slotNumber;
    }

    public void setSlotNumber(String slotNumber) {
        this.slotNumber = slotNumber;
    }

    public String getVehicleType() {
        return vehicleType;
    }

    public void setVehicleType(String vehicleType) {
        this.vehicleType = vehicleType;
    }

    public String getSlotStatus() {
        return slotStatus;
    }

    public void setSlotStatus(String slotStatus) {
        this.slotStatus = slotStatus;
    }
}
```

ParkingSlotDAO.java

```
package parking;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.CopyOnWriteArrayList;
```

```
public class ParkingSlotDAO {

    // In-memory fallback slots for demo and when database is unreachable
    private static final List<ParkingSlot> demoSlots = new CopyOnWriteArrayList<>();

    static {
        initDemoSlots();
    }

    private static void initDemoSlots() {
        if (!demoSlots.isEmpty()) return;
        // Zone 1: Main Gate - Two Wheeler & Four Wheeler & EV
        demoSlots.add(new ParkingSlot(101, 1, "A-01", "Four Wheeler", "Available"));
        demoSlots.add(new ParkingSlot(102, 1, "A-02", "Four Wheeler", "Occupied"));
        demoSlots.add(new ParkingSlot(103, 1, "A-03", "Four Wheeler", "Available"));
        demoSlots.add(new ParkingSlot(104, 1, "A-04", "EV", "Occupied"));
        demoSlots.add(new ParkingSlot(105, 1, "A-05", "Two Wheeler", "Available"));
        demoSlots.add(new ParkingSlot(106, 1, "A-06", "Two Wheeler", "Occupied"));
        demoSlots.add(new ParkingSlot(107, 1, "A-07", "Two Wheeler", "Available"));
        demoSlots.add(new ParkingSlot(108, 1, "A-08", "Four Wheeler", "Reserved"));

        // Zone 2: Science Block - Faculty & Student
        demoSlots.add(new ParkingSlot(201, 2, "B-01", "Four Wheeler", "Occupied"));
        demoSlots.add(new ParkingSlot(202, 2, "B-02", "Four Wheeler", "Available"));
        demoSlots.add(new ParkingSlot(203, 2, "B-03", "EV", "Available"));
        demoSlots.add(new ParkingSlot(204, 2, "B-04", "Two Wheeler", "Occupied"));
        demoSlots.add(new ParkingSlot(205, 2, "B-05", "Two Wheeler", "Available"));
        demoSlots.add(new ParkingSlot(206, 2, "B-06", "Four Wheeler", "Maintenance"));

        // Zone 3: Admin Block - VIP & Staff
        demoSlots.add(new ParkingSlot(301, 3, "C-01", "Four Wheeler", "Reserved"));
        demoSlots.add(new ParkingSlot(302, 3, "C-02", "Four Wheeler", "Available"));
        demoSlots.add(new ParkingSlot(303, 3, "C-03", "EV", "Occupied"));
        demoSlots.add(new ParkingSlot(304, 3, "C-04", "Two Wheeler", "Available"));
        demoSlots.add(new ParkingSlot(305, 3, "C-05", "Four Wheeler", "Available"));
        demoSlots.add(new ParkingSlot(306, 3, "C-06", "Four Wheeler", "Occupied"));

        // Zone 4: Academic Block - Students
        demoSlots.add(new ParkingSlot(401, 4, "D-01", "Two Wheeler", "Available"));
        demoSlots.add(new ParkingSlot(402, 4, "D-02", "Two Wheeler", "Occupied"));
        demoSlots.add(new ParkingSlot(403, 4, "D-03", "Two Wheeler", "Available"));
        demoSlots.add(new ParkingSlot(404, 4, "D-04", "Four Wheeler", "Available"));
        demoSlots.add(new ParkingSlot(405, 4, "D-05", "Four Wheeler", "Occupied"));
        demoSlots.add(new ParkingSlot(406, 4, "D-06", "EV", "Available"));
    }

    // GET NEXT AUTO-GENERATED SLOT ID
    public int getNextSlotId() {
        String sql = "SELECT COALESCE(MAX(slot_id), 0) + 1 FROM parking_slots";
```

```
    Connection con = DBConnection.getConnection();
    if (con != null) {
        try (con; PreparedStatement ps = con.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {
            if (rs.next()) {
                return rs.getInt(1);
            }
        } catch (SQLException ignored) {}
    }
    int max = 0;
    for (ParkingSlot s : demoSlots) {
        if (s.getSlotId() > max) max = s.getSlotId();
    }
    return max + 1;
}

// ====================================================
// ADD PARKING SLOT
// ====================================================

public boolean addSlot(ParkingSlot slot) {

    String sql = "INSERT INTO parking_slots "
               + "(zone_id, slot_number, vehicle_type, slot_status) "
               + "VALUES (?, ?, ?, ?)";

    Connection con = DBConnection.getConnection();
    if (con != null) {
        try (con; PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, slot.getZoneId());
            ps.setString(2, slot.getSlotNumber());
            ps.setString(3, slot.getVehicleType());
            ps.setString(4, slot.getSlotStatus());

            int rows = ps.executeUpdate();
            return rows > 0;

        } catch (SQLException e) {
            System.out.println("Error adding parking slot in DB: " + e.getMessage());
        }
    }

    // In-memory fallback
    if (slot.getSlotId() == 0) {
        slot.setSlotId(500 + demoSlots.size() + 1);
    }
    demoSlots.add(slot);
    return true;
}
```

```java
// ====================================================
// SEARCH PARKING SLOT BY ID
// ====================================================

public ParkingSlot searchSlot(int slotId) {

    String sql = "SELECT * FROM parking_slots "
               + "WHERE slot_id = ?";

    Connection con = DBConnection.getConnection();
    if (con != null) {
        try (con; PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, slotId);

            try (ResultSet rs = ps.executeQuery()) {

                if (rs.next()) {

                    ParkingSlot slot = new ParkingSlot();
                    slot.setSlotId(rs.getInt("slot_id"));
                    slot.setZoneId(rs.getInt("zone_id"));
                    slot.setSlotNumber(rs.getString("slot_number"));
                    slot.setVehicleType(rs.getString("vehicle_type"));
                    slot.setSlotStatus(rs.getString("slot_status"));

                    return slot;
                }
            }

        } catch (SQLException e) {
            System.out.println("Error searching parking slot in DB: " + e.getMessage());
        }
    }

    // In-memory fallback
    for (ParkingSlot s : demoSlots) {
        if (s.getSlotId() == slotId) {
            return s;
        }
    }

    return null;
}


// ====================================================
// GET ALL PARKING SLOTS
```

```java
// ====================================================

public List<ParkingSlot> getAllSlots() {

    List<ParkingSlot> slots = new ArrayList<>();

    String sql = "SELECT * FROM parking_slots "
               + "ORDER BY zone_id, slot_number";

    Connection con = DBConnection.getConnection();
    if (con != null) {
        try (con;
             PreparedStatement ps = con.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {

            while (rs.next()) {

                ParkingSlot slot = new ParkingSlot();
                slot.setSlotId(rs.getInt("slot_id"));
                slot.setZoneId(rs.getInt("zone_id"));
                slot.setSlotNumber(rs.getString("slot_number"));
                slot.setVehicleType(rs.getString("vehicle_type"));
                slot.setSlotStatus(rs.getString("slot_status"));

                slots.add(slot);
            }

            if (!slots.isEmpty()) {
                return slots;
            }

        } catch (SQLException e) {
            System.out.println("Error loading parking slots from DB: " + e.getMessage());
        }
    }

    // Return in-memory demo slots
    return new ArrayList<>(demoSlots);
}


// ====================================================
// UPDATE PARKING SLOT STATUS
// ====================================================

public boolean updateSlotStatus(int slotId, String newStatus) {

    String sql = "UPDATE parking_slots SET slot_status = ? WHERE slot_id = ?";

    Connection con = DBConnection.getConnection();
```

```
        if (con != null) {
            try (con; PreparedStatement ps = con.prepareStatement(sql)) {

                ps.setString(1, newStatus);
                ps.setInt(2, slotId);

                int rows = ps.executeUpdate();
                if (rows > 0) {
                    return true;
                }

            } catch (SQLException e) {
                System.out.println("Error updating slot status in DB: " + e.getMessage());
            }
        }

        // In-memory fallback
        for (ParkingSlot s : demoSlots) {
            if (s.getSlotId() == slotId) {
                s.setSlotStatus(newStatus);
                return true;
            }
        }

        return false;
    }


    // ========================================================
    // UPDATE PARKING SLOT
    // ========================================================

    public boolean updateSlot(ParkingSlot slot) {

        String sql = "UPDATE parking_slots SET zone_id = ?, slot_number = ?, vehicle_type =
?, slot_status = ? WHERE slot_id = ?";

        Connection con = DBConnection.getConnection();
        if (con != null) {
            try (con; PreparedStatement ps = con.prepareStatement(sql)) {

                ps.setInt(1, slot.getZoneId());
                ps.setString(2, slot.getSlotNumber());
                ps.setString(3, slot.getVehicleType());
                ps.setString(4, slot.getSlotStatus());
                ps.setInt(5, slot.getSlotId());

                int rows = ps.executeUpdate();
                return rows > 0;
```

```
            } catch (SQLException e) {
                System.out.println("Error updating slot in DB: " + e.getMessage());
            }
        }

        // In-memory fallback
        for (int i = 0; i < demoSlots.size(); i++) {
            if (demoSlots.get(i).getSlotId() == slot.getSlotId()) {
                demoSlots.set(i, slot);
                return true;
            }
        }

        return false;
    }


    // ========================================================
    // DELETE PARKING SLOT
    // ========================================================

    public boolean deleteSlot(int slotId) {

        String sql = "DELETE FROM parking_slots "
                   + "WHERE slot_id = ?";

        Connection con = DBConnection.getConnection();
        if (con != null) {
            try (con; PreparedStatement ps = con.prepareStatement(sql)) {

                ps.setInt(1, slotId);

                int rows = ps.executeUpdate();
                return rows > 0;

            } catch (SQLException e) {
                System.out.println("Error deleting parking slot in DB: " + e.getMessage());
            }
        }

        // In-memory fallback
        return demoSlots.removeIf(s -> s.getSlotId() == slotId);
    }
}
```

ParkingSlotManagement.java

```
package parking;
```

```
import java.awt.*;
import java.awt.event.*;
import java.util.List;

public class ParkingSlotManagement extends Frame implements ActionListener {

    // ================================================
    // COLORS
    // ================================================
    private final Color DARK_BLUE  = new Color(25, 55, 95);
    private final Color BLUE       = new Color(45, 95, 150);
    private final Color LIGHT_BLUE = new Color(235, 243, 252);
    private final Color LIGHT_BG   = new Color(245, 247, 250);
    private final Color DARK_TEXT  = new Color(30, 41, 59);
    private final Color WHITE      = Color.WHITE;

    private final Color PURPLE     = new Color(124, 58, 237);
    private final Color GREEN      = new Color(40, 140, 85);
    private final Color RED        = new Color(190, 65, 65);
    private final Color GRAY       = new Color(100, 116, 139);
    private final Color CHARCOAL   = new Color(51, 65, 85);

    // ================================================
    // FORM INPUTS
    // ================================================
    private TextField slotIdField;
    private TextField zoneIdField;
    private TextField slotNumberField;
    private Choice vehicleTypeChoice;
    private Choice slotStatusChoice;

    // ================================================
    // CUSTOM AWT BUTTONS (Cross-platform visible text)
    // ================================================
    private AwtButton visualMapButton;
    private AwtButton addButton;
    private AwtButton searchButton;
    private AwtButton viewButton;
    private AwtButton deleteButton;
    private AwtButton clearButton;
    private AwtButton closeButton;

    // ================================================
    // OUTPUT & STATUS
    // ================================================
    private TextArea resultArea;
    private Label statusLabel;

    private ParkingSlotDAO slotDAO;
```

```
    // ================================================
    // CONSTRUCTOR
    // ================================================
    public ParkingSlotManagement() {

        slotDAO = new ParkingSlotDAO();

        setTitle("Smart Campus Parking - Parking Slot Management");
        setSize(1020, 720);
        setLayout(new BorderLayout(0, 0));
        setBackground(LIGHT_BG);

        // ================================================
        // 1. HEADER
        // ================================================
        Panel headerPanel = new Panel(new BorderLayout(5, 5));
        headerPanel.setBackground(DARK_BLUE);

        Panel titleBox = new Panel(new GridLayout(2, 1));
        titleBox.setBackground(DARK_BLUE);

        Label title = new Label("PARKING SLOT MANAGEMENT", Label.CENTER);
        title.setFont(new Font("Arial", Font.BOLD, 22));
        title.setForeground(WHITE);
        titleBox.add(title);

         Label subtitle = new Label("Configure Campus Parking Slots  |  Zone Mapping  |  Slot
Status", Label.CENTER);
        subtitle.setFont(new Font("Arial", Font.PLAIN, 13));
        subtitle.setForeground(new Color(210, 225, 245));
        titleBox.add(subtitle);

        headerPanel.add(titleBox, BorderLayout.CENTER);
        add(headerPanel, BorderLayout.NORTH);

        // ================================================
        // 2. MAIN CONTENT (Split: Left Form, Right Records)
        // ================================================
        Panel mainContent = new Panel(new GridLayout(1, 2, 16, 0));
        mainContent.setBackground(LIGHT_BG);

        // --- LEFT CARD: FORM ---
        Panel formCard = new Panel(new BorderLayout(0, 0));
        formCard.setBackground(WHITE);

        Panel formCardHeader = new Panel(new BorderLayout());
        formCardHeader.setBackground(BLUE);
        Label formTitle = new Label("  SLOT CONFIGURATION FORM", Label.LEFT);
        formTitle.setFont(new Font("Arial", Font.BOLD, 14));
        formTitle.setForeground(WHITE);
```

```
        formCardHeader.add(formTitle, BorderLayout.CENTER);
        formCard.add(formCardHeader, BorderLayout.NORTH);

        Panel formBody = new Panel(new GridBagLayout());
        formBody.setBackground(WHITE);
        GridBagConstraints gbc = new GridBagConstraints();
        gbc.insets = new Insets(10, 16, 10, 16);
        gbc.fill = GridBagConstraints.HORIZONTAL;
        gbc.anchor = GridBagConstraints.NORTHWEST;

        // Row 0: Slot ID
        gbc.gridx = 0; gbc.gridy = 0; gbc.weightx = 0.3;
        formBody.add(createLabel("Slot ID (Auto):"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.7;
        slotIdField = new TextField(String.valueOf(slotDAO.getNextSlotId()));
        slotIdField.setFont(new Font("Arial", Font.BOLD, 13));
        formBody.add(slotIdField, gbc);

        // Row 1: Zone ID
        gbc.gridx = 0; gbc.gridy = 1; gbc.weightx = 0.3;
        formBody.add(createLabel("Zone ID (1-4):"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.7;
        zoneIdField = new TextField();
        zoneIdField.setFont(new Font("Arial", Font.PLAIN, 14));
        formBody.add(zoneIdField, gbc);

        // Row 2: Slot Number
        gbc.gridx = 0; gbc.gridy = 2; gbc.weightx = 0.3;
        formBody.add(createLabel("Slot Code:"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.7;
        slotNumberField = new TextField();
        slotNumberField.setFont(new Font("Arial", Font.PLAIN, 14));
        formBody.add(slotNumberField, gbc);

        // Row 3: Vehicle Type
        gbc.gridx = 0; gbc.gridy = 3; gbc.weightx = 0.3;
        formBody.add(createLabel("Vehicle Type:"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.7;
        vehicleTypeChoice = new Choice();
        vehicleTypeChoice.add("Four Wheeler");
        vehicleTypeChoice.add("Two Wheeler");
        vehicleTypeChoice.add("EV");
        vehicleTypeChoice.setFont(new Font("Arial", Font.PLAIN, 13));
        formBody.add(vehicleTypeChoice, gbc);

        // Row 4: Slot Status
        gbc.gridx = 0; gbc.gridy = 4; gbc.weightx = 0.3;
        formBody.add(createLabel("Slot Status:"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.7;
        slotStatusChoice = new Choice();
```

```
        slotStatusChoice.add("Available");
        slotStatusChoice.add("Occupied");
        slotStatusChoice.add("Reserved");
        slotStatusChoice.add("Maintenance");
        slotStatusChoice.setFont(new Font("Arial", Font.PLAIN, 13));
        formBody.add(slotStatusChoice, gbc);

        // Row 5: Helper Notice Box
        gbc.gridx = 0; gbc.gridy = 5; gbc.gridwidth = 2;
        Panel noticeBox = new Panel(new BorderLayout(5, 5));
        noticeBox.setBackground(new Color(241, 245, 249));
        Label noticeLbl = new Label(" □ Tip: Use Slot ID to Search or Delete. Use 'LIVE MAP'
for visual control.", Label.CENTER);
        noticeLbl.setFont(new Font("Arial", Font.ITALIC, 11));
        noticeLbl.setForeground(new Color(71, 85, 105));
        noticeBox.add(noticeLbl, BorderLayout.CENTER);
        formBody.add(noticeBox, gbc);

        // Row 6: Vertical filler to keep everything neatly top-aligned
        gbc.gridx = 0; gbc.gridy = 6; gbc.gridwidth = 2; gbc.weighty = 1.0;
        formBody.add(new Panel(), gbc);

        formCard.add(formBody, BorderLayout.CENTER);
        mainContent.add(formCard);

        // --- RIGHT CARD: RECORDS TABLE ---
        Panel tableCard = new Panel(new BorderLayout(0, 0));
        tableCard.setBackground(WHITE);

        Panel tableCardHeader = new Panel(new BorderLayout());
        tableCardHeader.setBackground(DARK_BLUE);
            Label tableTitle = new Label("  PARKING SLOT INVENTORY RECORDS",
Label.LEFT);
        tableTitle.setFont(new Font("Arial", Font.BOLD, 14));
        tableTitle.setForeground(WHITE);
        tableCardHeader.add(tableTitle, BorderLayout.CENTER);
        tableCard.add(tableCardHeader, BorderLayout.NORTH);

        resultArea = new TextArea("", 20, 50, TextArea.SCROLLBARS_VERTICAL_ONLY);
        resultArea.setFont(new Font("Monospaced", Font.PLAIN, 12));
        resultArea.setEditable(false);
        resultArea.setBackground(new Color(250, 252, 255));
        tableCard.add(resultArea, BorderLayout.CENTER);

        mainContent.add(tableCard);

        add(mainContent, BorderLayout.CENTER);

        // ==========================================
        // 3. BOTTOM ACTIONS & STATUS BAR
```

```
        // ================================================
        Panel bottomContainer = new Panel(new BorderLayout(0, 0));
        bottomContainer.setBackground(LIGHT_BG);

        // Button Toolbar
        Panel buttonPanel = new Panel(new FlowLayout(FlowLayout.CENTER, 12, 10));
        buttonPanel.setBackground(new Color(230, 238, 248));

        visualMapButton = new AwtButton("□ LIVE MAP", PURPLE, WHITE);
        addButton       = new AwtButton("□  ADD", GREEN, WHITE);
        searchButton    = new AwtButton("□ SEARCH", BLUE, WHITE);
        viewButton      = new AwtButton("□ VIEW ALL", DARK_BLUE, WHITE);
        deleteButton    = new AwtButton("□ DELETE", RED, WHITE);
        clearButton     = new AwtButton("□ CLEAR", GRAY, WHITE);
        closeButton     = new AwtButton("□  CLOSE", CHARCOAL, WHITE);

        buttonPanel.add(visualMapButton);
        buttonPanel.add(addButton);
        buttonPanel.add(searchButton);
        buttonPanel.add(viewButton);
        buttonPanel.add(deleteButton);
        buttonPanel.add(clearButton);
        buttonPanel.add(closeButton);

        bottomContainer.add(buttonPanel, BorderLayout.CENTER);

        // Status bar
        statusLabel = new Label(" Ready | Ready to manage parking slots.", Label.LEFT);
        statusLabel.setFont(new Font("Arial", Font.PLAIN, 12));
        statusLabel.setForeground(DARK_TEXT);
        statusLabel.setBackground(new Color(220, 230, 242));
        bottomContainer.add(statusLabel, BorderLayout.SOUTH);

        add(bottomContainer, BorderLayout.SOUTH);

        // ================================================
        // EVENT LISTENERS
        // ================================================
        visualMapButton.addActionListener(this);
        addButton.addActionListener(this);
        searchButton.addActionListener(this);
        viewButton.addActionListener(this);
        deleteButton.addActionListener(this);
        clearButton.addActionListener(this);
        closeButton.addActionListener(this);

        addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                dispose();
```

```
        }
    });

    // Load records immediately into right-side table
    viewAllSlots();

    setLocationRelativeTo(null);
    setVisible(true);
}

private Label createLabel(String text) {
    Label lbl = new Label(text);
    lbl.setFont(new Font("Arial", Font.BOLD, 13));
    lbl.setForeground(DARK_TEXT);
    return lbl;
}

// ========================================================
// ACTION DISPATCHER
// ========================================================
@Override
public void actionPerformed(ActionEvent e) {
    Object src = e.getSource();

    if (src == visualMapButton) {
        new ParkingAvailabilityDashboard();
    } else if (src == addButton) {
        addSlot();
    } else if (src == searchButton) {
        searchSlot();
    } else if (src == viewButton) {
        viewAllSlots();
    } else if (src == deleteButton) {
        deleteSlot();
    } else if (src == clearButton) {
        clearFields();
    } else if (src == closeButton) {
        dispose();
    }
}

// ========================================================
// ADD SLOT
// ========================================================
private void addSlot() {
    String zoneStr = zoneIdField.getText().trim();
    String slotNum = slotNumberField.getText().trim();
    String vehicleType = vehicleTypeChoice.getSelectedItem();
    String slotStatus = slotStatusChoice.getSelectedItem();
```

```
    if (zoneStr.isEmpty() || slotNum.isEmpty()) {
        showMessage("Please fill Zone ID and Slot Number.");
        return;
    }

    try {
        int zoneId = Integer.parseInt(zoneStr);
        ParkingSlot slot = new ParkingSlot();
        String idStr = slotIdField.getText().trim();
        if (!idStr.isEmpty()) {
            try {
                slot.setSlotId(Integer.parseInt(idStr));
            } catch (NumberFormatException ignored) {}
        }
        slot.setZoneId(zoneId);
        slot.setSlotNumber(slotNum);
        slot.setVehicleType(vehicleType);
        slot.setSlotStatus(slotStatus);

        boolean ok = slotDAO.addSlot(slot);
        if (ok) {
            statusLabel.setText("□  Slot "+ slotNum + " Added Successfully!");
            showMessage("Parking Slot Added Successfully!");
            clearFields();
            viewAllSlots();
        } else {
            showMessage("Failed to add parking slot.");
        }
    } catch (NumberFormatException ex) {
        showMessage("Zone ID must be a valid number (e.g. 1, 2, 3).");
    }
}

// ================================================
// SEARCH SLOT
// ================================================
private void searchSlot() {
    String idStr = slotIdField.getText().trim();
    if (idStr.isEmpty()) {
        showMessage("Please enter a Slot ID to search.");
        return;
    }

    try {
        int slotId = Integer.parseInt(idStr);
        ParkingSlot slot = slotDAO.searchSlot(slotId);

        if (slot != null) {
            zoneIdField.setText(String.valueOf(slot.getZoneId()));
            slotNumberField.setText(slot.getSlotNumber());
```

```
            vehicleTypeChoice.select(slot.getVehicleType());
            slotStatusChoice.select(slot.getSlotStatus());
                statusLabel.setText("□ Found Slot " + slot.getSlotNumber() + " (ID: " +
slot.getSlotId() + ")");
            } else {
                showMessage("No parking slot found with ID: " + slotId);
            }
        } catch (NumberFormatException ex) {
            showMessage("Slot ID must be a valid integer.");
        }
    }

    // ========================================================
    // VIEW ALL SLOTS
    // ========================================================
    private void viewAllSlots() {
        List<ParkingSlot> slots = slotDAO.getAllSlots();
        resultArea.setText("");

         resultArea.append("=============================================
==========================\n");
        resultArea.append("                CAMPUS PARKING SLOTS INVENTORY\n");
         resultArea.append("=============================================
==========================\n");
         resultArea.append(String.format("%-8s %-10s %-14s %-18s %-12s%n", "SLOT ID",
"ZONE ID", "SLOT CODE", "VEHICLE TYPE", "STATUS"));
        resultArea.append("-------------------------------------------------------------------------\n");

        if (slots == null || slots.isEmpty()) {
            resultArea.append("\nNo parking slot records found.\n");
            return;
        }

        int availCount = 0;
        for (ParkingSlot s : slots) {
            if ("Available".equalsIgnoreCase(s.getSlotStatus())) availCount++;
            resultArea.append(String.format("%-8d %-10d %-14s %-18s %-12s%n",
                    s.getSlotId(),
                    s.getZoneId(),
                    s.getSlotNumber(),
                    s.getVehicleType(),
                    s.getSlotStatus()
            ));
        }
        resultArea.append("-------------------------------------------------------------------------\n");
            resultArea.append("Total Slots: " + slots.size() + " | Free Slots Available: " +
availCount + "\n");
         statusLabel.setText(" Loaded " + slots.size() + " total slots (" + availCount + " currently
Available).");
    }
```

```
// ========================================================
// DELETE SLOT
// ========================================================
private void deleteSlot() {
    String idStr = slotIdField.getText().trim();
    if (idStr.isEmpty()) {
        showMessage("Please enter a Slot ID to delete.");
        return;
    }

    try {
        int slotId = Integer.parseInt(idStr);
        boolean ok = slotDAO.deleteSlot(slotId);
        if (ok) {
            statusLabel.setText("□ Slot ID " + slotId + " deleted successfully.");
            showMessage("Slot deleted successfully.");
            clearFields();
            viewAllSlots();
        } else {
            showMessage("Could not delete slot with ID: " + slotId);
        }
    } catch (NumberFormatException ex) {
        showMessage("Slot ID must be a valid integer.");
    }
}

// ========================================================
// CLEAR FIELDS
// ========================================================
private void clearFields() {
    slotIdField.setText(String.valueOf(slotDAO.getNextSlotId()));
    zoneIdField.setText("1");
    slotNumberField.setText("");
    vehicleTypeChoice.select(0);
    slotStatusChoice.select(0);
    statusLabel.setText(" Ready | Next Slot ID auto-assigned: " + slotDAO.getNextSlotId());
}

// ========================================================
// SHOW MESSAGE DIALOG
// ========================================================
private void showMessage(String msg) {
    Dialog d = new Dialog(this, "Notification", true);
    d.setSize(360, 160);
    d.setLayout(new BorderLayout(10, 10));
    d.setBackground(LIGHT_BG);

    Label lbl = new Label(msg, Label.CENTER);
    lbl.setFont(new Font("Arial", Font.PLAIN, 13));
```

```
        d.add(lbl, BorderLayout.CENTER);

        AwtButton btnOk = new AwtButton("OK", BLUE, WHITE);
        btnOk.addActionListener(e -> d.dispose());
        Panel p = new Panel(new FlowLayout());
        p.add(btnOk);
        d.add(p, BorderLayout.SOUTH);

        d.addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                d.dispose();
            }
        });

        d.setLocationRelativeTo(this);
        d.setVisible(true);
    }

    public static void main(String[] args) {
        new ParkingSlotManagement();
    }
}
```

ParkingZone.java

```
package parking;

public class ParkingZone {

    private int zoneId;
    private String zoneName;
    private String location;
    private String zoneType;
    private int totalSlots;
    private int availableSlots;
    private String status;

    public ParkingZone() {
    }

    public ParkingZone(int zoneId, String zoneName, String location,
                       String zoneType, int totalSlots,
                       int availableSlots, String status) {

        this.zoneId = zoneId;
        this.zoneName = zoneName;
        this.location = location;
        this.zoneType = zoneType;
        this.totalSlots = totalSlots;
```

```
        this.availableSlots = availableSlots;
        this.status = status;
    }

    public int getZoneId() {
        return zoneId;
    }

    public void setZoneId(int zoneId) {
        this.zoneId = zoneId;
    }

    public String getZoneName() {
        return zoneName;
    }

    public void setZoneName(String zoneName) {
        this.zoneName = zoneName;
    }

    public String getLocation() {
        return location;
    }

    public void setLocation(String location) {
        this.location = location;
    }

    public String getZoneType() {
        return zoneType;
    }

    public void setZoneType(String zoneType) {
        this.zoneType = zoneType;
    }

    public int getTotalSlots() {
        return totalSlots;
    }

    public void setTotalSlots(int totalSlots) {
        this.totalSlots = totalSlots;
    }

    public int getAvailableSlots() {
        return availableSlots;
    }

    public void setAvailableSlots(int availableSlots) {
        this.availableSlots = availableSlots;
```

```
    }

    public String getStatus() {
        return status;
    }

    public void setStatus(String status) {
        this.status = status;
    }
}
```

ParkingZoneDAO.java

```
package parking;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.CopyOnWriteArrayList;

public class ParkingZoneDAO {

    private static final List<ParkingZone> demoZones = new CopyOnWriteArrayList<>();

    static {
        initDemoZones();
    }

    private static void initDemoZones() {
        if (!demoZones.isEmpty()) return;
          demoZones.add(new ParkingZone(1, "Main Gate Parking", "Near Gate 1 - Campus Entrance", "Two & Four Wheeler", 8, 4, "Active"));
           demoZones.add(new ParkingZone(2, "Science Block Parking", "Behind Science & Research Center", "Faculty & Student", 6, 3, "Active"));
        demoZones.add(new ParkingZone(3, "Admin Block Parking", "Opposite Administrative Building", "VIP & Staff", 6, 3, "Active"));
        demoZones.add(new ParkingZone(4, "Academic Block Parking", "Next to Main Library & Lecture Halls", "Student Only", 6, 4, "Active"));
    }

    // GET NEXT AUTO-GENERATED ZONE ID
    public int getNextZoneId() {
        String sql = "SELECT COALESCE(MAX(zone_id), 0) + 1 FROM parking_zones";
        Connection con = DBConnection.getConnection();
        if (con != null) {
            try (con; PreparedStatement ps = con.prepareStatement(sql);
                 ResultSet rs = ps.executeQuery()) {
```

```
            if (rs.next()) {
                return rs.getInt(1);
            }
        } catch (SQLException ignored) {}
    }
    int max = 0;
    for (ParkingZone z : demoZones) {
        if (z.getZoneId() > max) max = z.getZoneId();
    }
    return max + 1;
}

// ==========================================
// ADD PARKING ZONE
// ==========================================

public boolean addZone(ParkingZone zone) {

    String sql = "INSERT INTO parking_zones "
               + "(zone_name, location, zone_type, total_slots, "
               + "available_slots, status) "
               + "VALUES (?, ?, ?, ?, ?, ?)";

    Connection con = DBConnection.getConnection();
    if (con != null) {
        try (con; PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setString(1, zone.getZoneName());
            ps.setString(2, zone.getLocation());
            ps.setString(3, zone.getZoneType());
            ps.setInt(4, zone.getTotalSlots());
            ps.setInt(5, zone.getAvailableSlots());
            ps.setString(6, zone.getStatus());

            int rows = ps.executeUpdate();
            return rows > 0;

        } catch (SQLException e) {
            System.out.println("Error adding parking zone in DB: " + e.getMessage());
        }
    }

    if (zone.getZoneId() == 0) {
        zone.setZoneId(demoZones.size() + 1);
    }
    demoZones.add(zone);
    return true;
}
```

```
// ========================================
// SEARCH PARKING ZONE BY ID
// ========================================

public ParkingZone searchZone(int zoneId) {

    String sql = "SELECT * FROM parking_zones WHERE zone_id = ?";

    Connection con = DBConnection.getConnection();
    if (con != null) {
        try (con; PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, zoneId);

            ResultSet rs = ps.executeQuery();
            if (rs.next()) {
                return createZoneFromResultSet(rs);
            }

        } catch (SQLException e) {
            System.out.println("Error searching parking zone in DB: " + e.getMessage());
        }
    }

    for (ParkingZone z : demoZones) {
        if (z.getZoneId() == zoneId) return z;
    }

    return null;
}

public ParkingZone getZoneById(int zoneId) {
    return searchZone(zoneId);
}


// ========================================
// UPDATE PARKING ZONE
// ========================================

public boolean updateZone(ParkingZone zone) {

    String sql = "UPDATE parking_zones SET "
               + "zone_name = ?, "
               + "location = ?, "
               + "zone_type = ?, "
               + "total_slots = ?, "
               + "available_slots = ?, "
               + "status = ? "
               + "WHERE zone_id = ?";
```

```
    Connection con = DBConnection.getConnection();
    if (con != null) {
        try (con; PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setString(1, zone.getZoneName());
            ps.setString(2, zone.getLocation());
            ps.setString(3, zone.getZoneType());
            ps.setInt(4, zone.getTotalSlots());
            ps.setInt(5, zone.getAvailableSlots());
            ps.setString(6, zone.getStatus());
            ps.setInt(7, zone.getZoneId());

            int rows = ps.executeUpdate();
            return rows > 0;

        } catch (SQLException e) {
            System.out.println("Error updating parking zone in DB: " + e.getMessage());
        }
    }

    for (int i = 0; i < demoZones.size(); i++) {
        if (demoZones.get(i).getZoneId() == zone.getZoneId()) {
            demoZones.set(i, zone);
            return true;
        }
    }

    return false;
}


// ========================================
// DELETE PARKING ZONE
// ========================================

public boolean deleteZone(int zoneId) {

    String sql = "DELETE FROM parking_zones WHERE zone_id = ?";

    Connection con = DBConnection.getConnection();
    if (con != null) {
        try (con; PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, zoneId);

            int rows = ps.executeUpdate();
            return rows > 0;

        } catch (SQLException e) {
```

```
            System.out.println("Error deleting parking zone in DB: " + e.getMessage());
        }
    }

    return demoZones.removeIf(z -> z.getZoneId() == zoneId);
}


// ========================================
// VIEW ALL PARKING ZONES
// ========================================

public List<ParkingZone> getAllZones() {

    List<ParkingZone> zones = new ArrayList<>();
    String sql = "SELECT * FROM parking_zones ORDER BY zone_id";

    Connection con = DBConnection.getConnection();
    if (con != null) {
        try (con;
             PreparedStatement ps = con.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {

            while (rs.next()) {
                zones.add(createZoneFromResultSet(rs));
            }

            if (!zones.isEmpty()) {
                return zones;
            }

        } catch (SQLException e) {
            System.out.println("Error retrieving parking zones from DB: " + e.getMessage());
        }
    }

    return new ArrayList<>(demoZones);
}


// ========================================
// CREATE OBJECT FROM RESULT SET
// ========================================

private ParkingZone createZoneFromResultSet(ResultSet rs)
        throws SQLException {

    ParkingZone zone = new ParkingZone();

    zone.setZoneId(rs.getInt("zone_id"));
```

```
        zone.setZoneName(rs.getString("zone_name"));
        zone.setLocation(rs.getString("location"));
        zone.setZoneType(rs.getString("zone_type"));
        zone.setTotalSlots(rs.getInt("total_slots"));
        zone.setAvailableSlots(rs.getInt("available_slots"));
        zone.setStatus(rs.getString("status"));

        return zone;
    }
}
```

ParkingZoneManagement.java

```
package parking;

import java.awt.*;
import java.awt.event.*;
import java.util.List;

public class ParkingZoneManagement extends Frame implements ActionListener {

    // COLORS
    private final Color DARK_BLUE  = new Color(25, 55, 95);
    private final Color BLUE       = new Color(45, 95, 150);
    private final Color LIGHT_BG   = new Color(245, 247, 250);
    private final Color DARK_TEXT  = new Color(30, 41, 59);
    private final Color WHITE      = Color.WHITE;

    private final Color GREEN      = new Color(40, 140, 85);
    private final Color ORANGE     = new Color(230, 145, 40);
    private final Color RED        = new Color(190, 65, 65);
    private final Color GRAY       = new Color(100, 116, 139);
    private final Color CHARCOAL   = new Color(51, 65, 85);

    // FORM INPUTS
    private TextField zoneIdField;
    private TextField zoneNameField;
    private TextField locationField;
    private TextField totalSlotsField;
    private TextField availableSlotsField;
    private Choice zoneTypeChoice;
    private Choice statusChoice;

    // CUSTOM AWT BUTTONS
    private AwtButton addButton;
    private AwtButton searchButton;
    private AwtButton updateButton;
    private AwtButton deleteButton;
    private AwtButton viewButton;
```

```
    private AwtButton clearButton;
    private AwtButton closeButton;

    private TextArea output;
    private Label statusLabel;
    private ParkingZoneDAO zoneDAO;

    public ParkingZoneManagement() {

        zoneDAO = new ParkingZoneDAO();

        setTitle("Smart Campus Parking - Parking Zone Management");
        setSize(1040, 740);
        setLayout(new BorderLayout(0, 0));
        setBackground(LIGHT_BG);

        // HEADER
        Panel headerPanel = new Panel(new BorderLayout(5, 5));
        headerPanel.setBackground(DARK_BLUE);

        Panel titleBox = new Panel(new GridLayout(2, 1));
        titleBox.setBackground(DARK_BLUE);

        Label title = new Label("PARKING ZONE MANAGEMENT", Label.CENTER);
        title.setFont(new Font("Arial", Font.BOLD, 22));
        title.setForeground(WHITE);
        titleBox.add(title);

        Label subtitle = new Label("Configure Campus Parking Zones, Slot Capacity & Locations", Label.CENTER);
        subtitle.setFont(new Font("Arial", Font.PLAIN, 13));
        subtitle.setForeground(new Color(210, 225, 245));
        titleBox.add(subtitle);

        headerPanel.add(titleBox, BorderLayout.CENTER);
        add(headerPanel, BorderLayout.NORTH);

        // MAIN CONTENT
        Panel mainContent = new Panel(new GridLayout(1, 2, 16, 0));
        mainContent.setBackground(LIGHT_BG);

        // LEFT CARD: FORM
        Panel formCard = new Panel(new BorderLayout(0, 0));
        formCard.setBackground(WHITE);

        Panel formCardHeader = new Panel(new BorderLayout());
        formCardHeader.setBackground(BLUE);
        Label formTitle = new Label("  ZONE CONFIGURATION FORM", Label.LEFT);
        formTitle.setFont(new Font("Arial", Font.BOLD, 14));
        formTitle.setForeground(WHITE);
```

```
        formCardHeader.add(formTitle, BorderLayout.CENTER);
        formCard.add(formCardHeader, BorderLayout.NORTH);

        Panel formBody = new Panel(new GridBagLayout());
        formBody.setBackground(WHITE);
        GridBagConstraints gbc = new GridBagConstraints();
        gbc.insets = new Insets(7, 16, 7, 16);
        gbc.fill = GridBagConstraints.HORIZONTAL;
        gbc.anchor = GridBagConstraints.NORTHWEST;

        // Row 0: Zone ID
        gbc.gridx = 0; gbc.gridy = 0; gbc.weightx = 0.35;
        formBody.add(createLabel("Zone ID (Auto):"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.65;
        zoneIdField = new TextField(String.valueOf(zoneDAO.getNextZoneId()));
        zoneIdField.setFont(new Font("Arial", Font.BOLD, 13));
        formBody.add(zoneIdField, gbc);

        // Row 1: Zone Name
        gbc.gridx = 0; gbc.gridy = 1; gbc.weightx = 0.35;
        formBody.add(createLabel("Zone Name:"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.65;
        zoneNameField = new TextField();
        formBody.add(zoneNameField, gbc);

        // Row 2: Location
        gbc.gridx = 0; gbc.gridy = 2; gbc.weightx = 0.35;
        formBody.add(createLabel("Location / Landmark:"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.65;
        locationField = new TextField();
        formBody.add(locationField, gbc);

        // Row 3: Zone Type
        gbc.gridx = 0; gbc.gridy = 3; gbc.weightx = 0.35;
        formBody.add(createLabel("Zone Type:"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.65;
        zoneTypeChoice = new Choice();
        zoneTypeChoice.add("Two & Four Wheeler");
        zoneTypeChoice.add("Two Wheeler Only");
        zoneTypeChoice.add("Four Wheeler Only");
        zoneTypeChoice.add("Faculty & Student");
        zoneTypeChoice.add("VIP & Staff");
        zoneTypeChoice.add("EV Charging Hub");
        formBody.add(zoneTypeChoice, gbc);

        // Row 4: Total Slots
        gbc.gridx = 0; gbc.gridy = 4; gbc.weightx = 0.35;
        formBody.add(createLabel("Total Slots:"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.65;
        totalSlotsField = new TextField("10");
```

```
        formBody.add(totalSlotsField, gbc);

        // Row 5: Available Slots
        gbc.gridx = 0; gbc.gridy = 5; gbc.weightx = 0.35;
        formBody.add(createLabel("Available Slots:"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.65;
        availableSlotsField = new TextField("10");
        formBody.add(availableSlotsField, gbc);

        // Row 6: Status
        gbc.gridx = 0; gbc.gridy = 6; gbc.weightx = 0.35;
        formBody.add(createLabel("Status:"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.65;
        statusChoice = new Choice();
        statusChoice.add("Active");
        statusChoice.add("Inactive");
        statusChoice.add("Under Maintenance");
        formBody.add(statusChoice, gbc);

        // Row 7: Vertical filler
        gbc.gridx = 0; gbc.gridy = 7; gbc.gridwidth = 2; gbc.weighty = 1.0;
        formBody.add(new Panel(), gbc);

        formCard.add(formBody, BorderLayout.CENTER);
        mainContent.add(formCard);

        // RIGHT CARD: RECORDS
        Panel tableCard = new Panel(new BorderLayout(0, 0));
        tableCard.setBackground(WHITE);

        Panel tableCardHeader = new Panel(new BorderLayout());
        tableCardHeader.setBackground(DARK_BLUE);
            Label tableTitle = new Label("  CAMPUS PARKING ZONES DIRECTORY",
Label.LEFT);
        tableTitle.setFont(new Font("Arial", Font.BOLD, 14));
        tableTitle.setForeground(WHITE);
        tableCardHeader.add(tableTitle, BorderLayout.CENTER);
        tableCard.add(tableCardHeader, BorderLayout.NORTH);

        output = new TextArea("", 20, 50, TextArea.SCROLLBARS_VERTICAL_ONLY);
        output.setFont(new Font("Monospaced", Font.PLAIN, 12));
        output.setEditable(false);
        output.setBackground(new Color(250, 252, 255));
        tableCard.add(output, BorderLayout.CENTER);

        mainContent.add(tableCard);
        add(mainContent, BorderLayout.CENTER);

        // BOTTOM BAR
        Panel bottomContainer = new Panel(new BorderLayout(0, 0));
```

```
        bottomContainer.setBackground(LIGHT_BG);

        Panel buttonPanel = new Panel(new FlowLayout(FlowLayout.CENTER, 10, 10));
        buttonPanel.setBackground(new Color(230, 238, 248));

        addButton    = new AwtButton("□  ADD", GREEN, WHITE);
        searchButton = new AwtButton("□ SEARCH", BLUE, WHITE);
        updateButton = new AwtButton("✏□ UPDATE", ORANGE, WHITE);
        deleteButton = new AwtButton("□ DELETE", RED, WHITE);
        viewButton   = new AwtButton("□ VIEW ALL", DARK_BLUE, WHITE);
        clearButton  = new AwtButton("□ CLEAR", GRAY, WHITE);
        closeButton  = new AwtButton("□  CLOSE", CHARCOAL, WHITE);

        buttonPanel.add(addButton);
        buttonPanel.add(searchButton);
        buttonPanel.add(updateButton);
        buttonPanel.add(deleteButton);
        buttonPanel.add(viewButton);
        buttonPanel.add(clearButton);
        buttonPanel.add(closeButton);

        bottomContainer.add(buttonPanel, BorderLayout.CENTER);

        statusLabel = new Label(" Ready | Parking Zone Management.", Label.LEFT);
        statusLabel.setFont(new Font("Arial", Font.PLAIN, 12));
        statusLabel.setForeground(DARK_TEXT);
        statusLabel.setBackground(new Color(220, 230, 242));
        bottomContainer.add(statusLabel, BorderLayout.SOUTH);

        add(bottomContainer, BorderLayout.SOUTH);

        addButton.addActionListener(this);
        searchButton.addActionListener(this);
        updateButton.addActionListener(this);
        deleteButton.addActionListener(this);
        viewButton.addActionListener(this);
        clearButton.addActionListener(this);
        closeButton.addActionListener(this);

        addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                dispose();
            }
        });

        viewZones();

        setLocationRelativeTo(null);
        setVisible(true);
```

```
}

private Label createLabel(String text) {
    Label lbl = new Label(text);
    lbl.setFont(new Font("Arial", Font.BOLD, 12));
    lbl.setForeground(DARK_TEXT);
    return lbl;
}

@Override
public void actionPerformed(ActionEvent e) {
    Object src = e.getSource();
    if (src == addButton) {
        addZone();
    } else if (src == searchButton) {
        searchZone();
    } else if (src == updateButton) {
        updateZone();
    } else if (src == deleteButton) {
        deleteZone();
    } else if (src == viewButton) {
        viewZones();
    } else if (src == clearButton) {
        clearForm();
    } else if (src == closeButton) {
        dispose();
    }
}

private void addZone() {
    try {
        String name = zoneNameField.getText().trim();
        String loc = locationField.getText().trim();
        String type = zoneTypeChoice.getSelectedItem();
        int total = Integer.parseInt(totalSlotsField.getText().trim());
        int avail = Integer.parseInt(availableSlotsField.getText().trim());
        String status = statusChoice.getSelectedItem();

        if (name.isEmpty()) {
            showMessage("Zone Name is required.");
            return;
        }

        ParkingZone z = new ParkingZone();
        String idStr = zoneIdField.getText().trim();
        if (!idStr.isEmpty()) {
            try {
                z.setZoneId(Integer.parseInt(idStr));
            } catch (NumberFormatException ignored) {}
        }
```

```
            z.setZoneName(name);
            z.setLocation(loc);
            z.setZoneType(type);
            z.setTotalSlots(total);
            z.setAvailableSlots(avail);
            z.setStatus(status);

            boolean ok = zoneDAO.addZone(z);
            if (ok) {
                statusLabel.setText("□ Zone "+ name + " Added Successfully!");
                showMessage("Parking Zone Added Successfully!");
                clearForm();
                viewZones();
            } else {
                showMessage("Failed to add parking zone.");
            }
        } catch (Exception ex) {
            showMessage("Invalid input. Please check slot counts.");
        }
    }

    private void searchZone() {
        try {
            int id = Integer.parseInt(zoneIdField.getText().trim());
            ParkingZone z = zoneDAO.getZoneById(id);
            if (z != null) {
                zoneNameField.setText(z.getZoneName());
                locationField.setText(z.getLocation());
                zoneTypeChoice.select(z.getZoneType());
                totalSlotsField.setText(String.valueOf(z.getTotalSlots()));
                availableSlotsField.setText(String.valueOf(z.getAvailableSlots()));
                statusChoice.select(z.getStatus());
                        statusLabel.setText("□ Found Zone: " + z.getZoneName() + " (ID: " +
z.getZoneId() + ")");
            } else {
                showMessage("No zone found with ID: " + id);
            }
        } catch (Exception ex) {
            showMessage("Please enter a valid numeric Zone ID.");
        }
    }

    private void updateZone() {
        try {
            int id = Integer.parseInt(zoneIdField.getText().trim());
            String name = zoneNameField.getText().trim();
            String loc = locationField.getText().trim();
            String type = zoneTypeChoice.getSelectedItem();
            int total = Integer.parseInt(totalSlotsField.getText().trim());
            int avail = Integer.parseInt(availableSlotsField.getText().trim());
```

```
            String status = statusChoice.getSelectedItem();

            ParkingZone z = new ParkingZone();
            z.setZoneId(id);
            z.setZoneName(name);
            z.setLocation(loc);
            z.setZoneType(type);
            z.setTotalSlots(total);
            z.setAvailableSlots(avail);
            z.setStatus(status);

            boolean ok = zoneDAO.updateZone(z);
            if (ok) {
                statusLabel.setText("✏ Zone ID " + id + " updated successfully.");
                showMessage("Parking Zone Updated Successfully!");
                clearForm();
                viewZones();
            } else {
                showMessage("Failed to update parking zone.");
            }
        } catch (Exception ex) {
            showMessage("Invalid input. Please check numeric values.");
        }
    }

    private void deleteZone() {
        try {
            int id = Integer.parseInt(zoneIdField.getText().trim());
            boolean ok = zoneDAO.deleteZone(id);
            if (ok) {
                statusLabel.setText(" Zone ID " + id + " deleted.");
                showMessage("Parking Zone Deleted Successfully.");
                clearForm();
                viewZones();
            } else {
                showMessage("Could not delete zone ID: " + id);
            }
        } catch (Exception ex) {
            showMessage("Please enter a valid numeric Zone ID.");
        }
    }

    private void viewZones() {
        List<ParkingZone> list = zoneDAO.getAllZones();
        output.setText("");

         output.append("==========================================================
==============================\n");
        output.append("                          CAMPUS PARKING ZONES\n");
```

```
        output.append("==========================================================
==================================\n");
        output.append(String.format("%-5s %-22s %-16s %-7s %-7s %-10s%n", "ID", "ZONE
NAME", "TYPE", "TOTAL", "OPEN", "STATUS"));
            output.append("-----------------------------------------------------------------------------------
\n");

        if (list == null || list.isEmpty()) {
            output.append("\nNo parking zone records found.\n");
            return;
        }

        for (ParkingZone z : list) {
            output.append(String.format("%-5d %-22s %-16s %-7d %-7d %-10s%n",
                    z.getZoneId(),
                    z.getZoneName(),
                    z.getZoneType(),
                    z.getTotalSlots(),
                    z.getAvailableSlots(),
                    z.getStatus()
            ));
        }

            output.append("-----------------------------------------------------------------------------------
\n");
        output.append("Total Zones: " + list.size() + "\n");
        statusLabel.setText(" Loaded " + list.size() + " campus parking zones.");
    }

    private void clearForm() {
        zoneIdField.setText(String.valueOf(zoneDAO.getNextZoneId()));
        zoneNameField.setText("");
        locationField.setText("");
        zoneTypeChoice.select(0);
        totalSlotsField.setText("10");
        availableSlotsField.setText("10");
        statusChoice.select(0);
                statusLabel.setText("  Ready  |  Next  Zone  ID  auto-assigned:  "  +
zoneDAO.getNextZoneId());
    }

    private void showMessage(String msg) {
        Dialog d = new Dialog(this, "Notification", true);
        d.setSize(380, 160);
        d.setLayout(new BorderLayout(10, 10));
        d.setBackground(LIGHT_BG);

        Label lbl = new Label(msg, Label.CENTER);
        lbl.setFont(new Font("Arial", Font.PLAIN, 12));
        d.add(lbl, BorderLayout.CENTER);
```

```
        AwtButton btnOk = new AwtButton("OK", BLUE, WHITE);
        btnOk.addActionListener(e -> d.dispose());
        Panel p = new Panel(new FlowLayout());
        p.add(btnOk);
        d.add(p, BorderLayout.SOUTH);

        d.addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                d.dispose();
            }
        });

        d.setLocationRelativeTo(this);
        d.setVisible(true);
    }

    public static void main(String[] args) {
        new ParkingZoneManagement();
    }
}
```

Payment.java

```
package parking;

import java.sql.Timestamp;

public class Payment {

    private int paymentId;
    private int sessionId;
    private int passId;
    private int userId;
    private double amount;
    private String paymentMethod;
    private Timestamp paymentDate;
    private String paymentStatus;
    private String transactionReference;

    public Payment() {
    }

    public Payment(int paymentId, int sessionId, int passId,
                   int userId, double amount, String paymentMethod,
                   Timestamp paymentDate, String paymentStatus,
                   String transactionReference) {

        this.paymentId = paymentId;
```

```
        this.sessionId = sessionId;
        this.passId = passId;
        this.userId = userId;
        this.amount = amount;
        this.paymentMethod = paymentMethod;
        this.paymentDate = paymentDate;
        this.paymentStatus = paymentStatus;
        this.transactionReference = transactionReference;
    }

    public int getPaymentId() {
        return paymentId;
    }

    public void setPaymentId(int paymentId) {
        this.paymentId = paymentId;
    }

    public int getSessionId() {
        return sessionId;
    }

    public void setSessionId(int sessionId) {
        this.sessionId = sessionId;
    }

    public int getPassId() {
        return passId;
    }

    public void setPassId(int passId) {
        this.passId = passId;
    }

    public int getUserId() {
        return userId;
    }

    public void setUserId(int userId) {
        this.userId = userId;
    }

    public double getAmount() {
        return amount;
    }

    public void setAmount(double amount) {
        this.amount = amount;
    }
```

```
    public String getPaymentMethod() {
        return paymentMethod;
    }

    public void setPaymentMethod(String paymentMethod) {
        this.paymentMethod = paymentMethod;
    }

    public Timestamp getPaymentDate() {
        return paymentDate;
    }

    public void setPaymentDate(Timestamp paymentDate) {
        this.paymentDate = paymentDate;
    }

    public String getPaymentStatus() {
        return paymentStatus;
    }

    public void setPaymentStatus(String paymentStatus) {
        this.paymentStatus = paymentStatus;
    }

    public String getTransactionReference() {
        return transactionReference;
    }

    public void setTransactionReference(String transactionReference) {
        this.transactionReference = transactionReference;
    }
}
```

PaymentDAO.java

```
package parking;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Timestamp;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.CopyOnWriteArrayList;

public class PaymentDAO {

    private static final List<Payment> demoPayments = new CopyOnWriteArrayList<>();
```

```
    static {
        initDemoPayments();
    }

    private static void initDemoPayments() {
        if (!demoPayments.isEmpty()) return;
        long now = System.currentTimeMillis();
          demoPayments.add(new Payment(1, 1, 0, 101, 60.0, "UPI", new Timestamp(now -
3600000), "Success", "UPI/TXN984201"));
         demoPayments.add(new Payment(2, 2, 0, 102, 30.0, "Fastag", new Timestamp(now -
1800000), "Success", "FTAG/TXN552190"));
              demoPayments.add(new Payment(3, 0, 1, 103, 500.0, "Credit Card", new
Timestamp(now - 86400000), "Success", "CC/TXN110943"));
    }

    // GET NEXT AUTO-GENERATED PAYMENT ID
    public int getNextPaymentId() {
        String sql = "SELECT COALESCE(MAX(payment_id), 0) + 1 FROM payments";
        Connection con = DBConnection.getConnection();
        if (con != null) {
            try (con; PreparedStatement ps = con.prepareStatement(sql);
                 ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    return rs.getInt(1);
                }
            } catch (SQLException ignored) {}
        }
        int max = 0;
        for (Payment p : demoPayments) {
            if (p.getPaymentId() > max) max = p.getPaymentId();
        }
        return max + 1;
    }

    // ADD PAYMENT
    public boolean addPayment(Payment payment) {

        String sql = "INSERT INTO payments "
                   + "(session_id, pass_id, user_id, amount, "
                   + "payment_method, payment_status, transaction_reference) "
                   + "VALUES (?, ?, ?, ?, ?, ?, ?)";

        Connection con = DBConnection.getConnection();
        if (con != null) {
            try (con; PreparedStatement ps = con.prepareStatement(sql)) {

                ps.setInt(1, payment.getSessionId());

                if (payment.getPassId() > 0) {
                    ps.setInt(2, payment.getPassId());
```

```
            } else {
                ps.setNull(2, java.sql.Types.INTEGER);
            }

            ps.setInt(3, payment.getUserId());
            ps.setDouble(4, payment.getAmount());
            ps.setString(5, payment.getPaymentMethod());
            ps.setString(6, payment.getPaymentStatus());
            ps.setString(7, payment.getTransactionReference());

            int rows = ps.executeUpdate();
            return rows > 0;

        } catch (SQLException e) {
            System.out.println("Error adding payment in DB: " + e.getMessage());
        }
    }

    if (payment.getPaymentId() == 0) {
        payment.setPaymentId(demoPayments.size() + 1);
    }
    demoPayments.add(payment);
    return true;
}


// SEARCH PAYMENT BY ID
public Payment searchPayment(int paymentId) {

    String sql = "SELECT * FROM payments WHERE payment_id = ?";

    Connection con = DBConnection.getConnection();
    if (con != null) {
        try (con; PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, paymentId);
            ResultSet rs = ps.executeQuery();

            if (rs.next()) {
                Payment payment = new Payment();
                payment.setPaymentId(rs.getInt("payment_id"));
                payment.setSessionId(rs.getInt("session_id"));
                payment.setPassId(rs.getInt("pass_id"));
                payment.setUserId(rs.getInt("user_id"));
                payment.setAmount(rs.getDouble("amount"));
                payment.setPaymentMethod(rs.getString("payment_method"));
                payment.setPaymentDate(rs.getTimestamp("payment_date"));
                payment.setPaymentStatus(rs.getString("payment_status"));
                payment.setTransactionReference(rs.getString("transaction_reference"));
                return payment;
```

```
            }

        } catch (SQLException e) {
            System.out.println("Error searching payment in DB: " + e.getMessage());
        }
    }

    for (Payment p : demoPayments) {
        if (p.getPaymentId() == paymentId) return p;
    }

    return null;
}

// GET ALL PAYMENTS
public List<Payment> getAllPayments() {

    List<Payment> list = new ArrayList<>();
    String sql = "SELECT * FROM payments ORDER BY payment_id";

    Connection con = DBConnection.getConnection();
    if (con != null) {
        try (con;
             PreparedStatement ps = con.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {

            while (rs.next()) {
                Payment p = new Payment();
                p.setPaymentId(rs.getInt("payment_id"));
                p.setSessionId(rs.getInt("session_id"));
                p.setPassId(rs.getInt("pass_id"));
                p.setUserId(rs.getInt("user_id"));
                p.setAmount(rs.getDouble("amount"));
                p.setPaymentMethod(rs.getString("payment_method"));
                p.setPaymentDate(rs.getTimestamp("payment_date"));
                p.setPaymentStatus(rs.getString("payment_status"));
                p.setTransactionReference(rs.getString("transaction_reference"));
                list.add(p);
            }

            if (!list.isEmpty()) {
                return list;
            }

        } catch (SQLException e) {
            System.out.println("Error retrieving payments from DB: " + e.getMessage());
        }
    }

    return new ArrayList<>(demoPayments);
```

```
    }
}
```

PaymentManagement.java

```
package parking;

import java.awt.*;
import java.awt.event.*;
import java.sql.Timestamp;
import java.util.List;

public class PaymentManagement extends Frame implements ActionListener {

    // COLORS
    private final Color DARK_BLUE  = new Color(25, 55, 95);
    private final Color BLUE       = new Color(45, 95, 150);
    private final Color LIGHT_BG   = new Color(245, 247, 250);
    private final Color DARK_TEXT  = new Color(30, 41, 59);
    private final Color WHITE      = Color.WHITE;

    private final Color GREEN      = new Color(40, 140, 85);
    private final Color GRAY       = new Color(100, 116, 139);
    private final Color CHARCOAL   = new Color(51, 65, 85);

    // FORM INPUTS
    private TextField txtPaymentId;
    private TextField txtSessionId;
    private TextField txtPassId;
    private TextField txtUserId;
    private TextField txtAmount;
    private TextField txtTxnRef;
    private Choice methodChoice;
    private Choice statusChoice;

    // CUSTOM AWT BUTTONS
    private AwtButton addButton;
    private AwtButton searchButton;
    private AwtButton viewButton;
    private AwtButton clearButton;
    private AwtButton closeButton;

    private TextArea output;
    private Label statusLabel;
    private PaymentDAO paymentDAO;

    public PaymentManagement() {

        paymentDAO = new PaymentDAO();
```

```
        setTitle("Smart Campus Parking - Payment Management");
        setSize(1040, 740);
        setLayout(new BorderLayout(0, 0));
        setBackground(LIGHT_BG);

        // HEADER
        Panel headerPanel = new Panel(new BorderLayout(5, 5));
        headerPanel.setBackground(DARK_BLUE);

        Panel titleBox = new Panel(new GridLayout(2, 1));
        titleBox.setBackground(DARK_BLUE);

        Label title = new Label("PAYMENT MANAGEMENT", Label.CENTER);
        title.setFont(new Font("Arial", Font.BOLD, 22));
        title.setForeground(WHITE);
        titleBox.add(title);

            Label subtitle = new Label("Process Parking Fees, Subscriptions & Payment
Transactions", Label.CENTER);
        subtitle.setFont(new Font("Arial", Font.PLAIN, 13));
        subtitle.setForeground(new Color(210, 225, 245));
        titleBox.add(subtitle);

        headerPanel.add(titleBox, BorderLayout.CENTER);
        add(headerPanel, BorderLayout.NORTH);

        // MAIN CONTENT
        Panel mainContent = new Panel(new GridLayout(1, 2, 16, 0));
        mainContent.setBackground(LIGHT_BG);

        // LEFT CARD: FORM
        Panel formCard = new Panel(new BorderLayout(0, 0));
        formCard.setBackground(WHITE);

        Panel formCardHeader = new Panel(new BorderLayout());
        formCardHeader.setBackground(BLUE);
        Label formTitle = new Label("  TRANSACTION DETAILS", Label.LEFT);
        formTitle.setFont(new Font("Arial", Font.BOLD, 14));
        formTitle.setForeground(WHITE);
        formCardHeader.add(formTitle, BorderLayout.CENTER);
        formCard.add(formCardHeader, BorderLayout.NORTH);

        Panel formBody = new Panel(new GridBagLayout());
        formBody.setBackground(WHITE);
        GridBagConstraints gbc = new GridBagConstraints();
        gbc.insets = new Insets(6, 16, 6, 16);
        gbc.fill = GridBagConstraints.HORIZONTAL;
        gbc.anchor = GridBagConstraints.NORTHWEST;

        gbc.gridx = 0; gbc.gridy = 0; gbc.weightx = 0.35;
```

```
        formBody.add(createLabel("Payment ID (Auto):"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.65;
        txtPaymentId = new TextField(String.valueOf(paymentDAO.getNextPaymentId()));
        txtPaymentId.setFont(new Font("Arial", Font.BOLD, 13));
        formBody.add(txtPaymentId, gbc);

        gbc.gridx = 0; gbc.gridy = 1; gbc.weightx = 0.35;
        formBody.add(createLabel("Session ID:"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.65;
        txtSessionId = new TextField("1");
        formBody.add(txtSessionId, gbc);

        gbc.gridx = 0; gbc.gridy = 2; gbc.weightx = 0.35;
        formBody.add(createLabel("Pass ID (Optional):"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.65;
        txtPassId = new TextField("0");
        formBody.add(txtPassId, gbc);

        gbc.gridx = 0; gbc.gridy = 3; gbc.weightx = 0.35;
        formBody.add(createLabel("User ID:"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.65;
        txtUserId = new TextField("101");
        formBody.add(txtUserId, gbc);

        gbc.gridx = 0; gbc.gridy = 4; gbc.weightx = 0.35;
        formBody.add(createLabel("Amount (₹):"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.65;
        txtAmount = new TextField("60.0");
        formBody.add(txtAmount, gbc);

        gbc.gridx = 0; gbc.gridy = 5; gbc.weightx = 0.35;
        formBody.add(createLabel("Method:"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.65;
        methodChoice = new Choice();
        methodChoice.add("UPI");
        methodChoice.add("Fastag");
        methodChoice.add("Credit Card");
        methodChoice.add("Debit Card");
        methodChoice.add("Cash");
        formBody.add(methodChoice, gbc);

        gbc.gridx = 0; gbc.gridy = 6; gbc.weightx = 0.35;
        formBody.add(createLabel("Status:"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.65;
        statusChoice = new Choice();
        statusChoice.add("Success");
        statusChoice.add("Pending");
        statusChoice.add("Failed");
        formBody.add(statusChoice, gbc);
```

```
        gbc.gridx = 0; gbc.gridy = 7; gbc.weightx = 0.35;
        formBody.add(createLabel("Txn Ref:"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.65;
        txtTxnRef = new TextField("UPI/TXN" + System.currentTimeMillis() % 1000000);
        formBody.add(txtTxnRef, gbc);

        gbc.gridx = 0; gbc.gridy = 8; gbc.gridwidth = 2; gbc.weighty = 1.0;
        formBody.add(new Panel(), gbc);

        formCard.add(formBody, BorderLayout.CENTER);
        mainContent.add(formCard);

        // RIGHT CARD: RECORDS
        Panel tableCard = new Panel(new BorderLayout(0, 0));
        tableCard.setBackground(WHITE);

        Panel tableCardHeader = new Panel(new BorderLayout());
        tableCardHeader.setBackground(DARK_BLUE);
        Label tableTitle = new Label("  PAYMENT TRANSACTIONS", Label.LEFT);
        tableTitle.setFont(new Font("Arial", Font.BOLD, 14));
        tableTitle.setForeground(WHITE);
        tableCardHeader.add(tableTitle, BorderLayout.CENTER);
        tableCard.add(tableCardHeader, BorderLayout.NORTH);

        output = new TextArea("", 20, 50, TextArea.SCROLLBARS_VERTICAL_ONLY);
        output.setFont(new Font("Monospaced", Font.PLAIN, 12));
        output.setEditable(false);
        output.setBackground(new Color(250, 252, 255));
        tableCard.add(output, BorderLayout.CENTER);

        mainContent.add(tableCard);
        add(mainContent, BorderLayout.CENTER);

        // BOTTOM BAR
        Panel bottomContainer = new Panel(new BorderLayout(0, 0));
        bottomContainer.setBackground(LIGHT_BG);

        Panel buttonPanel = new Panel(new FlowLayout(FlowLayout.CENTER, 12, 10));
        buttonPanel.setBackground(new Color(230, 238, 248));

        addButton    = new AwtButton("□ MAKE PAYMENT", GREEN, WHITE);
        searchButton = new AwtButton("□ SEARCH", BLUE, WHITE);
        viewButton   = new AwtButton("□ VIEW ALL", DARK_BLUE, WHITE);
        clearButton  = new AwtButton("□ CLEAR", GRAY, WHITE);
        closeButton  = new AwtButton("□  CLOSE", CHARCOAL, WHITE);

        buttonPanel.add(addButton);
        buttonPanel.add(searchButton);
        buttonPanel.add(viewButton);
        buttonPanel.add(clearButton);
```

```
    buttonPanel.add(closeButton);

    bottomContainer.add(buttonPanel, BorderLayout.CENTER);

    statusLabel = new Label(" Ready | Payment Management.", Label.LEFT);
    statusLabel.setFont(new Font("Arial", Font.PLAIN, 12));
    statusLabel.setForeground(DARK_TEXT);
    statusLabel.setBackground(new Color(220, 230, 242));
    bottomContainer.add(statusLabel, BorderLayout.SOUTH);

    add(bottomContainer, BorderLayout.SOUTH);

    addButton.addActionListener(this);
    searchButton.addActionListener(this);
    viewButton.addActionListener(this);
    clearButton.addActionListener(this);
    closeButton.addActionListener(this);

    addWindowListener(new WindowAdapter() {
        @Override
        public void windowClosing(WindowEvent e) {
            dispose();
        }
    });

    viewPayments();

    setLocationRelativeTo(null);
    setVisible(true);
}

private Label createLabel(String text) {
    Label lbl = new Label(text);
    lbl.setFont(new Font("Arial", Font.BOLD, 12));
    lbl.setForeground(DARK_TEXT);
    return lbl;
}

@Override
public void actionPerformed(ActionEvent e) {
    Object src = e.getSource();
    if (src == addButton) {
        makePayment();
    } else if (src == searchButton) {
        searchPayment();
    } else if (src == viewButton) {
        viewPayments();
    } else if (src == clearButton) {
        clearForm();
    } else if (src == closeButton) {
```

```
        dispose();
    }
}

private void makePayment() {
    try {
        int sid = Integer.parseInt(txtSessionId.getText().trim());
        int pid = Integer.parseInt(txtPassId.getText().trim());
        int uid = Integer.parseInt(txtUserId.getText().trim());
        double amt = Double.parseDouble(txtAmount.getText().trim());
        String method = methodChoice.getSelectedItem();
        String status = statusChoice.getSelectedItem();
        String ref = txtTxnRef.getText().trim();

        Payment p = new Payment();
        String idStr = txtPaymentId.getText().trim();
        if (!idStr.isEmpty()) {
            try {
                p.setPaymentId(Integer.parseInt(idStr));
            } catch (NumberFormatException ignored) {}
        }
        p.setSessionId(sid);
        p.setPassId(pid);
        p.setUserId(uid);
        p.setAmount(amt);
        p.setPaymentMethod(method);
        p.setPaymentStatus(status);
        p.setTransactionReference(ref);
        p.setPaymentDate(new Timestamp(System.currentTimeMillis()));

        boolean ok = paymentDAO.addPayment(p);
        if (ok) {
            statusLabel.setText("□  Payment Processed Successfully!");
            showMessage("Payment Processed Successfully!");
            clearForm();
            viewPayments();
        } else {
            showMessage("Payment failed.");
        }
    } catch (Exception ex) {
        showMessage("Invalid input. Please check numbers and amount.");
    }
}

private void searchPayment() {
    try {
        int id = Integer.parseInt(txtPaymentId.getText().trim());
        Payment p = paymentDAO.searchPayment(id);
        if (p != null) {
            txtSessionId.setText(String.valueOf(p.getSessionId()));
```

```
            txtPassId.setText(String.valueOf(p.getPassId()));
            txtUserId.setText(String.valueOf(p.getUserId()));
            txtAmount.setText(String.valueOf(p.getAmount()));
            methodChoice.select(p.getPaymentMethod());
            statusChoice.select(p.getPaymentStatus());
            txtTxnRef.setText(p.getTransactionReference());
            statusLabel.setText("□ Found Payment ID " + id);
        } else {
            showMessage("No payment found with ID: " + id);
        }
    } catch (Exception ex) {
        showMessage("Please enter a valid numeric Payment ID.");
    }
}

private void viewPayments() {
    List<Payment> list = paymentDAO.getAllPayments();
    output.setText("");

     output.append("================================================
================================\n");
    output.append("                        PAYMENT TRANSACTION LOGS\n");
     output.append("================================================
================================\n");
     output.append(String.format("%-6s %-6s %-8s %-10s %-12s %-10s %-18s%n", "PAY
ID", "USER", "AMOUNT", "METHOD", "STATUS", "REF", "DATE"));
         output.append("----------------------------------------------------------------------------------
\n");

    if (list == null || list.isEmpty()) {
        output.append("\nNo payment records found.\n");
        return;
    }

    double totalRevenue = 0.0;
    for (Payment p : list) {
        totalRevenue += p.getAmount();
        output.append(String.format("%-6d %-6d ₹%-7.2f %-10s %-12s %-10s %-18s%n",
                p.getPaymentId(),
                p.getUserId(),
                p.getAmount(),
                p.getPaymentMethod(),
                p.getPaymentStatus(),
                p.getTransactionReference() != null ? p.getTransactionReference() : "N/A",
                  p.getPaymentDate() != null ? p.getPaymentDate().toString().substring(0, 16) :
"N/A"
        ));
    }
         output.append("----------------------------------------------------------------------------------
\n");
```

```
            output.append("Total Payments: " + list.size() + "  |  Total Revenue: ₹" +
String.format("%.2f", totalRevenue) + "\n");
            statusLabel.setText("  Loaded " + list.size() + " transactions (Total: ₹" +
String.format("%.2f", totalRevenue) + ").");
    }

    private void clearForm() {
        txtPaymentId.setText(String.valueOf(paymentDAO.getNextPaymentId()));
        txtSessionId.setText("1");
        txtPassId.setText("0");
        txtUserId.setText("101");
        txtAmount.setText("60.0");
        txtTxnRef.setText("UPI/TXN" + System.currentTimeMillis() % 1000000);
        methodChoice.select(0);
        statusChoice.select(0);
                statusLabel.setText("  Ready  |  Next Payment ID auto-assigned: " +
paymentDAO.getNextPaymentId());
    }

    private void showMessage(String msg) {
        Dialog d = new Dialog(this, "Notification", true);
        d.setSize(380, 160);
        d.setLayout(new BorderLayout(10, 10));
        d.setBackground(LIGHT_BG);

        Label lbl = new Label(msg, Label.CENTER);
        lbl.setFont(new Font("Arial", Font.PLAIN, 12));
        d.add(lbl, BorderLayout.CENTER);

        AwtButton btnOk = new AwtButton("OK", BLUE, WHITE);
        btnOk.addActionListener(e -> d.dispose());
        Panel p = new Panel(new FlowLayout());
        p.add(btnOk);
        d.add(p, BorderLayout.SOUTH);

        d.addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                d.dispose();
            }
        });

        d.setLocationRelativeTo(this);
        d.setVisible(true);
    }

    public static void main(String[] args) {
        new PaymentManagement();
    }
}
```

Reservation.java

```
package parking;

import java.sql.Date;
import java.sql.Timestamp;

public class Reservation {

    private int reservationId;
    private int userId;
    private int vehicleId;
    private int slotId;
    private Date reservationDate;
    private Timestamp startTime;
    private Timestamp endTime;
    private String reservationStatus;
    private Timestamp createdAt;

    public Reservation() {
    }

    public Reservation(int reservationId, int userId, int vehicleId,
                       int slotId, Date reservationDate,
                       Timestamp startTime, Timestamp endTime,
                       String reservationStatus, Timestamp createdAt) {

        this.reservationId = reservationId;
        this.userId = userId;
        this.vehicleId = vehicleId;
        this.slotId = slotId;
        this.reservationDate = reservationDate;
        this.startTime = startTime;
        this.endTime = endTime;
        this.reservationStatus = reservationStatus;
        this.createdAt = createdAt;
    }

    public int getReservationId() {
        return reservationId;
    }

    public void setReservationId(int reservationId) {
        this.reservationId = reservationId;
    }

    public int getUserId() {
        return userId;
    }
```

```
public void setUserId(int userId) {
    this.userId = userId;
}

public int getVehicleId() {
    return vehicleId;
}

public void setVehicleId(int vehicleId) {
    this.vehicleId = vehicleId;
}

public int getSlotId() {
    return slotId;
}

public void setSlotId(int slotId) {
    this.slotId = slotId;
}

public Date getReservationDate() {
    return reservationDate;
}

public void setReservationDate(Date reservationDate) {
    this.reservationDate = reservationDate;
}

public Timestamp getStartTime() {
    return startTime;
}

public void setStartTime(Timestamp startTime) {
    this.startTime = startTime;
}

public Timestamp getEndTime() {
    return endTime;
}

public void setEndTime(Timestamp endTime) {
    this.endTime = endTime;
}

public String getReservationStatus() {
    return reservationStatus;
}

public void setReservationStatus(String reservationStatus) {
```

```
        this.reservationStatus = reservationStatus;
    }

    public Timestamp getCreatedAt() {
        return createdAt;
    }

    public void setCreatedAt(Timestamp createdAt) {
        this.createdAt = createdAt;
    }
}
```

ReservationDAO.java

```
package parking;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Date;
import java.sql.Timestamp;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.CopyOnWriteArrayList;

public class ReservationDAO {

            private static final List<Reservation> demoReservations = new
CopyOnWriteArrayList<>();

    static {
        initDemoReservations();
    }

    private static void initDemoReservations() {
        if (!demoReservations.isEmpty()) return;
        long now = System.currentTimeMillis();
            demoReservations.add(new Reservation(1, 101, 1, 108, new Date(now), new
Timestamp(now), new Timestamp(now + 7200000), "Active", new Timestamp(now)));
            demoReservations.add(new Reservation(2, 102, 2, 301, new Date(now), new
Timestamp(now + 3600000), new Timestamp(now + 10800000), "Confirmed", new
Timestamp(now)));
    }

    // GET NEXT AUTO-GENERATED RESERVATION ID
    public int getNextReservationId() {
        String sql = "SELECT COALESCE(MAX(reservation_id), 0) + 1 FROM reservations";
        Connection con = DBConnection.getConnection();
        if (con != null) {
```

```
        try (con; PreparedStatement ps = con.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {
            if (rs.next()) {
                return rs.getInt(1);
            }
        } catch (SQLException ignored) {}
    }
    int max = 0;
    for (Reservation r : demoReservations) {
        if (r.getReservationId() > max) max = r.getReservationId();
    }
    return max + 1;
}

// ADD RESERVATION
public boolean addReservation(Reservation reservation) {

    String sql = "INSERT INTO reservations "
               + "(user_id, vehicle_id, slot_id, reservation_date, "
               + "start_time, end_time, reservation_status) "
               + "VALUES (?, ?, ?, ?, ?, ?, ?)";

    Connection con = DBConnection.getConnection();
    if (con != null) {
        try (con; PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, reservation.getUserId());
            ps.setInt(2, reservation.getVehicleId());
            ps.setInt(3, reservation.getSlotId());
            ps.setDate(4, reservation.getReservationDate());
            ps.setTimestamp(5, reservation.getStartTime());
            ps.setTimestamp(6, reservation.getEndTime());
            ps.setString(7, reservation.getReservationStatus());

            int rows = ps.executeUpdate();
            return rows > 0;

        } catch (SQLException e) {
            System.out.println("Error adding reservation in DB: " + e.getMessage());
        }
    }

    if (reservation.getReservationId() == 0) {
        reservation.setReservationId(demoReservations.size() + 1);
    }
    demoReservations.add(reservation);
    return true;
}
```

```java
    // SEARCH RESERVATION BY ID
    public Reservation searchReservation(int reservationId) {

        String sql = "SELECT * FROM reservations WHERE reservation_id = ?";

        Connection con = DBConnection.getConnection();
        if (con != null) {
            try (con; PreparedStatement ps = con.prepareStatement(sql)) {

                ps.setInt(1, reservationId);
                ResultSet rs = ps.executeQuery();

                if (rs.next()) {
                    Reservation reservation = new Reservation();
                    reservation.setReservationId(rs.getInt("reservation_id"));
                    reservation.setUserId(rs.getInt("user_id"));
                    reservation.setVehicleId(rs.getInt("vehicle_id"));
                    reservation.setSlotId(rs.getInt("slot_id"));
                    reservation.setReservationDate(rs.getDate("reservation_date"));
                    reservation.setStartTime(rs.getTimestamp("start_time"));
                    reservation.setEndTime(rs.getTimestamp("end_time"));
                    reservation.setReservationStatus(rs.getString("reservation_status"));
                    reservation.setCreatedAt(rs.getTimestamp("created_at"));
                    return reservation;
                }

            } catch (SQLException e) {
                System.out.println("Error searching reservation in DB: " + e.getMessage());
            }
        }

        for (Reservation r : demoReservations) {
            if (r.getReservationId() == reservationId) return r;
        }

        return null;
    }

    // UPDATE RESERVATION
    public boolean updateReservation(Reservation res) {

        String sql = "UPDATE reservations SET user_id = ?, vehicle_id = ?, slot_id = ?, reservation_date = ?, start_time = ?, end_time = ?, reservation_status = ? WHERE reservation_id = ?";

        Connection con = DBConnection.getConnection();
        if (con != null) {
            try (con; PreparedStatement ps = con.prepareStatement(sql)) {

                ps.setInt(1, res.getUserId());
```

```
            ps.setInt(2, res.getVehicleId());
            ps.setInt(3, res.getSlotId());
            ps.setDate(4, res.getReservationDate());
            ps.setTimestamp(5, res.getStartTime());
            ps.setTimestamp(6, res.getEndTime());
            ps.setString(7, res.getReservationStatus());
            ps.setInt(8, res.getReservationId());

            int rows = ps.executeUpdate();
            return rows > 0;

        } catch (SQLException e) {
            System.out.println("Error updating reservation in DB: " + e.getMessage());
        }
    }

    for (int i = 0; i < demoReservations.size(); i++) {
        if (demoReservations.get(i).getReservationId() == res.getReservationId()) {
            demoReservations.set(i, res);
            return true;
        }
    }

    return false;
}

// DELETE RESERVATION
public boolean deleteReservation(int reservationId) {

    String sql = "DELETE FROM reservations WHERE reservation_id = ?";

    Connection con = DBConnection.getConnection();
    if (con != null) {
        try (con; PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, reservationId);
            int rows = ps.executeUpdate();
            return rows > 0;

        } catch (SQLException e) {
            System.out.println("Error deleting reservation in DB: " + e.getMessage());
        }
    }

    return demoReservations.removeIf(r -> r.getReservationId() == reservationId);
}

// GET ALL RESERVATIONS
public List<Reservation> getAllReservations() {
```

```
        List<Reservation> list = new ArrayList<>();
        String sql = "SELECT * FROM reservations ORDER BY reservation_id";

        Connection con = DBConnection.getConnection();
        if (con != null) {
            try (con;
                 PreparedStatement ps = con.prepareStatement(sql);
                 ResultSet rs = ps.executeQuery()) {

                while (rs.next()) {
                    Reservation res = new Reservation();
                    res.setReservationId(rs.getInt("reservation_id"));
                    res.setUserId(rs.getInt("user_id"));
                    res.setVehicleId(rs.getInt("vehicle_id"));
                    res.setSlotId(rs.getInt("slot_id"));
                    res.setReservationDate(rs.getDate("reservation_date"));
                    res.setStartTime(rs.getTimestamp("start_time"));
                    res.setEndTime(rs.getTimestamp("end_time"));
                    res.setReservationStatus(rs.getString("reservation_status"));
                    res.setCreatedAt(rs.getTimestamp("created_at"));
                    list.add(res);
                }

                if (!list.isEmpty()) {
                    return list;
                }

            } catch (SQLException e) {
                System.out.println("Error retrieving reservations from DB: " + e.getMessage());
            }
        }

        return new ArrayList<>(demoReservations);
    }
}
```

ReservationManagement.java

```
package parking;

import java.awt.*;
import java.awt.event.*;
import java.sql.Date;
import java.sql.Timestamp;
import java.util.List;

public class ReservationManagement extends Frame implements ActionListener {

    // COLORS
    private final Color DARK_BLUE  = new Color(25, 55, 95);
```

```
    private final Color BLUE       = new Color(45, 95, 150);
    private final Color LIGHT_BG   = new Color(245, 247, 250);
    private final Color DARK_TEXT  = new Color(30, 41, 59);
    private final Color WHITE      = Color.WHITE;

    private final Color GREEN      = new Color(40, 140, 85);
    private final Color ORANGE     = new Color(230, 145, 40);
    private final Color RED        = new Color(190, 65, 65);
    private final Color GRAY       = new Color(100, 116, 139);
    private final Color CHARCOAL   = new Color(51, 65, 85);

    // FORM INPUTS
    private TextField txtReservationId;
    private TextField txtUserId;
    private TextField txtVehicleId;
    private TextField txtSlotId;
    private TextField txtDate;
    private TextField txtStartTime;
    private TextField txtEndTime;
    private Choice statusChoice;

    // CUSTOM AWT BUTTONS
    private AwtButton addButton;
    private AwtButton searchButton;
    private AwtButton updateButton;
    private AwtButton deleteButton;
    private AwtButton viewButton;
    private AwtButton clearButton;
    private AwtButton closeButton;

    private TextArea output;
    private Label statusLabel;
    private ReservationDAO resDAO;

    public ReservationManagement() {

        resDAO = new ReservationDAO();

        setTitle("Smart Campus Parking - Reservation Management");
        setSize(1040, 740);
        setLayout(new BorderLayout(0, 0));
        setBackground(LIGHT_BG);

        // HEADER
        Panel headerPanel = new Panel(new BorderLayout(5, 5));
        headerPanel.setBackground(DARK_BLUE);

        Panel titleBox = new Panel(new GridLayout(2, 1));
        titleBox.setBackground(DARK_BLUE);
```

```
        Label title = new Label("RESERVATION MANAGEMENT", Label.CENTER);
        title.setFont(new Font("Arial", Font.BOLD, 22));
        title.setForeground(WHITE);
        titleBox.add(title);

            Label subtitle = new Label("Book & Manage Advance Campus Parking Slot Reservations", Label.CENTER);
        subtitle.setFont(new Font("Arial", Font.PLAIN, 13));
        subtitle.setForeground(new Color(210, 225, 245));
        titleBox.add(subtitle);

        headerPanel.add(titleBox, BorderLayout.CENTER);
        add(headerPanel, BorderLayout.NORTH);

        // MAIN CONTENT
        Panel mainContent = new Panel(new GridLayout(1, 2, 16, 0));
        mainContent.setBackground(LIGHT_BG);

        // LEFT CARD: FORM
        Panel formCard = new Panel(new BorderLayout(0, 0));
        formCard.setBackground(WHITE);

        Panel formCardHeader = new Panel(new BorderLayout());
        formCardHeader.setBackground(BLUE);
        Label formTitle = new Label("  RESERVATION DETAILS", Label.LEFT);
        formTitle.setFont(new Font("Arial", Font.BOLD, 14));
        formTitle.setForeground(WHITE);
        formCardHeader.add(formTitle, BorderLayout.CENTER);
        formCard.add(formCardHeader, BorderLayout.NORTH);

        Panel formBody = new Panel(new GridBagLayout());
        formBody.setBackground(WHITE);
        GridBagConstraints gbc = new GridBagConstraints();
        gbc.insets = new Insets(6, 16, 6, 16);
        gbc.fill = GridBagConstraints.HORIZONTAL;
        gbc.anchor = GridBagConstraints.NORTHWEST;

        gbc.gridx = 0; gbc.gridy = 0; gbc.weightx = 0.35;
        formBody.add(createLabel("Reservation ID (Auto):"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.65;
        txtReservationId = new TextField(String.valueOf(resDAO.getNextReservationId()));
        txtReservationId.setFont(new Font("Arial", Font.BOLD, 13));
        formBody.add(txtReservationId, gbc);

        gbc.gridx = 0; gbc.gridy = 1; gbc.weightx = 0.35;
        formBody.add(createLabel("User ID:"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.65;
        txtUserId = new TextField("101");
        formBody.add(txtUserId, gbc);
```

```
        gbc.gridx = 0; gbc.gridy = 2; gbc.weightx = 0.35;
        formBody.add(createLabel("Vehicle ID:"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.65;
        txtVehicleId = new TextField("1");
        formBody.add(txtVehicleId, gbc);

        gbc.gridx = 0; gbc.gridy = 3; gbc.weightx = 0.35;
        formBody.add(createLabel("Slot ID:"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.65;
        txtSlotId = new TextField("101");
        formBody.add(txtSlotId, gbc);

        gbc.gridx = 0; gbc.gridy = 2; gbc.weightx = 0.35;
        formBody.add(createLabel("Vehicle ID:"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.65;
        txtVehicleId = new TextField();
        formBody.add(txtVehicleId, gbc);

        gbc.gridx = 0; gbc.gridy = 3; gbc.weightx = 0.35;
        formBody.add(createLabel("Slot ID:"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.65;
        txtSlotId = new TextField();
        formBody.add(txtSlotId, gbc);

        gbc.gridx = 0; gbc.gridy = 4; gbc.weightx = 0.35;
        formBody.add(createLabel("Date (YYYY-MM-DD):"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.65;
        txtDate = new TextField("2026-09-01");
        formBody.add(txtDate, gbc);

        gbc.gridx = 0; gbc.gridy = 5; gbc.weightx = 0.35;
        formBody.add(createLabel("Start Time:"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.65;
        txtStartTime = new TextField("2026-09-01 09:00:00");
        formBody.add(txtStartTime, gbc);

        gbc.gridx = 0; gbc.gridy = 6; gbc.weightx = 0.35;
        formBody.add(createLabel("End Time:"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.65;
        txtEndTime = new TextField("2026-09-01 12:00:00");
        formBody.add(txtEndTime, gbc);

        gbc.gridx = 0; gbc.gridy = 7; gbc.weightx = 0.35;
        formBody.add(createLabel("Status:"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.65;
        statusChoice = new Choice();
        statusChoice.add("Active");
        statusChoice.add("Confirmed");
        statusChoice.add("Cancelled");
        statusChoice.add("Completed");
```

```
        formBody.add(statusChoice, gbc);

        gbc.gridx = 0; gbc.gridy = 8; gbc.gridwidth = 2; gbc.weighty = 1.0;
        formBody.add(new Panel(), gbc);

        formCard.add(formBody, BorderLayout.CENTER);
        mainContent.add(formCard);

        // RIGHT CARD: RECORDS
        Panel tableCard = new Panel(new BorderLayout(0, 0));
        tableCard.setBackground(WHITE);

        Panel tableCardHeader = new Panel(new BorderLayout());
        tableCardHeader.setBackground(DARK_BLUE);
        Label tableTitle = new Label("  CAMPUS RESERVATION RECORDS", Label.LEFT);
        tableTitle.setFont(new Font("Arial", Font.BOLD, 14));
        tableTitle.setForeground(WHITE);
        tableCardHeader.add(tableTitle, BorderLayout.CENTER);
        tableCard.add(tableCardHeader, BorderLayout.NORTH);

        output = new TextArea("", 20, 50, TextArea.SCROLLBARS_VERTICAL_ONLY);
        output.setFont(new Font("Monospaced", Font.PLAIN, 12));
        output.setEditable(false);
        output.setBackground(new Color(250, 252, 255));
        tableCard.add(output, BorderLayout.CENTER);

        mainContent.add(tableCard);
        add(mainContent, BorderLayout.CENTER);

        // BOTTOM BAR
        Panel bottomContainer = new Panel(new BorderLayout(0, 0));
        bottomContainer.setBackground(LIGHT_BG);

        Panel buttonPanel = new Panel(new FlowLayout(FlowLayout.CENTER, 10, 10));
        buttonPanel.setBackground(new Color(230, 238, 248));

        addButton    = new AwtButton("□  BOOK", GREEN, WHITE);
        searchButton = new AwtButton("□ SEARCH", BLUE, WHITE);
        updateButton = new AwtButton("✏□ UPDATE", ORANGE, WHITE);
        deleteButton = new AwtButton("□ CANCEL", RED, WHITE);
        viewButton   = new AwtButton("□ VIEW ALL", DARK_BLUE, WHITE);
        clearButton  = new AwtButton("□ CLEAR", GRAY, WHITE);
        closeButton  = new AwtButton("□  CLOSE", CHARCOAL, WHITE);

        buttonPanel.add(addButton);
        buttonPanel.add(searchButton);
        buttonPanel.add(updateButton);
        buttonPanel.add(deleteButton);
        buttonPanel.add(viewButton);
        buttonPanel.add(clearButton);
```

```
        buttonPanel.add(closeButton);

        bottomContainer.add(buttonPanel, BorderLayout.CENTER);

        statusLabel = new Label(" Ready | Reservation Management.", Label.LEFT);
        statusLabel.setFont(new Font("Arial", Font.PLAIN, 12));
        statusLabel.setForeground(DARK_TEXT);
        statusLabel.setBackground(new Color(220, 230, 242));
        bottomContainer.add(statusLabel, BorderLayout.SOUTH);

        add(bottomContainer, BorderLayout.SOUTH);

        addButton.addActionListener(this);
        searchButton.addActionListener(this);
        updateButton.addActionListener(this);
        deleteButton.addActionListener(this);
        viewButton.addActionListener(this);
        clearButton.addActionListener(this);
        closeButton.addActionListener(this);

        addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                dispose();
            }
        });

        viewReservations();

        setLocationRelativeTo(null);
        setVisible(true);
    }

    private Label createLabel(String text) {
        Label lbl = new Label(text);
        lbl.setFont(new Font("Arial", Font.BOLD, 12));
        lbl.setForeground(DARK_TEXT);
        return lbl;
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        Object src = e.getSource();
        if (src == addButton) {
            addReservation();
        } else if (src == searchButton) {
            searchReservation();
        } else if (src == updateButton) {
            updateReservation();
        } else if (src == deleteButton) {
```

```
            deleteReservation();
        } else if (src == viewButton) {
            viewReservations();
        } else if (src == clearButton) {
            clearForm();
        } else if (src == closeButton) {
            dispose();
        }
    }

    private void addReservation() {
        try {
            int uid = Integer.parseInt(txtUserId.getText().trim());
            int vid = Integer.parseInt(txtVehicleId.getText().trim());
            int sid = Integer.parseInt(txtSlotId.getText().trim());
            Date d = Date.valueOf(txtDate.getText().trim());
            Timestamp st = Timestamp.valueOf(txtStartTime.getText().trim());
            Timestamp et = Timestamp.valueOf(txtEndTime.getText().trim());
            String status = statusChoice.getSelectedItem();

            Reservation res = new Reservation();
            String idStr = txtReservationId.getText().trim();
            if (!idStr.isEmpty()) {
                try {
                    res.setReservationId(Integer.parseInt(idStr));
                } catch (NumberFormatException ignored) {}
            }
            res.setUserId(uid);
            res.setVehicleId(vid);
            res.setSlotId(sid);
            res.setReservationDate(d);
            res.setStartTime(st);
            res.setEndTime(et);
            res.setReservationStatus(status);

            boolean ok = resDAO.addReservation(res);
            if (ok) {
                statusLabel.setText("□  Reservation Booked Successfully!");
                showMessage("Reservation Booked Successfully!");
                clearForm();
                viewReservations();
            } else {
                showMessage("Failed to book reservation.");
            }
        } catch (Exception ex) {
            showMessage("Invalid input. Please check numbers and date formats (YYYY-MM-
DD, YYYY-MM-DD HH:MM:SS).");
        }
    }
```

```
    private void searchReservation() {
        try {
            int id = Integer.parseInt(txtReservationId.getText().trim());
            Reservation r = resDAO.searchReservation(id);
            if (r != null) {
                txtUserId.setText(String.valueOf(r.getUserId()));
                txtVehicleId.setText(String.valueOf(r.getVehicleId()));
                txtSlotId.setText(String.valueOf(r.getSlotId()));
                                                    if (r.getReservationDate() != null)
txtDate.setText(r.getReservationDate().toString());
                if (r.getStartTime() != null) txtStartTime.setText(r.getStartTime().toString());
                if (r.getEndTime() != null) txtEndTime.setText(r.getEndTime().toString());
                statusChoice.select(r.getReservationStatus());
                statusLabel.setText("□ Found Reservation ID " + id);
            } else {
                showMessage("No reservation found with ID: " + id);
            }
        } catch (Exception ex) {
            showMessage("Please enter a valid numeric Reservation ID.");
        }
    }

    private void updateReservation() {
        try {
            int id = Integer.parseInt(txtReservationId.getText().trim());
            int uid = Integer.parseInt(txtUserId.getText().trim());
            int vid = Integer.parseInt(txtVehicleId.getText().trim());
            int sid = Integer.parseInt(txtSlotId.getText().trim());
            Date d = Date.valueOf(txtDate.getText().trim());
            Timestamp st = Timestamp.valueOf(txtStartTime.getText().trim());
            Timestamp et = Timestamp.valueOf(txtEndTime.getText().trim());
            String status = statusChoice.getSelectedItem();

            Reservation res = new Reservation();
            res.setReservationId(id);
            res.setUserId(uid);
            res.setVehicleId(vid);
            res.setSlotId(sid);
            res.setReservationDate(d);
            res.setStartTime(st);
            res.setEndTime(et);
            res.setReservationStatus(status);

            boolean ok = resDAO.updateReservation(res);
            if (ok) {
                statusLabel.setText("✏□ Reservation ID " + id + " updated successfully.");
                showMessage("Reservation Updated Successfully!");
                clearForm();
                viewReservations();
            } else {
```

```
                showMessage("Failed to update reservation.");
            }
        } catch (Exception ex) {
            showMessage("Invalid input. Check numeric IDs and dates.");
        }
    }

    private void deleteReservation() {
        try {
            int id = Integer.parseInt(txtReservationId.getText().trim());
            boolean ok = resDAO.deleteReservation(id);
            if (ok) {
                statusLabel.setText("□ Reservation ID " + id + " cancelled/deleted.");
                showMessage("Reservation Cancelled/Deleted Successfully.");
                clearForm();
                viewReservations();
            } else {
                showMessage("Could not delete reservation ID: " + id);
            }
        } catch (Exception ex) {
            showMessage("Please enter a valid numeric Reservation ID.");
        }
    }

    private void viewReservations() {
        List<Reservation> list = resDAO.getAllReservations();
        output.setText("");

         output.append("==========================================================
==================================\n");
        output.append("                         CAMPUS RESERVATION LOGS\n");
         output.append("==========================================================
==================================\n");
           output.append(String.format("%-6s %-8s %-8s %-8s %-12s %-10s%n", "RES ID",
"USER", "VEHICLE", "SLOT", "DATE", "STATUS"));
            output.append("----------------------------------------------------------------------------------
\n");

        if (list == null || list.isEmpty()) {
            output.append("\nNo reservation records found.\n");
            return;
        }

        for (Reservation r : list) {
            output.append(String.format("%-6d %-8d %-8d %-8d %-12s %-10s%n",
                    r.getReservationId(),
                    r.getUserId(),
                    r.getVehicleId(),
                    r.getSlotId(),
                    r.getReservationDate() != null ? r.getReservationDate().toString() : "N/A",
```

```
                r.getReservationStatus()
            ));
        }
            output.append("-------------------------------------------------------------------------------
\n");
        output.append("Total Reservations: " + list.size() + "\n");
        statusLabel.setText(" Loaded " + list.size() + " reservations.");
    }

    private void clearForm() {
        txtReservationId.setText(String.valueOf(resDAO.getNextReservationId()));
        txtUserId.setText("101");
        txtVehicleId.setText("1");
        txtSlotId.setText("101");
        txtDate.setText("2026-09-01");
        txtStartTime.setText("2026-09-01 09:00:00");
        txtEndTime.setText("2026-09-01 12:00:00");
        statusChoice.select(0);
                statusLabel.setText(" Ready | Next Reservation ID auto-assigned: " +
resDAO.getNextReservationId());
    }

    private void showMessage(String msg) {
        Dialog d = new Dialog(this, "Notification", true);
        d.setSize(380, 160);
        d.setLayout(new BorderLayout(10, 10));
        d.setBackground(LIGHT_BG);

        Label lbl = new Label(msg, Label.CENTER);
        lbl.setFont(new Font("Arial", Font.PLAIN, 12));
        d.add(lbl, BorderLayout.CENTER);

        AwtButton btnOk = new AwtButton("OK", BLUE, WHITE);
        btnOk.addActionListener(e -> d.dispose());
        Panel p = new Panel(new FlowLayout());
        p.add(btnOk);
        d.add(p, BorderLayout.SOUTH);

        d.addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                d.dispose();
            }
        });

        d.setLocationRelativeTo(this);
        d.setVisible(true);
    }

    public static void main(String[] args) {
```

```
        new ReservationManagement();
    }
}
```

SmartCampusParkingSystem.java

```
package parking;

import java.awt.*;
import java.awt.event.*;

public class SmartCampusParkingSystem extends Frame implements ActionListener {

    private AwtButton availabilityButton;
    private AwtButton userButton;
    private AwtButton vehicleButton;
    private AwtButton zoneButton;
    private AwtButton slotButton;
    private AwtButton reservationButton;
    private AwtButton sessionButton;
    private AwtButton passButton;
    private AwtButton paymentButton;
    private AwtButton violationButton;
    private AwtButton exitButton;

    public SmartCampusParkingSystem() {

        // ========================================
        // WINDOW SETTINGS
        // ========================================

        setTitle("Smart Campus Parking & Traffic Management System");
        setSize(1040, 750);
        setLayout(new BorderLayout(10, 10));
        setBackground(new Color(245, 247, 250));

        // ========================================
        // HEADER
        // ========================================

        Panel headerPanel = new Panel();
        headerPanel.setLayout(new BorderLayout());
        headerPanel.setBackground(new Color(35, 55, 80));

        Label title = new Label(
                "SMART CAMPUS PARKING & TRAFFIC MANAGEMENT SYSTEM",
                Label.CENTER
        );

        title.setFont(new Font("Arial", Font.BOLD, 22));
```

```
        title.setForeground(Color.WHITE);

        headerPanel.add(title, BorderLayout.CENTER);

        Label subtitle = new Label(
                "Unified Campus Parking & Traffic Control Dashboard",
                Label.CENTER
        );

        subtitle.setFont(new Font("Arial", Font.PLAIN, 14));
        subtitle.setForeground(new Color(210, 225, 245));

        headerPanel.add(subtitle, BorderLayout.SOUTH);

        add(headerPanel, BorderLayout.NORTH);

        // ========================================
        // MAIN CONTENT
        // ========================================

        Panel contentPanel = new Panel();
        contentPanel.setLayout(new BorderLayout(12, 12));
        contentPanel.setBackground(new Color(245, 247, 250));

        // ========================================
        // WELCOME SECTION
        // ========================================

        Panel welcomePanel = new Panel();
        welcomePanel.setLayout(new GridLayout(2, 1));
        welcomePanel.setBackground(new Color(245, 247, 250));

        Label welcomeLabel = new Label(
                "Welcome to Smart Campus Parking System",
                Label.CENTER
        );
        welcomeLabel.setFont(new Font("Arial", Font.BOLD, 18));
        welcomePanel.add(welcomeLabel);

        Label instructionLabel = new Label(
                "Select a management module below to manage campus parking activities",
                Label.CENTER
        );
        instructionLabel.setFont(new Font("Arial", Font.PLAIN, 13));
        welcomePanel.add(instructionLabel);

        contentPanel.add(welcomePanel, BorderLayout.NORTH);

        // ========================================
        // BUTTON PANEL
```

```
        // ========================================

        Panel mainPanel = new Panel();
        mainPanel.setLayout(new GridLayout(6, 2, 20, 14));
        mainPanel.setBackground(new Color(245, 247, 250));

        // ========================================
        // CREATE BUTTONS (Pure AWT Custom Buttons)
        // ========================================
        Font btnFont = new Font("Arial", Font.BOLD, 14);

         availabilityButton = new AwtButton("□ LIVE PARKING AVAILABILITY MAP",
new Color(34, 139, 34), Color.WHITE, btnFont);
         slotButton         = new AwtButton("□ PARKING SLOTS", new Color(59, 130, 246),
Color.WHITE, btnFont);

         userButton         = new AwtButton("□ USER MANAGEMENT", new Color(45, 95,
150), Color.WHITE, btnFont);
         vehicleButton      = new AwtButton("□ VEHICLE MANAGEMENT", new Color(14,
116, 144), Color.WHITE, btnFont);

         zoneButton         = new AwtButton("□ PARKING ZONES", new Color(79, 70, 229),
Color.WHITE, btnFont);
         reservationButton  = new AwtButton("□ RESERVATION", new Color(139, 92, 246),
Color.WHITE, btnFont);

         sessionButton      = new AwtButton("□ PARKING SESSION", new Color(168, 85,
247), Color.WHITE, btnFont);
         passButton         = new AwtButton("□ PARKING PASS", new Color(217, 119, 6),
Color.WHITE, btnFont);

         paymentButton      = new AwtButton("□ PAYMENT", new Color(16, 185, 129),
Color.WHITE, btnFont);
         violationButton    = new AwtButton("□ VIOLATIONS", new Color(225, 29, 72),
Color.WHITE, btnFont);

         exitButton         = new AwtButton("□ EXIT SYSTEM", new Color(71, 85, 105),
Color.WHITE, btnFont);

        // ========================================
        // ADD BUTTONS
        // ========================================

        mainPanel.add(availabilityButton);
        mainPanel.add(slotButton);

        mainPanel.add(userButton);
        mainPanel.add(vehicleButton);

        mainPanel.add(zoneButton);
```

```
        mainPanel.add(reservationButton);

        mainPanel.add(sessionButton);
        mainPanel.add(passButton);

        mainPanel.add(paymentButton);
        mainPanel.add(violationButton);

        mainPanel.add(new Panel()); // Spacer
        mainPanel.add(exitButton);

        contentPanel.add(mainPanel, BorderLayout.CENTER);
        add(contentPanel, BorderLayout.CENTER);

        // ========================================
        // FOOTER
        // ========================================

        Panel footerPanel = new Panel();
        footerPanel.setLayout(new BorderLayout());
        footerPanel.setBackground(new Color(35, 55, 80));

        Label footer = new Label(
                "Java AWT  |  JDBC  |  Smart Campus Traffic & Parking System",
                Label.CENTER
        );
        footer.setFont(new Font("Arial", Font.PLAIN, 13));
        footer.setForeground(Color.WHITE);

        footerPanel.add(footer, BorderLayout.CENTER);
        add(footerPanel, BorderLayout.SOUTH);

        // ========================================
        // EVENT LISTENERS
        // ========================================

        availabilityButton.addActionListener(this);
        userButton.addActionListener(this);
        vehicleButton.addActionListener(this);
        zoneButton.addActionListener(this);
        slotButton.addActionListener(this);
        reservationButton.addActionListener(this);
        sessionButton.addActionListener(this);
        passButton.addActionListener(this);
        paymentButton.addActionListener(this);
        violationButton.addActionListener(this);
        exitButton.addActionListener(this);

        // ========================================
        // WINDOW CLOSE
```

```
        // ========================================

        addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                System.exit(0);
            }
        });

        setLocationRelativeTo(null);
        setVisible(true);
    }

    // ========================================
    // BUTTON EVENTS
    // ========================================

    @Override
    public void actionPerformed(ActionEvent e) {
        Object src = e.getSource();

        if (src == availabilityButton) {
            new ParkingAvailabilityDashboard();
        } else if (src == userButton) {
            new UserManagement();
        } else if (src == vehicleButton) {
            new VehicleManagement();
        } else if (src == zoneButton) {
            new ParkingZoneManagement();
        } else if (src == slotButton) {
            new ParkingSlotManagement();
        } else if (src == reservationButton) {
            new ReservationManagement();
        } else if (src == sessionButton) {
            new ParkingSessionManagement();
        } else if (src == passButton) {
            new ParkingPassManagement();
        } else if (src == paymentButton) {
            new PaymentManagement();
        } else if (src == violationButton) {
            new ViolationManagement();
        } else if (src == exitButton) {
            System.exit(0);
        }
    }

    public static void main(String[] args) {
        new SmartCampusParkingSystem();
    }
}
```

TestConnection.java

```
package parking;

import java.sql.Connection;

public class TestConnection {

    public static void main(String[] args) {

        Connection con = DBConnection.getConnection();

        if (con != null) {

            System.out.println("Connection is working!");

            try {
                con.close();
                System.out.println("Connection closed successfully!");
            } catch (Exception e) {
                e.printStackTrace();
            }

        } else {

            System.out.println("Connection failed!");
        }
    }
}
```

TestParkingPassDAO.java

```
package parking;

import java.sql.Date;

public class TestParkingPassDAO {

    public static void main(String[] args) {

        ParkingPass pass = new ParkingPass();

        // Use existing IDs from your database
        pass.setUserId(1);
        pass.setVehicleId(1);

        pass.setPassType("Monthly");
```

```
        pass.setStartDate(
                Date.valueOf("2026-09-01"));

        pass.setEndDate(
                Date.valueOf("2026-09-30"));

        pass.setPassStatus("Active");

        pass.setPassAmount(1500.00);

        ParkingPassDAO dao = new ParkingPassDAO();

        boolean result = dao.addPass(pass);

        if (result) {

            System.out.println(
                    "Parking Pass Added Successfully!");

        } else {

            System.out.println(
                    "Parking Pass Addition Failed!");
        }
    }
}
```

TestParkingSessionDAO.java

```
package parking;

import java.sql.Timestamp;

public class TestParkingSessionDAO {

    public static void main(String[] args) {

        ParkingSession session = new ParkingSession();

        // Use existing vehicle and slot IDs
        session.setVehicleId(1);
        session.setSlotId(1);

        session.setEntryTime(
                Timestamp.valueOf("2026-09-01 09:00:00"));

        // Active session - no exit time yet
        session.setExitTime(null);

        session.setDurationMinutes(0);
```

```
        session.setParkingFee(0.00);

        session.setSessionStatus("Active");

        ParkingSessionDAO dao = new ParkingSessionDAO();

        boolean result = dao.addSession(session);

        if (result) {

            System.out.println(
                    "Parking Session Added Successfully!");

        } else {

            System.out.println(
                    "Parking Session Addition Failed!");
        }
    }
}
```

TestParkingSlotDAO.java

```
package parking;

public class TestParkingSlotDAO {

    public static void main(String[] args) {

        ParkingSlot slot = new ParkingSlot();

        // Zone D ID
        slot.setZoneId(4);

        slot.setSlotNumber("D-01");

        slot.setVehicleType("Two Wheeler");

        slot.setSlotStatus("Available");

        ParkingSlotDAO dao = new ParkingSlotDAO();

        boolean result = dao.addSlot(slot);

        if (result) {

            System.out.println("Parking Slot Added Successfully!");

        } else {
```

```
            System.out.println("Parking Slot Addition Failed!");
        }
    }
}
```

TestParkingZoneDAO.java

```
package parking;

public class TestParkingZoneDAO {

    public static void main(String[] args) {

        ParkingZone zone = new ParkingZone();

        zone.setZoneName("Zone D");
        zone.setLocation("Sports Ground");
        zone.setZoneType("Two Wheeler");
        zone.setTotalSlots(30);
        zone.setAvailableSlots(30);
        zone.setStatus("Active");

        ParkingZoneDAO dao = new ParkingZoneDAO();

        boolean result = dao.addZone(zone);

        if (result) {
            System.out.println("Parking Zone Added Successfully!");
        } else {
            System.out.println("Parking Zone Addition Failed!");
        }
    }
}
```

TestPaymentDAO.java

```
package parking;

public class TestPaymentDAO {

    public static void main(String[] args) {

        Payment payment = new Payment();

        // Use existing IDs from your database
        payment.setSessionId(1);
        payment.setPassId(1);
        payment.setUserId(1);
```

```
        payment.setAmount(1500.00);

        payment.setPaymentMethod("UPI");

        payment.setPaymentStatus("Paid");

        payment.setTransactionReference(
                "TXN20260901001");

        PaymentDAO dao = new PaymentDAO();

        boolean result = dao.addPayment(payment);

        if (result) {

            System.out.println(
                    "Payment Added Successfully!");

        } else {

            System.out.println(
                    "Payment Addition Failed!");
        }
    }
}
```

TestReservation.java

```
package parking;

import java.sql.Date;
import java.sql.Timestamp;

public class TestReservationDAO {

    public static void main(String[] args) {

        Reservation reservation = new Reservation();

        // Use existing IDs from your database
        reservation.setUserId(1);
        reservation.setVehicleId(1);
        reservation.setSlotId(1);

        reservation.setReservationDate(
                Date.valueOf("2026-09-01"));

        reservation.setStartTime(
                Timestamp.valueOf("2026-09-01 09:00:00"));
```

```
        reservation.setEndTime(
                Timestamp.valueOf("2026-09-01 11:00:00"));

        reservation.setReservationStatus("Reserved");

        ReservationDAO dao = new ReservationDAO();

        boolean result = dao.addReservation(reservation);

        if (result) {

            System.out.println(
                    "Reservation Added Successfully!");

        } else {

            System.out.println(
                    "Reservation Addition Failed!");
        }
    }
}
```

TestUserDAO.java

```
package parking;

public class TestUserDAO {

    public static void main(String[] args) {

        User user = new User();

        user.setName("Test Student");
        user.setEmail("teststudent@gmail.com");
        user.setPhone("9876543210");
        user.setUserType("Student");
        user.setPassword("1234");
        user.setStatus("Active");

        UserDAO dao = new UserDAO();

        boolean result = dao.addUser(user);

        if (result) {
            System.out.println("User Added Successfully!");
        } else {
            System.out.println("User Addition Failed!");
        }
    }
```

```
}
```

TestVehicleDAO.java

```
package parking;

public class TestVehicleDAO {

    public static void main(String[] args) {

        Vehicle vehicle = new Vehicle();

        // User ID 1 is the Test Student created earlier
        vehicle.setUserId(1);

        vehicle.setVehicleNumber("TN38ZZ9999");
        vehicle.setVehicleType("Bike");
        vehicle.setModel("Honda Shine");
        vehicle.setColor("Black");

        VehicleDAO dao = new VehicleDAO();

        boolean result = dao.addVehicle(vehicle);

        if (result) {
            System.out.println("Vehicle Added Successfully!");
        } else {
            System.out.println("Vehicle Addition Failed!");
        }
    }
}
```

TestVehicleDAO.java

```
package parking;

public class TestVehicleDAO {

    public static void main(String[] args) {

        Vehicle vehicle = new Vehicle();

        // User ID 1 is the Test Student created earlier
        vehicle.setUserId(1);

        vehicle.setVehicleNumber("TN38ZZ9999");
        vehicle.setVehicleType("Bike");
        vehicle.setModel("Honda Shine");
        vehicle.setColor("Black");
```

```
        VehicleDAO dao = new VehicleDAO();

        boolean result = dao.addVehicle(vehicle);

        if (result) {
            System.out.println("Vehicle Added Successfully!");
        } else {
            System.out.println("Vehicle Addition Failed!");
        }
    }
}
```

TestViolation.java

```
package parking;

public class TestViolationDAO {

    public static void main(String[] args) {

        Violation violation = new Violation();

        // Use existing IDs from your database
        violation.setUserId(1);
        violation.setVehicleId(1);

        violation.setViolationType(
                "Wrong Parking");

        violation.setDescription(
                "Vehicle parked in a restricted parking area.");

        violation.setFineAmount(500.00);

        violation.setViolationStatus(
                "Unpaid");

        ViolationDAO dao = new ViolationDAO();

        boolean result = dao.addViolation(violation);

        if (result) {

            System.out.println(
                    "Violation Added Successfully!");

        } else {

            System.out.println(
```

```
                "Violation Addition Failed!");
        }
    }
}
```

User.java

```
package parking;

public class User {

    private int userId;
    private String name;
    private String email;
    private String phone;
    private String userType;
    private String password;
    private String status;

    public User() {
    }

    public User(int userId, String name, String email,
                String phone, String userType,
                String password, String status) {

        this.userId = userId;
        this.name = name;
        this.email = email;
        this.phone = phone;
        this.userType = userType;
        this.password = password;
        this.status = status;
    }

    public int getUserId() {
        return userId;
    }

    public void setUserId(int userId) {
        this.userId = userId;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
```

```
    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getPhone() {
        return phone;
    }

    public void setPhone(String phone) {
        this.phone = phone;
    }

    public String getUserType() {
        return userType;
    }

    public void setUserType(String userType) {
        this.userType = userType;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }

    public String getStatus() {
        return status;
    }

    public void setStatus(String status) {
        this.status = status;
    }
}
```

UserDAO.java

```
package parking;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
```

```
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.CopyOnWriteArrayList;

public class UserDAO {

    private static final List<User> demoUsers = new CopyOnWriteArrayList<>();

    static {
        initDemoUsers();
    }

    private static void initDemoUsers() {
        if (!demoUsers.isEmpty()) return;
        demoUsers.add(new User(101, "Lithish Kumar", "lithish@campus.edu", "9876543210",
"Student", "pass123", "Active"));
            demoUsers.add(new User(102, "Dr. Rajesh Sharma", "rajesh.s@campus.edu",
"9845012345", "Faculty", "fac2026", "Active"));
        demoUsers.add(new User(103, "Priya Nair", "priya.n@campus.edu", "9740123456",
"Staff", "staff99", "Active"));
        demoUsers.add(new User(104, "Rahul Verma", "rahul.v@campus.edu", "9632145870",
"Student", "stud456", "Active"));
        demoUsers.add(new User(105, "Campus Security Admin", "security@campus.edu",
"9123456780", "Admin", "admin@2026", "Active"));
    }

    // GET NEXT AUTO-GENERATED USER ID
    public int getNextUserId() {
        String sql = "SELECT COALESCE(MAX(user_id), 100) + 1 FROM users";
        Connection con = DBConnection.getConnection();
        if (con != null) {
            try (con; PreparedStatement ps = con.prepareStatement(sql);
                 ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    return rs.getInt(1);
                }
            } catch (SQLException ignored) {}
        }
        int max = 100;
        for (User u : demoUsers) {
            if (u.getUserId() > max) max = u.getUserId();
        }
        return max + 1;
    }

    // ADD USER
    public boolean addUser(User user) {

        String sql = "INSERT INTO users "
                + "(name, email, phone, user_type, password, status) "
```

```
                + "VALUES (?, ?, ?, ?, ?, ?)";

        Connection con = DBConnection.getConnection();
        if (con != null) {
            try (con; PreparedStatement ps = con.prepareStatement(sql)) {

                ps.setString(1, user.getName());
                ps.setString(2, user.getEmail());
                ps.setString(3, user.getPhone());
                ps.setString(4, user.getUserType());
                ps.setString(5, user.getPassword());
                ps.setString(6, user.getStatus());

                int rows = ps.executeUpdate();
                return rows > 0;

            } catch (SQLException e) {
                System.out.println("Error adding user in DB: " + e.getMessage());
            }
        }

        if (user.getUserId() == 0) {
            user.setUserId(100 + demoUsers.size() + 1);
        }
        demoUsers.add(user);
        return true;
    }


    // SEARCH USER BY ID
    public User searchUser(int userId) {

        String sql = "SELECT * FROM users WHERE user_id = ?";

        Connection con = DBConnection.getConnection();
        if (con != null) {
            try (con; PreparedStatement ps = con.prepareStatement(sql)) {

                ps.setInt(1, userId);
                ResultSet rs = ps.executeQuery();

                if (rs.next()) {
                    User user = new User();
                    user.setUserId(rs.getInt("user_id"));
                    user.setName(rs.getString("name"));
                    user.setEmail(rs.getString("email"));
                    user.setPhone(rs.getString("phone"));
                    user.setUserType(rs.getString("user_type"));
                    user.setPassword(rs.getString("password"));
                    user.setStatus(rs.getString("status"));
```

```
                return user;
            }

        } catch (SQLException e) {
            System.out.println("Error searching user in DB: " + e.getMessage());
        }
    }

    for (User u : demoUsers) {
        if (u.getUserId() == userId) return u;
    }

    return null;
}

// UPDATE USER
public boolean updateUser(User user) {

    String sql = "UPDATE users SET name = ?, email = ?, phone = ?, user_type = ?,
password = ?, status = ? WHERE user_id = ?";

    Connection con = DBConnection.getConnection();
    if (con != null) {
        try (con; PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setString(1, user.getName());
            ps.setString(2, user.getEmail());
            ps.setString(3, user.getPhone());
            ps.setString(4, user.getUserType());
            ps.setString(5, user.getPassword());
            ps.setString(6, user.getStatus());
            ps.setInt(7, user.getUserId());

            int rows = ps.executeUpdate();
            return rows > 0;

        } catch (SQLException e) {
            System.out.println("Error updating user in DB: " + e.getMessage());
        }
    }

    for (int i = 0; i < demoUsers.size(); i++) {
        if (demoUsers.get(i).getUserId() == user.getUserId()) {
            demoUsers.set(i, user);
            return true;
        }
    }

    return false;
}
```

```java
// DELETE USER
public boolean deleteUser(int userId) {

    String sql = "DELETE FROM users WHERE user_id = ?";

    Connection con = DBConnection.getConnection();
    if (con != null) {
        try (con; PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, userId);
            int rows = ps.executeUpdate();
            return rows > 0;

        } catch (SQLException e) {
            System.out.println("Error deleting user in DB: " + e.getMessage());
        }
    }

    return demoUsers.removeIf(u -> u.getUserId() == userId);
}

// GET ALL USERS
public List<User> getAllUsers() {

    List<User> users = new ArrayList<>();
    String sql = "SELECT * FROM users ORDER BY user_id";

    Connection con = DBConnection.getConnection();
    if (con != null) {
        try (con;
             PreparedStatement ps = con.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {

            while (rs.next()) {
                User user = new User();
                user.setUserId(rs.getInt("user_id"));
                user.setName(rs.getString("name"));
                user.setEmail(rs.getString("email"));
                user.setPhone(rs.getString("phone"));
                user.setUserType(rs.getString("user_type"));
                user.setPassword(rs.getString("password"));
                user.setStatus(rs.getString("status"));
                users.add(user);
            }

            if (!users.isEmpty()) {
                return users;
            }
```

```
            } catch (SQLException e) {
                System.out.println("Error retrieving users from DB: " + e.getMessage());
            }
        }

        return new ArrayList<>(demoUsers);
    }
}
```

UserManagement.java

```
package parking;

import java.awt.*;
import java.awt.event.*;
import java.util.List;

public class UserManagement extends Frame implements ActionListener {

    // ====================================================
    // COLORS
    // ====================================================
    private final Color DARK_BLUE  = new Color(25, 55, 95);
    private final Color BLUE       = new Color(45, 95, 150);
    private final Color LIGHT_BG   = new Color(245, 247, 250);
    private final Color DARK_TEXT  = new Color(30, 41, 59);
    private final Color WHITE      = Color.WHITE;

    private final Color GREEN      = new Color(40, 140, 85);
    private final Color ORANGE     = new Color(230, 145, 40);
    private final Color RED        = new Color(190, 65, 65);
    private final Color GRAY       = new Color(100, 116, 139);
    private final Color CHARCOAL   = new Color(51, 65, 85);

    // ====================================================
    // FORM INPUTS
    // ====================================================
    private TextField txtUserId;
    private TextField txtName;
    private TextField txtEmail;
    private TextField txtPhone;
    private Choice userType;
    private Choice status;

    // ====================================================
    // CUSTOM AWT BUTTONS
    // ====================================================
    private AwtButton addButton;
    private AwtButton searchButton;
    private AwtButton updateButton;
```

```
    private AwtButton deleteButton;
    private AwtButton viewButton;
    private AwtButton clearButton;
    private AwtButton closeButton;

    // ====================================================
    // OUTPUT & STATUS
    // ====================================================
    private TextArea output;
    private Label statusLabel;

    private UserDAO userDAO;

    // ====================================================
    // CONSTRUCTOR
    // ====================================================
    public UserManagement() {

        userDAO = new UserDAO();

        setTitle("Smart Campus Parking - User Management");
        setSize(1020, 720);
        setLayout(new BorderLayout(0, 0));
        setBackground(LIGHT_BG);

        // ================================================
        // 1. HEADER
        // ================================================
        Panel headerPanel = new Panel(new BorderLayout(5, 5));
        headerPanel.setBackground(DARK_BLUE);

        Panel titleBox = new Panel(new GridLayout(2, 1));
        titleBox.setBackground(DARK_BLUE);

        Label title = new Label("USER MANAGEMENT", Label.CENTER);
        title.setFont(new Font("Arial", Font.BOLD, 22));
        title.setForeground(WHITE);
        titleBox.add(title);

        Label subtitle = new Label("Manage Students, Faculty, Staff & Parking Admins",
Label.CENTER);
        subtitle.setFont(new Font("Arial", Font.PLAIN, 13));
        subtitle.setForeground(new Color(210, 225, 245));
        titleBox.add(subtitle);

        headerPanel.add(titleBox, BorderLayout.CENTER);
        add(headerPanel, BorderLayout.NORTH);

        // ================================================
        // 2. MAIN CONTENT (Left Form, Right Records Table)
```

```
// ============================================
Panel mainContent = new Panel(new GridLayout(1, 2, 16, 0));
mainContent.setBackground(LIGHT_BG);

// --- LEFT CARD: FORM ---
Panel formCard = new Panel(new BorderLayout(0, 0));
formCard.setBackground(WHITE);

Panel formCardHeader = new Panel(new BorderLayout());
formCardHeader.setBackground(BLUE);
Label formTitle = new Label("  USER REGISTRATION FORM", Label.LEFT);
formTitle.setFont(new Font("Arial", Font.BOLD, 14));
formTitle.setForeground(WHITE);
formCardHeader.add(formTitle, BorderLayout.CENTER);
formCard.add(formCardHeader, BorderLayout.NORTH);

Panel formBody = new Panel(new GridBagLayout());
formBody.setBackground(WHITE);
GridBagConstraints gbc = new GridBagConstraints();
gbc.insets = new Insets(8, 16, 8, 16);
gbc.fill = GridBagConstraints.HORIZONTAL;
gbc.anchor = GridBagConstraints.NORTHWEST;

// Row 0: User ID
gbc.gridx = 0; gbc.gridy = 0; gbc.weightx = 0.3;
formBody.add(createLabel("User ID (Auto):"), gbc);
gbc.gridx = 1; gbc.weightx = 0.7;
txtUserId = new TextField(String.valueOf(userDAO.getNextUserId()));
txtUserId.setFont(new Font("Arial", Font.BOLD, 13));
formBody.add(txtUserId, gbc);

// Row 1: Name
gbc.gridx = 0; gbc.gridy = 1; gbc.weightx = 0.3;
formBody.add(createLabel("Full Name:"), gbc);
gbc.gridx = 1; gbc.weightx = 0.7;
txtName = new TextField();
txtName.setFont(new Font("Arial", Font.PLAIN, 13));
formBody.add(txtName, gbc);

// Row 2: Email
gbc.gridx = 0; gbc.gridy = 2; gbc.weightx = 0.3;
formBody.add(createLabel("Email:"), gbc);
gbc.gridx = 1; gbc.weightx = 0.7;
txtEmail = new TextField();
txtEmail.setFont(new Font("Arial", Font.PLAIN, 13));
formBody.add(txtEmail, gbc);

// Row 3: Phone
gbc.gridx = 0; gbc.gridy = 3; gbc.weightx = 0.3;
formBody.add(createLabel("Phone:"), gbc);
```

```
        gbc.gridx = 1; gbc.weightx = 0.7;
        txtPhone = new TextField();
        txtPhone.setFont(new Font("Arial", Font.PLAIN, 13));
        formBody.add(txtPhone, gbc);

        // Row 4: User Type
        gbc.gridx = 0; gbc.gridy = 4; gbc.weightx = 0.3;
        formBody.add(createLabel("User Type:"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.7;
        userType = new Choice();
        userType.add("Student");
        userType.add("Faculty");
        userType.add("Staff");
        userType.add("Admin");
        userType.setFont(new Font("Arial", Font.PLAIN, 12));
        formBody.add(userType, gbc);

        // Row 5: Status
        gbc.gridx = 0; gbc.gridy = 5; gbc.weightx = 0.3;
        formBody.add(createLabel("Status:"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.7;
        status = new Choice();
        status.add("Active");
        status.add("Inactive");
        status.setFont(new Font("Arial", Font.PLAIN, 12));
        formBody.add(status, gbc);

        // Row 6: Vertical filler to keep everything neatly top-aligned
        gbc.gridx = 0; gbc.gridy = 6; gbc.gridwidth = 2; gbc.weighty = 1.0;
        formBody.add(new Panel(), gbc);

        formCard.add(formBody, BorderLayout.CENTER);
        mainContent.add(formCard);

        // --- RIGHT CARD: RECORDS TABLE ---
        Panel tableCard = new Panel(new BorderLayout(0, 0));
        tableCard.setBackground(WHITE);

        Panel tableCardHeader = new Panel(new BorderLayout());
        tableCardHeader.setBackground(DARK_BLUE);
        Label tableTitle = new Label("  REGISTERED CAMPUS USERS", Label.LEFT);
        tableTitle.setFont(new Font("Arial", Font.BOLD, 14));
        tableTitle.setForeground(WHITE);
        tableCardHeader.add(tableTitle, BorderLayout.CENTER);
        tableCard.add(tableCardHeader, BorderLayout.NORTH);

        output = new TextArea("", 20, 50, TextArea.SCROLLBARS_VERTICAL_ONLY);
        output.setFont(new Font("Monospaced", Font.PLAIN, 12));
        output.setEditable(false);
        output.setBackground(new Color(250, 252, 255));
```

```
        tableCard.add(output, BorderLayout.CENTER);

        mainContent.add(tableCard);

        add(mainContent, BorderLayout.CENTER);

        // ================================================
        // 3. BOTTOM ACTIONS & STATUS BAR
        // ================================================
        Panel bottomContainer = new Panel(new BorderLayout(0, 0));
        bottomContainer.setBackground(LIGHT_BG);

        // Button Toolbar
        Panel buttonPanel = new Panel(new FlowLayout(FlowLayout.CENTER, 12, 10));
        buttonPanel.setBackground(new Color(230, 238, 248));

        addButton    = new AwtButton("□  ADD", GREEN, WHITE);
        searchButton = new AwtButton("□ SEARCH", BLUE, WHITE);
        updateButton = new AwtButton("✏□ UPDATE", ORANGE, WHITE);
        deleteButton = new AwtButton("□ DELETE", RED, WHITE);
        viewButton   = new AwtButton("□ VIEW ALL", DARK_BLUE, WHITE);
        clearButton  = new AwtButton("□ CLEAR", GRAY, WHITE);
        closeButton  = new AwtButton("□ CLOSE", CHARCOAL, WHITE);

        buttonPanel.add(addButton);
        buttonPanel.add(searchButton);
        buttonPanel.add(updateButton);
        buttonPanel.add(deleteButton);
        buttonPanel.add(viewButton);
        buttonPanel.add(clearButton);
        buttonPanel.add(closeButton);

        bottomContainer.add(buttonPanel, BorderLayout.CENTER);

        // Status bar
        statusLabel = new Label(" Ready | User Management.", Label.LEFT);
        statusLabel.setFont(new Font("Arial", Font.PLAIN, 12));
        statusLabel.setForeground(DARK_TEXT);
        statusLabel.setBackground(new Color(220, 230, 242));
        bottomContainer.add(statusLabel, BorderLayout.SOUTH);

        add(bottomContainer, BorderLayout.SOUTH);

        // ================================================
        // EVENT LISTENERS
        // ================================================
        addButton.addActionListener(this);
        searchButton.addActionListener(this);
        updateButton.addActionListener(this);
        deleteButton.addActionListener(this);
```

```
        viewButton.addActionListener(this);
        clearButton.addActionListener(this);
        closeButton.addActionListener(this);

        addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                dispose();
            }
        });

        // Load records immediately
        viewUsers();

        setLocationRelativeTo(null);
        setVisible(true);
    }

    private Label createLabel(String text) {
        Label lbl = new Label(text);
        lbl.setFont(new Font("Arial", Font.BOLD, 13));
        lbl.setForeground(DARK_TEXT);
        return lbl;
    }

    // ================================================
    // ACTION DISPATCHER
    // ================================================
    @Override
    public void actionPerformed(ActionEvent e) {
        Object src = e.getSource();

        if (src == addButton) {
            addUser();
        } else if (src == searchButton) {
            searchUser();
        } else if (src == updateButton) {
            updateUser();
        } else if (src == deleteButton) {
            deleteUser();
        } else if (src == viewButton) {
            viewUsers();
        } else if (src == clearButton) {
            clearForm();
        } else if (src == closeButton) {
            dispose();
        }
    }

    // ================================================
```

```java
// ADD USER
// ========================================================
private void addUser() {
    String name = txtName.getText().trim();
    String email = txtEmail.getText().trim();
    String phone = txtPhone.getText().trim();
    String type = userType.getSelectedItem();
    String st = status.getSelectedItem();

    if (name.isEmpty() || email.isEmpty()) {
        showMessage("Please fill all mandatory fields (Name, Email).");
        return;
    }

    User user = new User();
    String idStr = txtUserId.getText().trim();
    if (!idStr.isEmpty()) {
        try {
            user.setUserId(Integer.parseInt(idStr));
        } catch (NumberFormatException ignored) {}
    }
    user.setName(name);
    user.setEmail(email);
    user.setPhone(phone);
    user.setPassword("pass123");
    user.setUserType(type);
    user.setStatus(st);

    boolean ok = userDAO.addUser(user);
    if (ok) {
        statusLabel.setText("□  User "+ name + " Added Successfully!");
        showMessage("User Added Successfully!");
        clearForm();
        viewUsers();
    } else {
        showMessage("Failed to add user.");
    }
}

// ========================================================
// SEARCH USER
// ========================================================
private void searchUser() {
    String idStr = txtUserId.getText().trim();
    if (idStr.isEmpty()) {
        showMessage("Please enter a User ID to search.");
        return;
    }

    try {
```

```
            int userId = Integer.parseInt(idStr);
            User user = userDAO.searchUser(userId);

            if (user != null) {
                txtName.setText(user.getName());
                txtEmail.setText(user.getEmail());
                txtPhone.setText(user.getPhone());
                userType.select(user.getUserType());
                status.select(user.getStatus());
                        statusLabel.setText("□ Found User: " + user.getName() + " (ID: " +
user.getUserId() + ")");
            } else {
                showMessage("No user found with ID: " + userId);
            }
        } catch (NumberFormatException ex) {
            showMessage("User ID must be a valid number.");
        }
    }

    // ================================================
    // UPDATE USER
    // ================================================
    private void updateUser() {
        String idStr = txtUserId.getText().trim();
        if (idStr.isEmpty()) {
            showMessage("Please enter or search a User ID to update.");
            return;
        }

        try {
            int userId = Integer.parseInt(idStr);
            String name = txtName.getText().trim();
            String email = txtEmail.getText().trim();
            String phone = txtPhone.getText().trim();
            String type = userType.getSelectedItem();
            String st = status.getSelectedItem();

            if (name.isEmpty() || email.isEmpty()) {
                showMessage("Name and Email cannot be empty.");
                return;
            }

            User user = new User();
            user.setUserId(userId);
            user.setName(name);
            user.setEmail(email);
            user.setPhone(phone);
            user.setPassword("pass123");
            user.setUserType(type);
            user.setStatus(st);
```

```
            boolean ok = userDAO.updateUser(user);
            if (ok) {
                statusLabel.setText("✏ User ID " + userId + " updated successfully.");
                showMessage("User Updated Successfully!");
                clearForm();
                viewUsers();
            } else {
                showMessage("Failed to update user.");
            }
        } catch (NumberFormatException ex) {
            showMessage("User ID must be a valid number.");
        }
    }

    // ================================================
    // DELETE USER
    // ================================================
    private void deleteUser() {
        String idStr = txtUserId.getText().trim();
        if (idStr.isEmpty()) {
            showMessage("Please enter a User ID to delete.");
            return;
        }

        try {
            int userId = Integer.parseInt(idStr);
            boolean ok = userDAO.deleteUser(userId);
            if (ok) {
                statusLabel.setText("🗑 User ID " + userId + " deleted successfully.");
                showMessage("User Deleted Successfully.");
                clearForm();
                viewUsers();
            } else {
                showMessage("Could not delete user with ID: " + userId);
            }
        } catch (NumberFormatException ex) {
            showMessage("User ID must be a valid number.");
        }
    }

    // ================================================
    // VIEW ALL USERS
    // ================================================
    private void viewUsers() {
        List<User> users = userDAO.getAllUsers();
        output.setText("");

         output.append("================================================
================================\n");
```

```
        output.append("                          CAMPUS USER DIRECTORY\n");
         output.append("==========================================================
==================================\n");
            output.append(String.format("%-6s  %-20s  %-24s  %-13s  %-10s  %-8s%n",  "ID",
"NAME", "EMAIL", "PHONE", "TYPE", "STATUS"));
             output.append("-----------------------------------------------------------------------------------
\n");

        if (users == null || users.isEmpty()) {
            output.append("\nNo user records found.\n");
            return;
        }

        for (User u : users) {
            output.append(String.format("%-6d %-20s %-24s %-13s %-10s %-8s%n",
                    u.getUserId(),
                    u.getName(),
                    u.getEmail(),
                    u.getPhone(),
                    u.getUserType(),
                    u.getStatus()
            ));
        }

             output.append("-----------------------------------------------------------------------------------
\n");
        output.append("Total Registered Users: " + users.size() + "\n");
        statusLabel.setText(" Loaded " + users.size() + " campus users successfully.");
    }

    // ================================================
    // CLEAR FORM
    // ================================================
    private void clearForm() {
        txtUserId.setText(String.valueOf(userDAO.getNextUserId()));
        txtName.setText("");
        txtEmail.setText("");
        txtPhone.setText("");
        userType.select(0);
        status.select(0);
                    statusLabel.setText("  Ready  |  Next  User  ID  auto-assigned:  "  +
userDAO.getNextUserId());
    }

    // ================================================
    // SHOW MESSAGE DIALOG
    // ================================================
    private void showMessage(String msg) {
        Dialog d = new Dialog(this, "Notification", true);
        d.setSize(360, 160);
```

```
        d.setLayout(new BorderLayout(10, 10));
        d.setBackground(LIGHT_BG);

        Label lbl = new Label(msg, Label.CENTER);
        lbl.setFont(new Font("Arial", Font.PLAIN, 13));
        d.add(lbl, BorderLayout.CENTER);

        AwtButton btnOk = new AwtButton("OK", BLUE, WHITE);
        btnOk.addActionListener(e -> d.dispose());
        Panel p = new Panel(new FlowLayout());
        p.add(btnOk);
        d.add(p, BorderLayout.SOUTH);

        d.addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                d.dispose();
            }
        });

        d.setLocationRelativeTo(this);
        d.setVisible(true);
    }

    public static void main(String[] args) {
        new UserManagement();
    }
}
```

Vehicle.java

```
package parking;

public class Vehicle {

    private int vehicleId;
    private int userId;
    private String vehicleNumber;
    private String vehicleType;
    private String model;
    private String color;

    public Vehicle() {
    }

    public Vehicle(int vehicleId, int userId, String vehicleNumber,
                   String vehicleType, String model, String color) {

        this.vehicleId = vehicleId;
        this.userId = userId;
```

```
        this.vehicleNumber = vehicleNumber;
        this.vehicleType = vehicleType;
        this.model = model;
        this.color = color;
    }

    public int getVehicleId() {
        return vehicleId;
    }

    public void setVehicleId(int vehicleId) {
        this.vehicleId = vehicleId;
    }

    public int getUserId() {
        return userId;
    }

    public void setUserId(int userId) {
        this.userId = userId;
    }

    public String getVehicleNumber() {
        return vehicleNumber;
    }

    public void setVehicleNumber(String vehicleNumber) {
        this.vehicleNumber = vehicleNumber;
    }

    public String getVehicleType() {
        return vehicleType;
    }

    public void setVehicleType(String vehicleType) {
        this.vehicleType = vehicleType;
    }

    public String getModel() {
        return model;
    }

    public void setModel(String model) {
        this.model = model;
    }

    public String getColor() {
        return color;
    }
```

```
    public void setColor(String color) {
        this.color = color;
    }
}
```

VehicleDAO.java

```
package parking;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.CopyOnWriteArrayList;

public class VehicleDAO {

    private static final List<Vehicle> demoVehicles = new CopyOnWriteArrayList<>();

    static {
        initDemoVehicles();
    }

    private static void initDemoVehicles() {
        if (!demoVehicles.isEmpty()) return;
         demoVehicles.add(new Vehicle(1, 101, "KA-01-AB-1234", "Four Wheeler", "Honda City", "White"));
        demoVehicles.add(new Vehicle(2, 102, "KA-01-CD-5678", "Two Wheeler", "Yamaha FZ", "Black"));
         demoVehicles.add(new Vehicle(3, 103, "KA-05-EF-9012", "EV", "Tata Nexon EV", "Blue"));
        demoVehicles.add(new Vehicle(4, 104, "KA-03-GH-3456", "Four Wheeler", "Hyundai Creta", "Silver"));
         demoVehicles.add(new Vehicle(5, 105, "KA-02-JK-7890", "Two Wheeler", "Royal Enfield", "Red"));
    }

    // GET NEXT AUTO-GENERATED VEHICLE ID
    public int getNextVehicleId() {
        String sql = "SELECT COALESCE(MAX(vehicle_id), 0) + 1 FROM vehicles";
        Connection con = DBConnection.getConnection();
        if (con != null) {
            try (con; PreparedStatement ps = con.prepareStatement(sql);
                 ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    return rs.getInt(1);
                }
            } catch (SQLException ignored) {}
        }
        int max = 0;
        for (Vehicle v : demoVehicles) {
```

```
        if (v.getVehicleId() > max) max = v.getVehicleId();
    }
    return max + 1;
}

// ========================================
// ADD VEHICLE
// ========================================

public boolean addVehicle(Vehicle vehicle) {

    String sql =
            "INSERT INTO vehicles " +
            "(user_id, vehicle_number, vehicle_type, model, color) " +
            "VALUES (?, ?, ?, ?, ?)";

    Connection con = DBConnection.getConnection();
    if (con != null) {
        try (con; PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, vehicle.getUserId());
            ps.setString(2, vehicle.getVehicleNumber());
            ps.setString(3, vehicle.getVehicleType());
            ps.setString(4, vehicle.getModel());
            ps.setString(5, vehicle.getColor());

            int rows = ps.executeUpdate();
            return rows > 0;

        } catch (SQLException ex) {
            System.out.println("Error adding vehicle in DB: " + ex.getMessage());
        }
    }

    if (vehicle.getVehicleId() == 0) {
        vehicle.setVehicleId(demoVehicles.size() + 1);
    }
    demoVehicles.add(vehicle);
    return true;
}


// ========================================
// SEARCH VEHICLE BY ID
// ========================================

public Vehicle getVehicleById(int vehicleId) {

    String sql =
            "SELECT * FROM vehicles WHERE vehicle_id = ?";
```

```
    Connection con = DBConnection.getConnection();
    if (con != null) {
        try (con; PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, vehicleId);
            ResultSet rs = ps.executeQuery();

            if (rs.next()) {
                return createVehicleFromResultSet(rs);
            }

        } catch (SQLException ex) {
            System.out.println("Error searching vehicle in DB: " + ex.getMessage());
        }
    }

    for (Vehicle v : demoVehicles) {
        if (v.getVehicleId() == vehicleId) return v;
    }

    return null;
}


// ==========================================
// GET ALL VEHICLES
// ==========================================

public List<Vehicle> getAllVehicles() {

    List<Vehicle> vehicles = new ArrayList<>();
    String sql = "SELECT * FROM vehicles ORDER BY vehicle_id";

    Connection con = DBConnection.getConnection();
    if (con != null) {
        try (con;
             PreparedStatement ps = con.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {

            while (rs.next()) {
                vehicles.add(createVehicleFromResultSet(rs));
            }

            if (!vehicles.isEmpty()) {
                return vehicles;
            }

        } catch (SQLException ex) {
            System.out.println("Error viewing vehicles from DB: " + ex.getMessage());
```

```
        }
    }

    return new ArrayList<>(demoVehicles);
}


// ========================================
// UPDATE VEHICLE
// ========================================

public boolean updateVehicle(Vehicle vehicle) {

    String sql =
            "UPDATE vehicles SET " +
            "user_id = ?, " +
            "vehicle_number = ?, " +
            "vehicle_type = ?, " +
            "model = ?, " +
            "color = ? " +
            "WHERE vehicle_id = ?";

    Connection con = DBConnection.getConnection();
    if (con != null) {
        try (con; PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, vehicle.getUserId());
            ps.setString(2, vehicle.getVehicleNumber());
            ps.setString(3, vehicle.getVehicleType());
            ps.setString(4, vehicle.getModel());
            ps.setString(5, vehicle.getColor());
            ps.setInt(6, vehicle.getVehicleId());

            int rows = ps.executeUpdate();
            return rows > 0;

        } catch (SQLException ex) {
            System.out.println("Error updating vehicle in DB: " + ex.getMessage());
        }
    }

    for (int i = 0; i < demoVehicles.size(); i++) {
        if (demoVehicles.get(i).getVehicleId() == vehicle.getVehicleId()) {
            demoVehicles.set(i, vehicle);
            return true;
        }
    }

    return false;
}
```

```java
// ========================================
// DELETE VEHICLE
// ========================================

public boolean deleteVehicle(int vehicleId) {

    String sql =
            "DELETE FROM vehicles WHERE vehicle_id = ?";

    Connection con = DBConnection.getConnection();
    if (con != null) {
        try (con; PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, vehicleId);
            int rows = ps.executeUpdate();
            return rows > 0;

        } catch (SQLException ex) {
            System.out.println("Error deleting vehicle in DB: " + ex.getMessage());
        }
    }

    return demoVehicles.removeIf(v -> v.getVehicleId() == vehicleId);
}


// ========================================
// CREATE VEHICLE FROM RESULT SET
// ========================================

private Vehicle createVehicleFromResultSet(
        ResultSet rs) throws SQLException {

    Vehicle vehicle =
            new Vehicle();

    vehicle.setVehicleId(
            rs.getInt("vehicle_id")
    );

    vehicle.setUserId(
            rs.getInt("user_id")
    );

    vehicle.setVehicleNumber(
            rs.getString("vehicle_number")
    );
```

```
        vehicle.setVehicleType(
                rs.getString("vehicle_type")
        );

        vehicle.setModel(
                rs.getString("model")
        );

        vehicle.setColor(
                rs.getString("color")
        );

        return vehicle;
    }
}
```

VehicleManagement.java

```
package parking;

import java.awt.*;
import java.awt.event.*;
import java.util.List;

public class VehicleManagement extends Frame implements ActionListener {

    // COLORS
    private final Color DARK_BLUE  = new Color(25, 55, 95);
    private final Color BLUE       = new Color(45, 95, 150);
    private final Color LIGHT_BG   = new Color(245, 247, 250);
    private final Color DARK_TEXT  = new Color(30, 41, 59);
    private final Color WHITE      = Color.WHITE;

    private final Color GREEN      = new Color(40, 140, 85);
    private final Color ORANGE     = new Color(230, 145, 40);
    private final Color RED        = new Color(190, 65, 65);
    private final Color GRAY       = new Color(100, 116, 139);
    private final Color CHARCOAL   = new Color(51, 65, 85);

    // FORM INPUTS
    private TextField txtVehicleId;
    private TextField txtUserId;
    private TextField txtVehicleNumber;
    private TextField txtModel;
    private TextField txtColor;
    private Choice vehicleType;

    // CUSTOM AWT BUTTONS
    private AwtButton addButton;
```

```
    private AwtButton searchButton;
    private AwtButton updateButton;
    private AwtButton deleteButton;
    private AwtButton viewButton;
    private AwtButton clearButton;
    private AwtButton closeButton;

    private TextArea output;
    private Label statusLabel;
    private VehicleDAO vehicleDAO;

    public VehicleManagement() {

        vehicleDAO = new VehicleDAO();

        setTitle("Smart Campus Parking - Vehicle Management");
        setSize(1020, 720);
        setLayout(new BorderLayout(0, 0));
        setBackground(LIGHT_BG);

        // HEADER
        Panel headerPanel = new Panel(new BorderLayout(5, 5));
        headerPanel.setBackground(DARK_BLUE);

        Panel titleBox = new Panel(new GridLayout(2, 1));
        titleBox.setBackground(DARK_BLUE);

        Label title = new Label("VEHICLE MANAGEMENT", Label.CENTER);
        title.setFont(new Font("Arial", Font.BOLD, 22));
        title.setForeground(WHITE);
        titleBox.add(title);

        Label subtitle = new Label("Register & Track Campus Vehicles, Two-Wheelers, Cars &
EVs", Label.CENTER);
        subtitle.setFont(new Font("Arial", Font.PLAIN, 13));
        subtitle.setForeground(new Color(210, 225, 245));
        titleBox.add(subtitle);

        headerPanel.add(titleBox, BorderLayout.CENTER);
        add(headerPanel, BorderLayout.NORTH);

        // MAIN CONTENT
        Panel mainContent = new Panel(new GridLayout(1, 2, 16, 0));
        mainContent.setBackground(LIGHT_BG);

        // LEFT CARD: FORM
        Panel formCard = new Panel(new BorderLayout(0, 0));
        formCard.setBackground(WHITE);

        Panel formCardHeader = new Panel(new BorderLayout());
```

```
        formCardHeader.setBackground(BLUE);
        Label formTitle = new Label("  VEHICLE REGISTRATION FORM", Label.LEFT);
        formTitle.setFont(new Font("Arial", Font.BOLD, 14));
        formTitle.setForeground(WHITE);
        formCardHeader.add(formTitle, BorderLayout.CENTER);
        formCard.add(formCardHeader, BorderLayout.NORTH);

        Panel formBody = new Panel(new GridBagLayout());
        formBody.setBackground(WHITE);
        GridBagConstraints gbc = new GridBagConstraints();
        gbc.insets = new Insets(8, 16, 8, 16);
        gbc.fill = GridBagConstraints.HORIZONTAL;
        gbc.anchor = GridBagConstraints.NORTHWEST;

        // Row 0: Vehicle ID
        gbc.gridx = 0; gbc.gridy = 0; gbc.weightx = 0.35;
        formBody.add(createLabel("Vehicle ID (Auto):"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.65;
        txtVehicleId = new TextField(String.valueOf(vehicleDAO.getNextVehicleId()));
        txtVehicleId.setFont(new Font("Arial", Font.BOLD, 13));
        formBody.add(txtVehicleId, gbc);

        // Row 1: User ID
        gbc.gridx = 0; gbc.gridy = 1; gbc.weightx = 0.35;
        formBody.add(createLabel("Owner User ID:"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.65;
        txtUserId = new TextField("101");
        formBody.add(txtUserId, gbc);

        // Row 2: Vehicle Number
        gbc.gridx = 0; gbc.gridy = 2; gbc.weightx = 0.35;
        formBody.add(createLabel("Plate Number:"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.65;
        txtVehicleNumber = new TextField();
        formBody.add(txtVehicleNumber, gbc);

        // Row 3: Vehicle Type
        gbc.gridx = 0; gbc.gridy = 3; gbc.weightx = 0.35;
        formBody.add(createLabel("Vehicle Type:"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.65;
        vehicleType = new Choice();
        vehicleType.add("Four Wheeler");
        vehicleType.add("Two Wheeler");
        vehicleType.add("EV");
        formBody.add(vehicleType, gbc);

        // Row 4: Model
        gbc.gridx = 0; gbc.gridy = 4; gbc.weightx = 0.35;
        formBody.add(createLabel("Make / Model:"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.65;
```

```
        txtModel = new TextField();
        formBody.add(txtModel, gbc);

        // Row 5: Color
        gbc.gridx = 0; gbc.gridy = 5; gbc.weightx = 0.35;
        formBody.add(createLabel("Color:"), gbc);
        gbc.gridx = 1; gbc.weightx = 0.65;
        txtColor = new TextField();
        formBody.add(txtColor, gbc);

        // Row 6: Vertical filler
        gbc.gridx = 0; gbc.gridy = 6; gbc.gridwidth = 2; gbc.weighty = 1.0;
        formBody.add(new Panel(), gbc);

        formCard.add(formBody, BorderLayout.CENTER);
        mainContent.add(formCard);

        // RIGHT CARD: RECORDS
        Panel tableCard = new Panel(new BorderLayout(0, 0));
        tableCard.setBackground(WHITE);

        Panel tableCardHeader = new Panel(new BorderLayout());
        tableCardHeader.setBackground(DARK_BLUE);
            Label tableTitle = new Label("  REGISTERED VEHICLES DIRECTORY",
Label.LEFT);
        tableTitle.setFont(new Font("Arial", Font.BOLD, 14));
        tableTitle.setForeground(WHITE);
        tableCardHeader.add(tableTitle, BorderLayout.CENTER);
        tableCard.add(tableCardHeader, BorderLayout.NORTH);

        output = new TextArea("", 20, 50, TextArea.SCROLLBARS_VERTICAL_ONLY);
        output.setFont(new Font("Monospaced", Font.PLAIN, 12));
        output.setEditable(false);
        output.setBackground(new Color(250, 252, 255));
        tableCard.add(output, BorderLayout.CENTER);

        mainContent.add(tableCard);
        add(mainContent, BorderLayout.CENTER);

        // BOTTOM BAR
        Panel bottomContainer = new Panel(new BorderLayout(0, 0));
        bottomContainer.setBackground(LIGHT_BG);

        Panel buttonPanel = new Panel(new FlowLayout(FlowLayout.CENTER, 10, 10));
        buttonPanel.setBackground(new Color(230, 238, 248));

        addButton    = new AwtButton("□  ADD", GREEN, WHITE);
        searchButton = new AwtButton("□ SEARCH", BLUE, WHITE);
        updateButton = new AwtButton("✏□ UPDATE", ORANGE, WHITE);
        deleteButton = new AwtButton("□ DELETE", RED, WHITE);
```

```
        viewButton   = new AwtButton("□ VIEW ALL", DARK_BLUE, WHITE);
        clearButton  = new AwtButton("□ CLEAR", GRAY, WHITE);
        closeButton  = new AwtButton("□  CLOSE", CHARCOAL, WHITE);

        buttonPanel.add(addButton);
        buttonPanel.add(searchButton);
        buttonPanel.add(updateButton);
        buttonPanel.add(deleteButton);
        buttonPanel.add(viewButton);
        buttonPanel.add(clearButton);
        buttonPanel.add(closeButton);

        bottomContainer.add(buttonPanel, BorderLayout.CENTER);

        statusLabel = new Label(" Ready | Vehicle Management.", Label.LEFT);
        statusLabel.setFont(new Font("Arial", Font.PLAIN, 12));
        statusLabel.setForeground(DARK_TEXT);
        statusLabel.setBackground(new Color(220, 230, 242));
        bottomContainer.add(statusLabel, BorderLayout.SOUTH);

        add(bottomContainer, BorderLayout.SOUTH);

        addButton.addActionListener(this);
        searchButton.addActionListener(this);
        updateButton.addActionListener(this);
        deleteButton.addActionListener(this);
        viewButton.addActionListener(this);
        clearButton.addActionListener(this);
        closeButton.addActionListener(this);

        addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                dispose();
            }
        });

        viewVehicles();

        setLocationRelativeTo(null);
        setVisible(true);
    }

    private Label createLabel(String text) {
        Label lbl = new Label(text);
        lbl.setFont(new Font("Arial", Font.BOLD, 13));
        lbl.setForeground(DARK_TEXT);
        return lbl;
    }
```

```
@Override
public void actionPerformed(ActionEvent e) {
    Object src = e.getSource();
    if (src == addButton) {
        addVehicle();
    } else if (src == searchButton) {
        searchVehicle();
    } else if (src == updateButton) {
        updateVehicle();
    } else if (src == deleteButton) {
        deleteVehicle();
    } else if (src == viewButton) {
        viewVehicles();
    } else if (src == clearButton) {
        clearForm();
    } else if (src == closeButton) {
        dispose();
    }
}

private void addVehicle() {
    try {
        int uid = Integer.parseInt(txtUserId.getText().trim());
        String num = txtVehicleNumber.getText().trim();
        String type = vehicleType.getSelectedItem();
        String model = txtModel.getText().trim();
        String color = txtColor.getText().trim();

        if (num.isEmpty() || model.isEmpty()) {
            showMessage("Please fill Plate Number and Model.");
            return;
        }

        Vehicle v = new Vehicle();
        String idStr = txtVehicleId.getText().trim();
        if (!idStr.isEmpty()) {
            try {
                v.setVehicleId(Integer.parseInt(idStr));
            } catch (NumberFormatException ignored) {}
        }
        v.setUserId(uid);
        v.setVehicleNumber(num);
        v.setVehicleType(type);
        v.setModel(model);
        v.setColor(color);

        boolean ok = vehicleDAO.addVehicle(v);
        if (ok) {
            statusLabel.setText("□  Vehicle "+ num + " Added Successfully!");
            showMessage("Vehicle Added Successfully!");
```

```
            clearForm();
            viewVehicles();
        } else {
            showMessage("Failed to add vehicle.");
        }
    } catch (Exception ex) {
        showMessage("Invalid input. Please check Owner User ID.");
    }
}

private void searchVehicle() {
    try {
        int id = Integer.parseInt(txtVehicleId.getText().trim());
        Vehicle v = vehicleDAO.getVehicleById(id);
        if (v != null) {
            txtUserId.setText(String.valueOf(v.getUserId()));
            txtVehicleNumber.setText(v.getVehicleNumber());
            vehicleType.select(v.getVehicleType());
            txtModel.setText(v.getModel());
            txtColor.setText(v.getColor());
                statusLabel.setText("□ Found Vehicle: " + v.getVehicleNumber() + " (ID: " +
v.getVehicleId() + ")");
        } else {
            showMessage("No vehicle found with ID: " + id);
        }
    } catch (Exception ex) {
        showMessage("Please enter a valid numeric Vehicle ID.");
    }
}

private void updateVehicle() {
    try {
        int id = Integer.parseInt(txtVehicleId.getText().trim());
        int uid = Integer.parseInt(txtUserId.getText().trim());
        String num = txtVehicleNumber.getText().trim();
        String type = vehicleType.getSelectedItem();
        String model = txtModel.getText().trim();
        String color = txtColor.getText().trim();

        Vehicle v = new Vehicle();
        v.setVehicleId(id);
        v.setUserId(uid);
        v.setVehicleNumber(num);
        v.setVehicleType(type);
        v.setModel(model);
        v.setColor(color);

        boolean ok = vehicleDAO.updateVehicle(v);
        if (ok) {
            statusLabel.setText("✏□ Vehicle ID " + id + " updated successfully.");
```

```
            showMessage("Vehicle Updated Successfully!");
            clearForm();
            viewVehicles();
        } else {
            showMessage("Failed to update vehicle.");
        }
    } catch (Exception ex) {
        showMessage("Invalid input. Please check numeric IDs.");
    }
}

private void deleteVehicle() {
    try {
        int id = Integer.parseInt(txtVehicleId.getText().trim());
        boolean ok = vehicleDAO.deleteVehicle(id);
        if (ok) {
            statusLabel.setText("□ Vehicle ID " + id + " deleted.");
            showMessage("Vehicle Deleted Successfully.");
            clearForm();
            viewVehicles();
        } else {
            showMessage("Could not delete vehicle ID: " + id);
        }
    } catch (Exception ex) {
        showMessage("Please enter a valid numeric Vehicle ID.");
    }
}

private void viewVehicles() {
    List<Vehicle> list = vehicleDAO.getAllVehicles();
    output.setText("");

     output.append("==========================================================
================================\n");
    output.append("                          CAMPUS VEHICLES REGISTRY\n");
     output.append("==========================================================
================================\n");
        output.append(String.format("%-6s %-8s %-16s %-16s %-16s %-10s%n", "ID",
"OWNER", "PLATE NUMBER", "TYPE", "MODEL", "COLOR"));
         output.append("----------------------------------------------------------------------------------
\n");

    if (list == null || list.isEmpty()) {
        output.append("\nNo vehicle records found.\n");
        return;
    }

    for (Vehicle v : list) {
        output.append(String.format("%-6d %-8d %-16s %-16s %-16s %-10s%n",
                v.getVehicleId(),
```

```
                v.getUserId(),
                v.getVehicleNumber(),
                v.getVehicleType(),
                v.getModel(),
                v.getColor()
            ));
        }

            output.append("-------------------------------------------------------------------------------
\n");
        output.append("Total Vehicles: " + list.size() + "\n");
        statusLabel.setText(" Loaded " + list.size() + " campus vehicles.");
    }

    private void clearForm() {
        txtVehicleId.setText(String.valueOf(vehicleDAO.getNextVehicleId()));
        txtUserId.setText("101");
        txtVehicleNumber.setText("");
        txtModel.setText("");
        txtColor.setText("");
        vehicleType.select(0);
                statusLabel.setText("  Ready  |  Next  Vehicle  ID  auto-assigned:  "  +
vehicleDAO.getNextVehicleId());
    }

    private void showMessage(String msg) {
        Dialog d = new Dialog(this, "Notification", true);
        d.setSize(380, 160);
        d.setLayout(new BorderLayout(10, 10));
        d.setBackground(LIGHT_BG);

        Label lbl = new Label(msg, Label.CENTER);
        lbl.setFont(new Font("Arial", Font.PLAIN, 12));
        d.add(lbl, BorderLayout.CENTER);

        AwtButton btnOk = new AwtButton("OK", BLUE, WHITE);
        btnOk.addActionListener(e -> d.dispose());
        Panel p = new Panel(new FlowLayout());
        p.add(btnOk);
        d.add(p, BorderLayout.SOUTH);

        d.addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                d.dispose();
            }
        });

        d.setLocationRelativeTo(this);
        d.setVisible(true);
```

```
    }

    public static void main(String[] args) {
        new VehicleManagement();
    }
}
```

Violation.java

```
package parking;

import java.sql.Timestamp;

public class Violation {

    private int violationId;
    private int userId;
    private int vehicleId;
    private String violationType;
    private String description;
    private Timestamp violationDate;
    private double fineAmount;
    private String violationStatus;

    public Violation() {
    }

    public Violation(int violationId, int userId, int vehicleId,
                     String violationType, String description,
                     Timestamp violationDate, double fineAmount,
                     String violationStatus) {

        this.violationId = violationId;
        this.userId = userId;
        this.vehicleId = vehicleId;
        this.violationType = violationType;
        this.description = description;
        this.violationDate = violationDate;
        this.fineAmount = fineAmount;
        this.violationStatus = violationStatus;
    }

    public int getViolationId() {
        return violationId;
    }

    public void setViolationId(int violationId) {
        this.violationId = violationId;
    }
```

```java
public int getUserId() {
    return userId;
}

public void setUserId(int userId) {
    this.userId = userId;
}

public int getVehicleId() {
    return vehicleId;
}

public void setVehicleId(int vehicleId) {
    this.vehicleId = vehicleId;
}

public String getViolationType() {
    return violationType;
}

public void setViolationType(String violationType) {
    this.violationType = violationType;
}

public String getDescription() {
    return description;
}

public void setDescription(String description) {
    this.description = description;
}

public Timestamp getViolationDate() {
    return violationDate;
}

public void setViolationDate(Timestamp violationDate) {
    this.violationDate = violationDate;
}

public double getFineAmount() {
    return fineAmount;
}

public void setFineAmount(double fineAmount) {
    this.fineAmount = fineAmount;
}

public String getViolationStatus() {
    return violationStatus;
```

```
    }

    public void setViolationStatus(String violationStatus) {
        this.violationStatus = violationStatus;
    }
}
```

ViolationDAO.java

```
package parking;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Timestamp;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.CopyOnWriteArrayList;

public class ViolationDAO {

    private static final List<Violation> demoViolations = new CopyOnWriteArrayList<>();

    static {
        initDemoViolations();
    }

    private static void initDemoViolations() {
        if (!demoViolations.isEmpty()) return;
        long now = System.currentTimeMillis();
            demoViolations.add(new Violation(1, 104, 4, "No Parking Zone", "Parked in unauthorized emergency driveway", new Timestamp(now - 86400000), 200.0, "Pending"));
            demoViolations.add(new Violation(2, 101, 1, "Overstay", "Exceeded allocated reservation duration by 2 hours", new Timestamp(now - 43200000), 100.0, "Paid"));
    }

    // ADD VIOLATION
    public boolean addViolation(Violation violation) {

        String sql = "INSERT INTO violations "
                + "(user_id, vehicle_id, violation_type, description, "
                + "fine_amount, violation_status) "
                + "VALUES (?, ?, ?, ?, ?, ?)";

        Connection con = DBConnection.getConnection();
        if (con != null) {
            try (con; PreparedStatement ps = con.prepareStatement(sql)) {

                ps.setInt(1, violation.getUserId());
```

```
            ps.setInt(2, violation.getVehicleId());
            ps.setString(3, violation.getViolationType());
            ps.setString(4, violation.getDescription());
            ps.setDouble(5, violation.getFineAmount());
            ps.setString(6, violation.getViolationStatus());

            int rows = ps.executeUpdate();
            return rows > 0;

        } catch (SQLException e) {
            System.out.println("Error adding violation in DB: " + e.getMessage());
        }
    }

    if (violation.getViolationId() == 0) {
        violation.setViolationId(demoViolations.size() + 1);
    }
    demoViolations.add(violation);
    return true;
}


// SEARCH VIOLATION BY ID
public Violation searchViolation(int violationId) {

    String sql = "SELECT * FROM violations WHERE violation_id = ?";

    Connection con = DBConnection.getConnection();
    if (con != null) {
        try (con; PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, violationId);
            ResultSet rs = ps.executeQuery();

            if (rs.next()) {
                Violation violation = new Violation();
                violation.setViolationId(rs.getInt("violation_id"));
                violation.setUserId(rs.getInt("user_id"));
                violation.setVehicleId(rs.getInt("vehicle_id"));
                violation.setViolationType(rs.getString("violation_type"));
                violation.setDescription(rs.getString("description"));
                violation.setViolationDate(rs.getTimestamp("violation_date"));
                violation.setFineAmount(rs.getDouble("fine_amount"));
                violation.setViolationStatus(rs.getString("violation_status"));
                return violation;
            }

        } catch (SQLException e) {
            System.out.println("Error searching violation in DB: " + e.getMessage());
        }
```

```
        }

        for (Violation v : demoViolations) {
            if (v.getViolationId() == violationId) return v;
        }

        return null;
    }

    // GET ALL VIOLATIONS
    public List<Violation> getAllViolations() {

        List<Violation> list = new ArrayList<>();
        String sql = "SELECT * FROM violations ORDER BY violation_id";

        Connection con = DBConnection.getConnection();
        if (con != null) {
            try (con;
                 PreparedStatement ps = con.prepareStatement(sql);
                 ResultSet rs = ps.executeQuery()) {

                while (rs.next()) {
                    Violation v = new Violation();
                    v.setViolationId(rs.getInt("violation_id"));
                    v.setUserId(rs.getInt("user_id"));
                    v.setVehicleId(rs.getInt("vehicle_id"));
                    v.setViolationType(rs.getString("violation_type"));
                    v.setDescription(rs.getString("description"));
                    v.setViolationDate(rs.getTimestamp("violation_date"));
                    v.setFineAmount(rs.getDouble("fine_amount"));
                    v.setViolationStatus(rs.getString("violation_status"));
                    list.add(v);
                }

                if (!list.isEmpty()) {
                    return list;
                }

            } catch (SQLException e) {
                System.out.println("Error retrieving violations from DB: " + e.getMessage());
            }
        }

        return new ArrayList<>(demoViolations);
    }
}
```

ViolationManagement.java

```
package parking;
```

```
import java.awt.*;
import java.awt.event.*;
import java.sql.Timestamp;
import java.text.SimpleDateFormat;
import java.util.Date;

public class ViolationManagement extends Frame implements ActionListener {

    // ==========================
    // TEXT FIELDS
    // ==========================

    private TextField violationIdField;
    private TextField userIdField;
    private TextField vehicleIdField;
    private TextField violationTypeField;
    private TextField violationDateField;
    private TextField fineAmountField;

    // ==========================
    // DESCRIPTION
    // ==========================

    private TextArea descriptionArea;

    // ==========================
    // CHOICE
    // ==========================

    private Choice statusChoice;

    // ==========================
    // BUTTONS
    // ==========================

    private Button addButton;
    private Button searchButton;
    private Button clearButton;
    private Button backButton;

    // ==========================
    // COLORS
    // ==========================

    private final Color HEADER_COLOR = new Color(35, 55, 75);
    private final Color SECTION_COLOR = new Color(55, 75, 95);
    private final Color BUTTON_COLOR = new Color(45, 95, 140);
    private final Color CLEAR_COLOR = new Color(180, 120, 40);
    private final Color BACK_COLOR = new Color(150, 55, 55);
```

```
private final Color FIELD_COLOR = new Color(248, 250, 252);

// ========================
// CONSTRUCTOR
// ========================

public ViolationManagement() {

    setTitle("Smart Campus Parking - Violation Management");

    setSize(850, 700);

    setLayout(new BorderLayout(15, 15));

    setBackground(new Color(235, 240, 245));

    // ========================
    // WINDOW CLOSE
    // ========================

    addWindowListener(
            new WindowAdapter() {

                @Override
                public void windowClosing(WindowEvent e) {
                    dispose();
                }
            }
    );

    // ========================
    // HEADER
    // ========================

    Panel headerPanel = new Panel(new BorderLayout());

    headerPanel.setBackground(HEADER_COLOR);

    headerPanel.setPreferredSize(
            new Dimension(850, 80)
    );

    Label title = new Label(
            "VIOLATION MANAGEMENT",
            Label.CENTER
    );

    title.setFont(
            new Font("Arial", Font.BOLD, 26)
    );
```

```
title.setForeground(Color.WHITE);

headerPanel.add(
        title,
        BorderLayout.CENTER
);

add(
        headerPanel,
        BorderLayout.NORTH
);

// ========================
// MAIN PANEL
// ========================

Panel mainPanel = new Panel();

mainPanel.setLayout(
        new BorderLayout(15, 15)
);

mainPanel.setBackground(
        new Color(235, 240, 245)
);

// ========================
// LEFT FORM PANEL
// ========================

Panel leftPanel = new Panel();

leftPanel.setLayout(
        new BorderLayout(10, 10)
);

leftPanel.setBackground(Color.WHITE);

// ========================
// VIOLATION DETAILS HEADER
// ========================

Label detailsTitle = new Label(
        "  VIOLATION DETAILS",
        Label.LEFT
);

detailsTitle.setFont(
        new Font("Arial", Font.BOLD, 17)
```

```
);

detailsTitle.setForeground(Color.WHITE);

detailsTitle.setBackground(SECTION_COLOR);

leftPanel.add(
        detailsTitle,
        BorderLayout.NORTH
);

// ========================
// FORM
// ========================

Panel formPanel = new Panel();

formPanel.setLayout(
        new GridLayout(6, 2, 12, 15)
);

formPanel.setBackground(Color.WHITE);

// Violation ID

Label violationIdLabel =
        new Label("Violation ID:");

violationIdLabel.setFont(
        new Font("Arial", Font.BOLD, 14)
);

formPanel.add(violationIdLabel);

violationIdField = new TextField();

violationIdField.setFont(
        new Font("Arial", Font.PLAIN, 14)
);

violationIdField.setBackground(FIELD_COLOR);

formPanel.add(violationIdField);

// User ID

Label userIdLabel =
        new Label("User ID:");

userIdLabel.setFont(
```

```
        new Font("Arial", Font.BOLD, 14)
);

formPanel.add(userIdLabel);

userIdField = new TextField();

userIdField.setFont(
        new Font("Arial", Font.PLAIN, 14)
);

userIdField.setBackground(FIELD_COLOR);

formPanel.add(userIdField);

// Vehicle ID

Label vehicleIdLabel =
        new Label("Vehicle ID:");

vehicleIdLabel.setFont(
        new Font("Arial", Font.BOLD, 14)
);

formPanel.add(vehicleIdLabel);

vehicleIdField = new TextField();

vehicleIdField.setFont(
        new Font("Arial", Font.PLAIN, 14)
);

vehicleIdField.setBackground(FIELD_COLOR);

formPanel.add(vehicleIdField);

// Violation Type

Label typeLabel =
        new Label("Violation Type:");

typeLabel.setFont(
        new Font("Arial", Font.BOLD, 14)
);

formPanel.add(typeLabel);

violationTypeField = new TextField();

violationTypeField.setFont(
```

```
        new Font("Arial", Font.PLAIN, 14)
);

violationTypeField.setBackground(FIELD_COLOR);

formPanel.add(violationTypeField);

// Violation Date

Label dateLabel =
        new Label("Violation Date:");

dateLabel.setFont(
        new Font("Arial", Font.BOLD, 14)
);

formPanel.add(dateLabel);

violationDateField = new TextField();

violationDateField.setFont(
        new Font("Arial", Font.PLAIN, 14)
);

violationDateField.setBackground(FIELD_COLOR);

violationDateField.setText(
        new SimpleDateFormat(
                "yyyy-MM-dd HH:mm:ss"
        ).format(new Date())
);

formPanel.add(violationDateField);

// Fine Amount

Label fineLabel =
        new Label("Fine Amount:");

fineLabel.setFont(
        new Font("Arial", Font.BOLD, 14)
);

formPanel.add(fineLabel);

fineAmountField = new TextField();

fineAmountField.setFont(
        new Font("Arial", Font.PLAIN, 14)
);
```

```
fineAmountField.setBackground(FIELD_COLOR);

formPanel.add(fineAmountField);

leftPanel.add(
        formPanel,
        BorderLayout.CENTER
);

// ========================
// RIGHT PANEL
// ========================

Panel rightPanel = new Panel();

rightPanel.setLayout(
        new BorderLayout(10, 10)
);

rightPanel.setBackground(Color.WHITE);

// ========================
// DESCRIPTION TITLE
// ========================

Label descriptionTitle = new Label(
        "  DESCRIPTION",
        Label.LEFT
);

descriptionTitle.setFont(
        new Font("Arial", Font.BOLD, 17)
);

descriptionTitle.setForeground(Color.WHITE);

descriptionTitle.setBackground(SECTION_COLOR);

rightPanel.add(
        descriptionTitle,
        BorderLayout.NORTH
);

// ========================
// DESCRIPTION AREA
// ========================

descriptionArea = new TextArea(
        "",
```

```
            8,
            25,
            TextArea.SCROLLBARS_VERTICAL_ONLY
        );

        descriptionArea.setFont(
            new Font("Arial", Font.PLAIN, 14)
        );

        descriptionArea.setBackground(FIELD_COLOR);

        rightPanel.add(
            descriptionArea,
            BorderLayout.CENTER
        );

        // ========================
        // STATUS PANEL
        // ========================

        Panel statusPanel = new Panel();

        statusPanel.setLayout(
            new FlowLayout(
                FlowLayout.LEFT,
                10,
                10
            )
        );

        statusPanel.setBackground(Color.WHITE);

        Label statusLabel =
            new Label("Violation Status:");

        statusLabel.setFont(
            new Font("Arial", Font.BOLD, 14)
        );

        statusChoice = new Choice();

        statusChoice.setFont(
            new Font("Arial", Font.PLAIN, 14)
        );

        statusChoice.add("Pending");
        statusChoice.add("Paid");
        statusChoice.add("Cancelled");

        statusPanel.add(statusLabel);
```

```
statusPanel.add(statusChoice);

rightPanel.add(
        statusPanel,
        BorderLayout.SOUTH
);

// ========================
// ADD PANELS
// ========================

mainPanel.add(
        leftPanel,
        BorderLayout.CENTER
);

mainPanel.add(
        rightPanel,
        BorderLayout.EAST
);

add(
        mainPanel,
        BorderLayout.CENTER
);

// ========================
// BUTTON PANEL
// ========================

Panel buttonPanel = new Panel();

buttonPanel.setLayout(
        new FlowLayout(
                FlowLayout.CENTER,
                15,
                12
        )
);

buttonPanel.setBackground(
        new Color(220, 225, 230)
);

// ADD

addButton = new Button(
        "ADD VIOLATION"
);
```

```
addButton.setFont(
        new Font("Arial", Font.BOLD, 14)
);

addButton.setBackground(BUTTON_COLOR);

addButton.setForeground(Color.WHITE);

// SEARCH

searchButton = new Button(
        "SEARCH"
);

searchButton.setFont(
        new Font("Arial", Font.BOLD, 14)
);

searchButton.setBackground(BUTTON_COLOR);

searchButton.setForeground(Color.WHITE);

// CLEAR

clearButton = new Button(
        "CLEAR"
);

clearButton.setFont(
        new Font("Arial", Font.BOLD, 14)
);

clearButton.setBackground(CLEAR_COLOR);

clearButton.setForeground(Color.WHITE);

// BACK

backButton = new Button(
        "BACK"
);

backButton.setFont(
        new Font("Arial", Font.BOLD, 14)
);

backButton.setBackground(BACK_COLOR);

backButton.setForeground(Color.WHITE);
```

```
        buttonPanel.add(addButton);

        buttonPanel.add(searchButton);

        buttonPanel.add(clearButton);

        buttonPanel.add(backButton);

        add(
                buttonPanel,
                BorderLayout.SOUTH
        );

        // ========================
        // EVENT LISTENERS
        // ========================

        addButton.addActionListener(this);

        searchButton.addActionListener(this);

        clearButton.addActionListener(this);

        backButton.addActionListener(this);

        // ========================
        // CENTER WINDOW
        // ========================

        setLocationRelativeTo(null);

        setVisible(true);
    }

    // ================================================
    // ACTION PERFORMED
    // ================================================

    @Override
    public void actionPerformed(ActionEvent e) {

        if (e.getSource() == addButton) {

            addViolation();

        } else if (e.getSource() == searchButton) {

            searchViolation();
```

```
    } else if (e.getSource() == clearButton) {

        clearFields();

    } else if (e.getSource() == backButton) {

        dispose();
    }
}

// ================================================
// ADD VIOLATION
// ================================================

private void addViolation() {

    try {

        // =========================
        // GET VALUES
        // =========================

        String userText =
                userIdField.getText().trim();

        String vehicleText =
                vehicleIdField.getText().trim();

        String violationType =
                violationTypeField.getText().trim();

        String description =
                descriptionArea.getText().trim();

        String fineText =
                fineAmountField.getText().trim();

        // =========================
        // VALIDATION
        // =========================

        if (userText.isEmpty()
                || vehicleText.isEmpty()
                || violationType.isEmpty()
                || description.isEmpty()
                || fineText.isEmpty()) {

            showMessage(
                    "Please fill all mandatory fields."
            );
```

```
    return;
}

// ========================
// NUMERIC VALUES
// ========================

int userId =
        Integer.parseInt(userText);

int vehicleId =
        Integer.parseInt(vehicleText);

double fineAmount =
        Double.parseDouble(fineText);

if (userId <= 0) {

    showMessage(
            "User ID must be greater than 0."
    );

    return;
}

if (vehicleId <= 0) {

    showMessage(
            "Vehicle ID must be greater than 0."
    );

    return;
}

if (fineAmount < 0) {

    showMessage(
            "Fine amount cannot be negative."
    );

    return;
}

// ========================
// DATE
// ========================

Timestamp violationDate;
```

```
        String dateText =
                violationDateField
                        .getText()
                        .trim();

        if (dateText.isEmpty()) {

            violationDate =
                    new Timestamp(
                            System.currentTimeMillis()
                    );

        } else {

            violationDate =
                    Timestamp.valueOf(dateText);
        }

        // ==========================
        // STATUS
        // ==========================

        String status =
                statusChoice.getSelectedItem();

        // ==========================
        // CREATE VIOLATION OBJECT
        // ==========================

        Violation violation =
                new Violation();

        violation.setUserId(userId);

        violation.setVehicleId(vehicleId);

        violation.setViolationType(
                violationType
        );

        violation.setDescription(
                description
        );

        violation.setViolationDate(
                violationDate
        );

        violation.setFineAmount(
                fineAmount
```

```
            );

            violation.setViolationStatus(
                    status
            );

            // ========================
            // DAO
            // ========================

            ViolationDAO dao =
                    new ViolationDAO();

            boolean result =
                    dao.addViolation(violation);

            // ========================
            // RESULT
            // ========================

            if (result) {

                showMessage(
                        "Violation Added Successfully!"
                );

                clearFields();

            } else {

                showMessage(
                        "Violation Addition Failed!"
                );
            }

        } catch (NumberFormatException ex) {

            showMessage(
                    "Please enter valid numeric values for User ID, Vehicle ID and Fine Amount."
            );

        } catch (IllegalArgumentException ex) {

            showMessage(
                    "Invalid date format.\n\n"
                    + "Use:\n"
                    + "yyyy-MM-dd HH:mm:ss"
            );

        } catch (Exception ex) {
```

```
            showMessage(
                    "Error adding violation:\n\n"
                    + ex.getMessage()
            );

            ex.printStackTrace();
        }
    }

    // ================================================
    // SEARCH VIOLATION
    // ================================================

    private void searchViolation() {

        try {

            String idText =
                    violationIdField
                            .getText()
                            .trim();

            if (idText.isEmpty()) {

                showMessage(
                        "Please enter Violation ID."
                );

                return;
            }

            int violationId =
                    Integer.parseInt(idText);

            ViolationDAO dao =
                    new ViolationDAO();

            Violation violation =
                    dao.searchViolation(
                            violationId
                    );

            if (violation != null) {

                // ==========================
                // SET VALUES
                // ==========================

                userIdField.setText(
```

```
            String.valueOf(
                    violation.getUserId()
            )
    );

    vehicleIdField.setText(
            String.valueOf(
                    violation.getVehicleId()
            )
    );

    violationTypeField.setText(
            violation.getViolationType()
    );

    descriptionArea.setText(
            violation.getDescription()
    );

    // ========================
    // DATE
    // ========================

    if (violation.getViolationDate()
            != null) {

        violationDateField.setText(
                violation
                        .getViolationDate()
                        .toString()
        );
    }

    // ========================
    // FINE
    // ========================

    fineAmountField.setText(
            String.valueOf(
                    violation.getFineAmount()
            )
    );

    // ========================
    // STATUS
    // ========================

    if (violation.getViolationStatus()
            != null) {
```

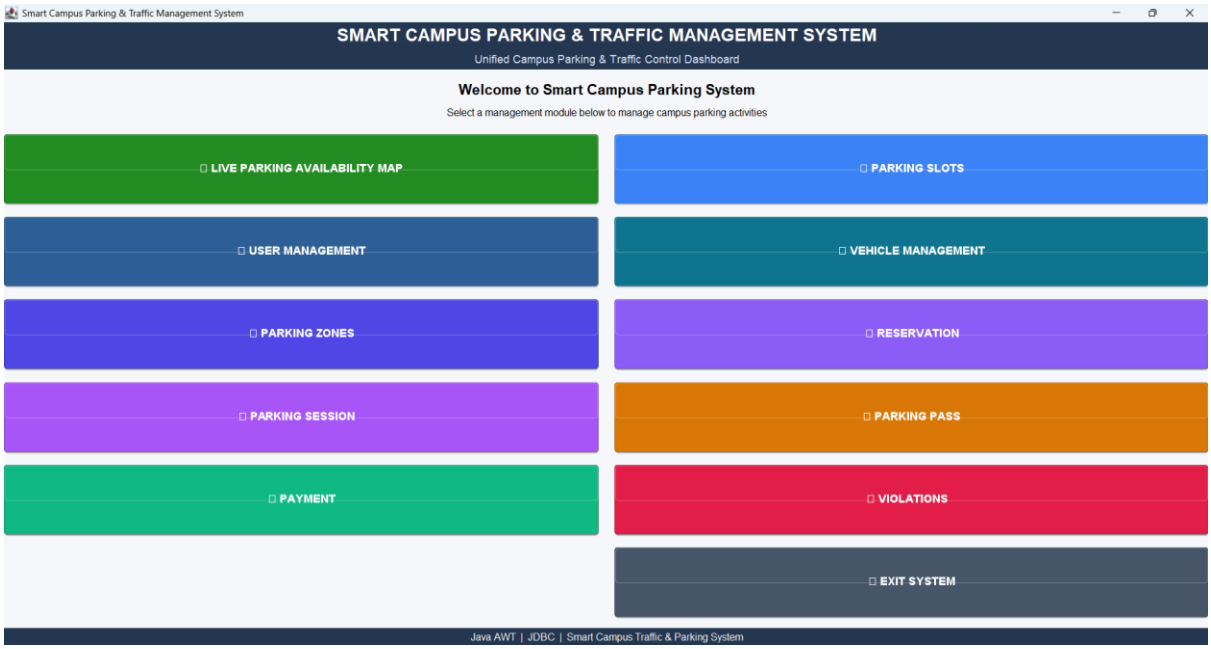
```
                statusChoice.select(
                        violation
                                .getViolationStatus()
                );
            }

            showMessage(
                    "Violation Found Successfully!"
            );

        } else {

            showMessage(
                    "Violation Not Found."
            );
        }

    } catch (NumberFormatException ex) {

        showMessage(
                "Please enter a valid Violation ID."
        );

    } catch (Exception ex) {

        showMessage(
                "Error searching violation:\n\n"
                + ex.getMessage()
        );

        ex.printStackTrace();
    }
}

// ================================================
// CLEAR FIELDS
// ================================================

private void clearFields() {

    violationIdField.setText("");

    userIdField.setText("");

    vehicleIdField.setText("");

    violationTypeField.setText("");

    descriptionArea.setText("");
```

```
        violationDateField.setText(
                new SimpleDateFormat(
                        "yyyy-MM-dd HH:mm:ss"
                ).format(new Date())
        );

        fineAmountField.setText("");

        statusChoice.select("Pending");
    }

    // ================================================
    // MESSAGE DIALOG
    // ================================================

    private void showMessage(String message) {

        final Dialog dialog =
                new Dialog(
                        this,
                        "Parking System",
                        true
                );

        dialog.setSize(450, 190);

        dialog.setLayout(
                new BorderLayout(10, 10)
        );

        dialog.setBackground(
                new Color(245, 247, 250)
        );

        // ========================
        // MESSAGE
        // ========================

        TextArea messageArea =
                new TextArea(
                        message,
                        5,
                        40,
                        TextArea.SCROLLBARS_NONE
                );

        messageArea.setEditable(false);

        messageArea.setFont(
                new Font(
```

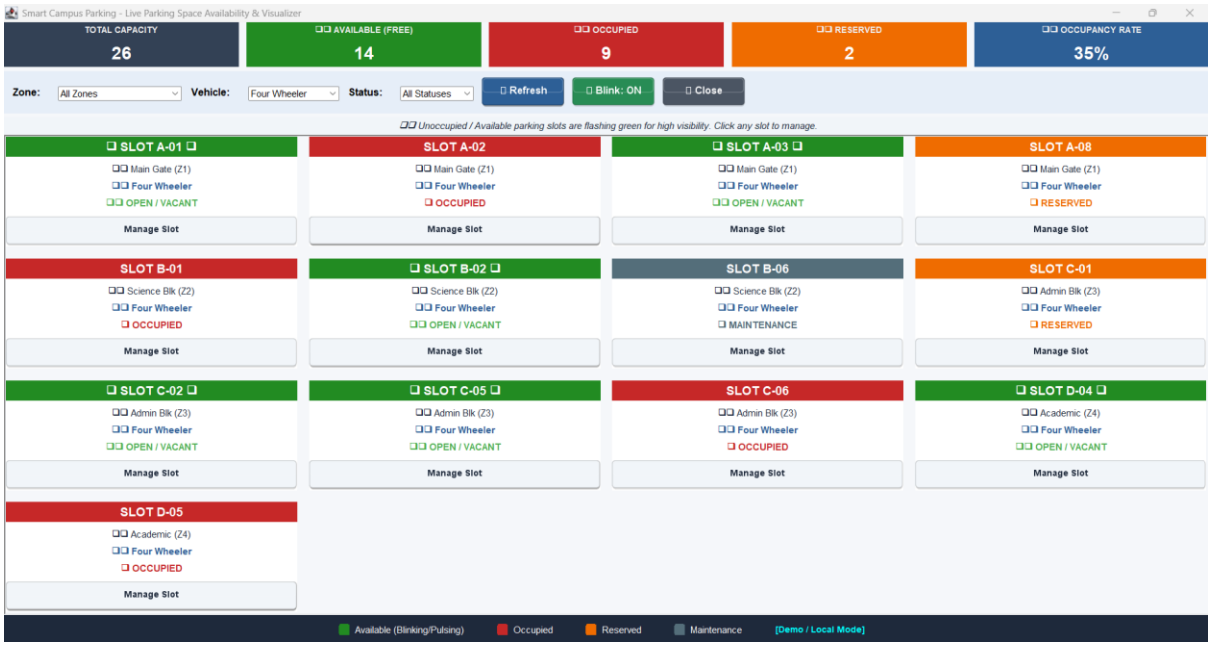
```
                "Arial",
                Font.PLAIN,
                14
        )
);

messageArea.setBackground(
        Color.WHITE
);

dialog.add(
        messageArea,
        BorderLayout.CENTER
);

// ========================
// OK BUTTON
// ========================

Panel buttonPanel =
        new Panel(
                new FlowLayout(
                        FlowLayout.CENTER
                )
        );

Button okButton =
        new Button("OK");

okButton.setFont(
        new Font(
                "Arial",
                Font.BOLD,
                13
        )
);

okButton.setBackground(
        BUTTON_COLOR
);

okButton.setForeground(
        Color.WHITE
);

okButton.addActionListener(
        new ActionListener() {

            @Override
            public void actionPerformed(
```

```
                    ActionEvent e) {

                dialog.dispose();
            }
        }
    );

    buttonPanel.add(okButton);

    dialog.add(
        buttonPanel,
        BorderLayout.SOUTH
    );

    dialog.setLocationRelativeTo(this);

    dialog.setVisible(true);
  }

  // ================================================
  // MAIN METHOD
  // ================================================

  public static void main(String[] args) {

    new ViolationManagement();
  }
}
```

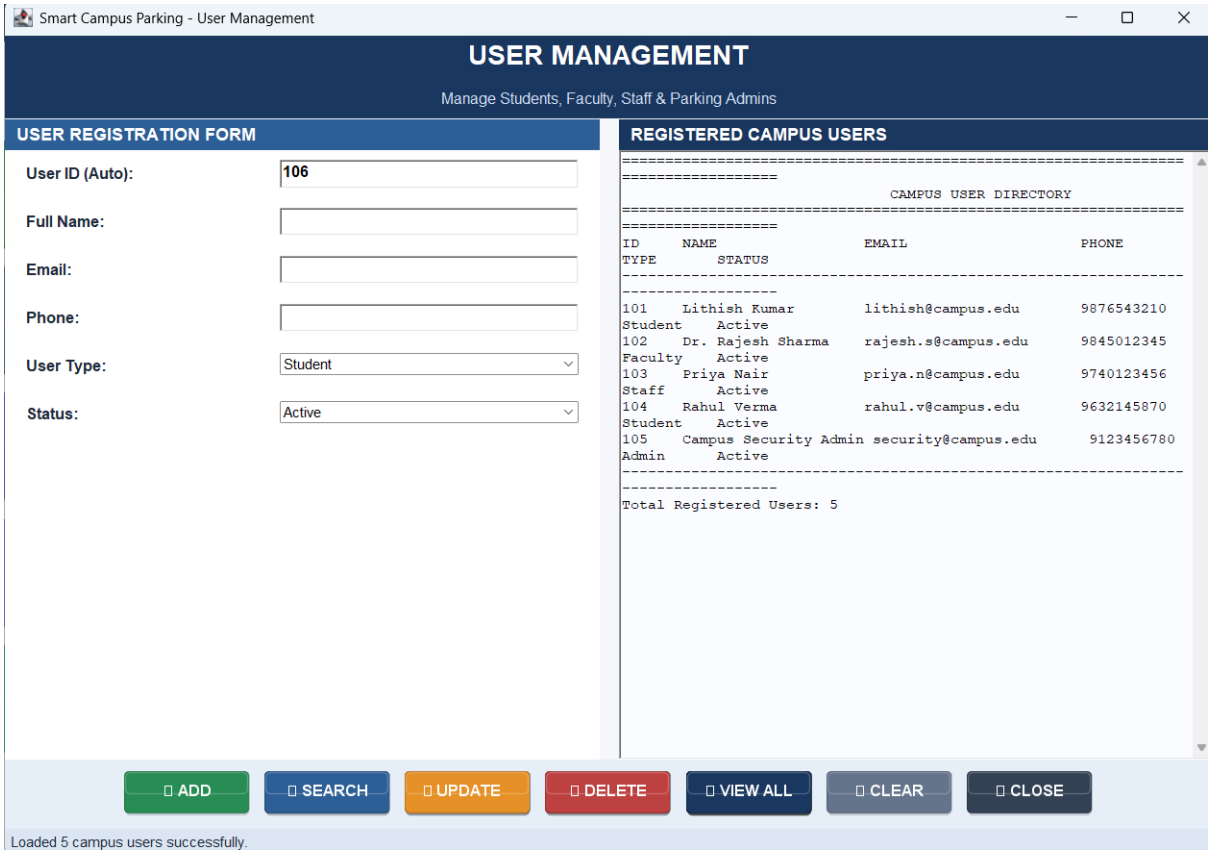
Workflow:

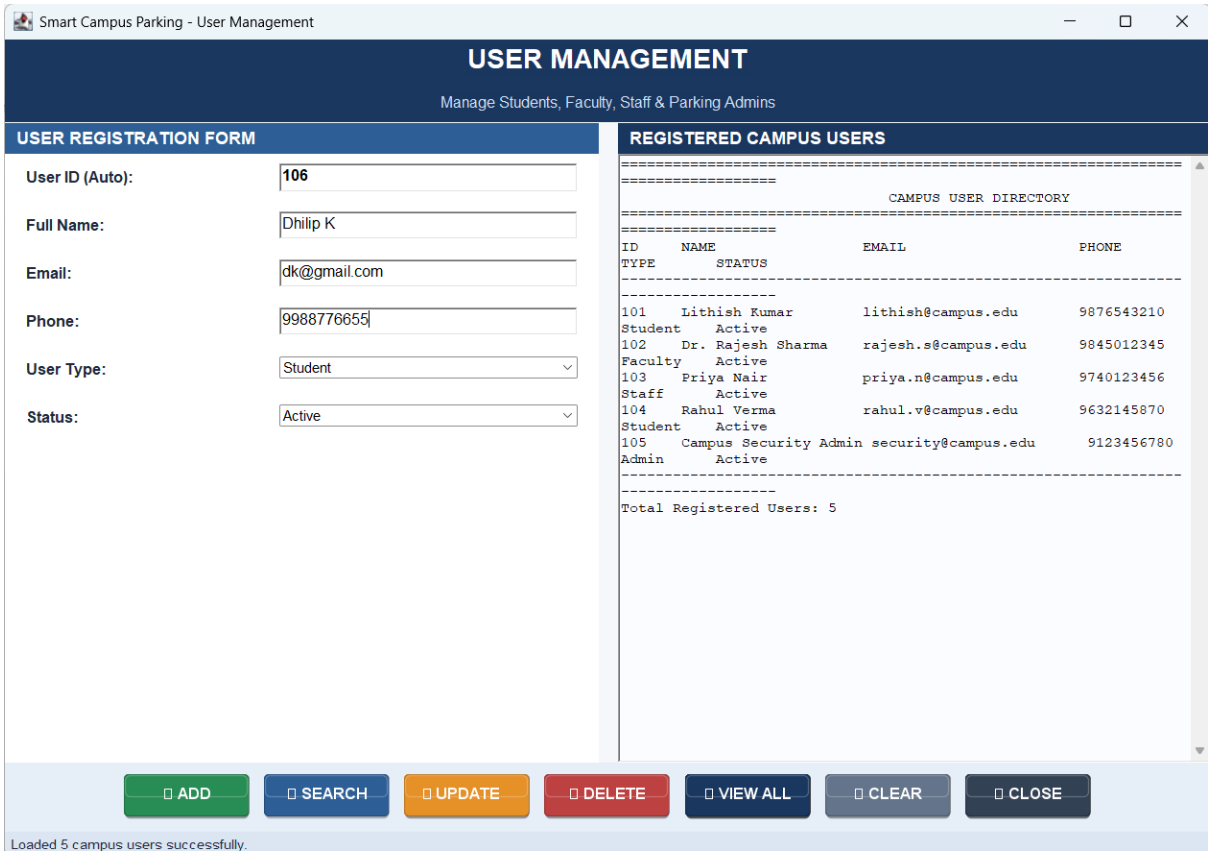


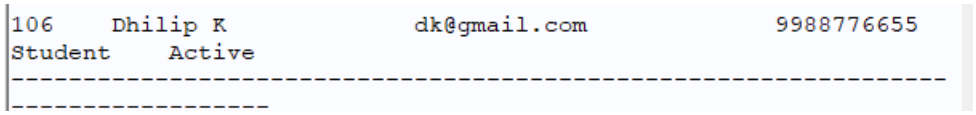
Live Parking Availability Map:



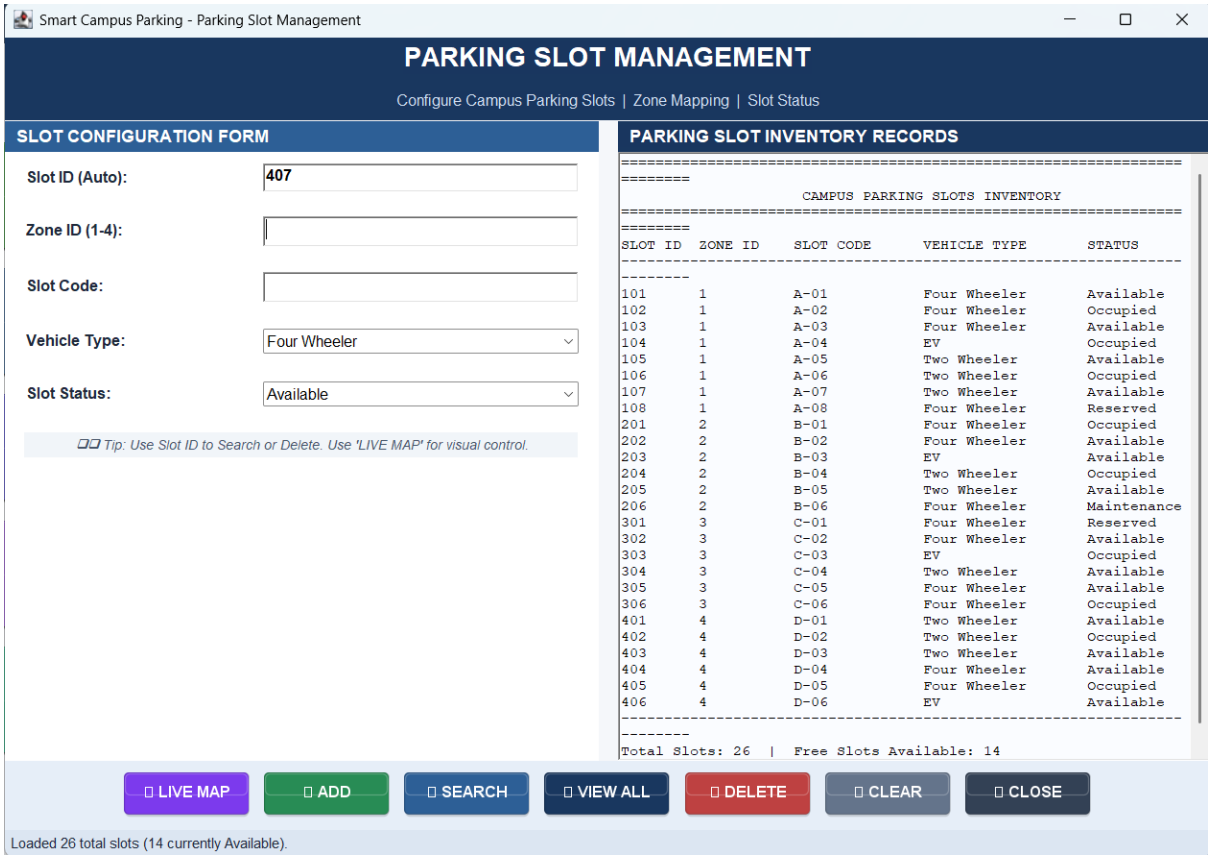
User Management:

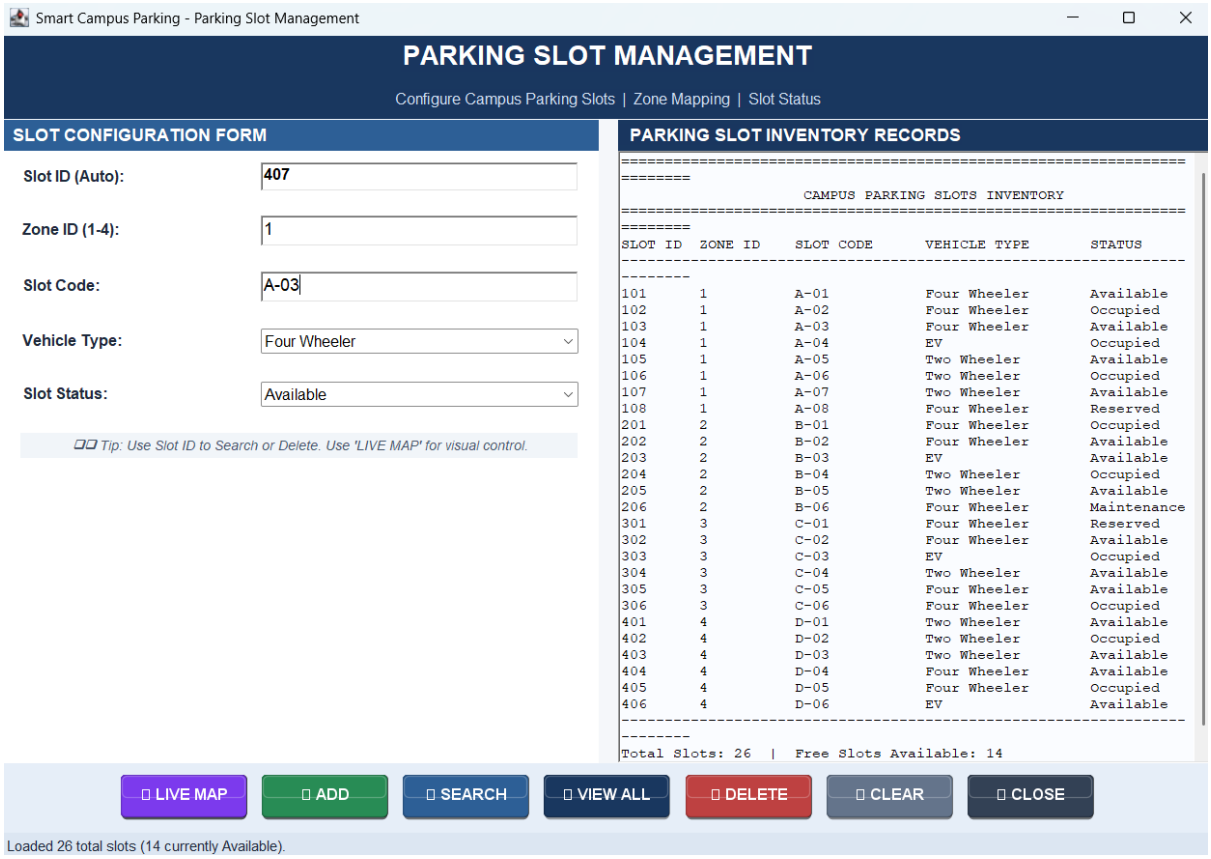

Smart Campus Parking - User Management
USER MANAGEMENT
Manage Students, Faculty, Staff & Parking Admins
USER REGISTRATION FORM
User ID (Auto):
106
Full Name:
Email:
Phone:
User Type:
Student
Status:
Active
REGISTERED CAMPUS USERS
CAMPUS USER DIRECTORY
ID NAME EMAIL PHONE
TYPE STATUS
101 Lithish Kumar lithish@campus.edu 9876543210
Student Active
102 Dr. Rajesh Sharma rajesh.s@campus.edu 9845012345
Faculty Active
103 Priya Nair priya.n@campus.edu 9740123456
Staff Active
104 Rahul Verma rahul.v@campus.edu 9632145870
Student Active
105 Campus Security Admin security@campus.edu 9123456780
Admin Active
Total Registered Users: 5
ADD
SEARCH
UPDATE
DELETE
VIEW ALL
CLEAR
CLOSE
Loaded 5 campus users successfully.


Smart Campus Parking - User Management
USER MANAGEMENT
Manage Students, Faculty, Staff & Parking Admins
USER REGISTRATION FORM
User ID (Auto):
106
Full Name:
Dhilip K
Email:
dk@gmail.com
Phone:
9988776655
User Type:
Student
Status:
Active
REGISTERED CAMPUS USERS
CAMPUS USER DIRECTORY
ID NAME EMAIL PHONE
TYPE STATUS
101 Lithish Kumar lithish@campus.edu 9876543210
Student Active
102 Dr. Rajesh Sharma rajesh.s@campus.edu 9845012345
Faculty Active
103 Priya Nair priya.n@campus.edu 9740123456
Staff Active
104 Rahul Verma rahul.v@campus.edu 9632145870
Student Active
105 Campus Security Admin security@campus.edu 9123456780
Admin Active
Total Registered Users: 5
ADD
SEARCH
UPDATE
DELETE
VIEW ALL
CLEAR
CLOSE
Loaded 5 campus users successfully.


106 Dhilip K dk@gmail.com 9988776655
Student Active

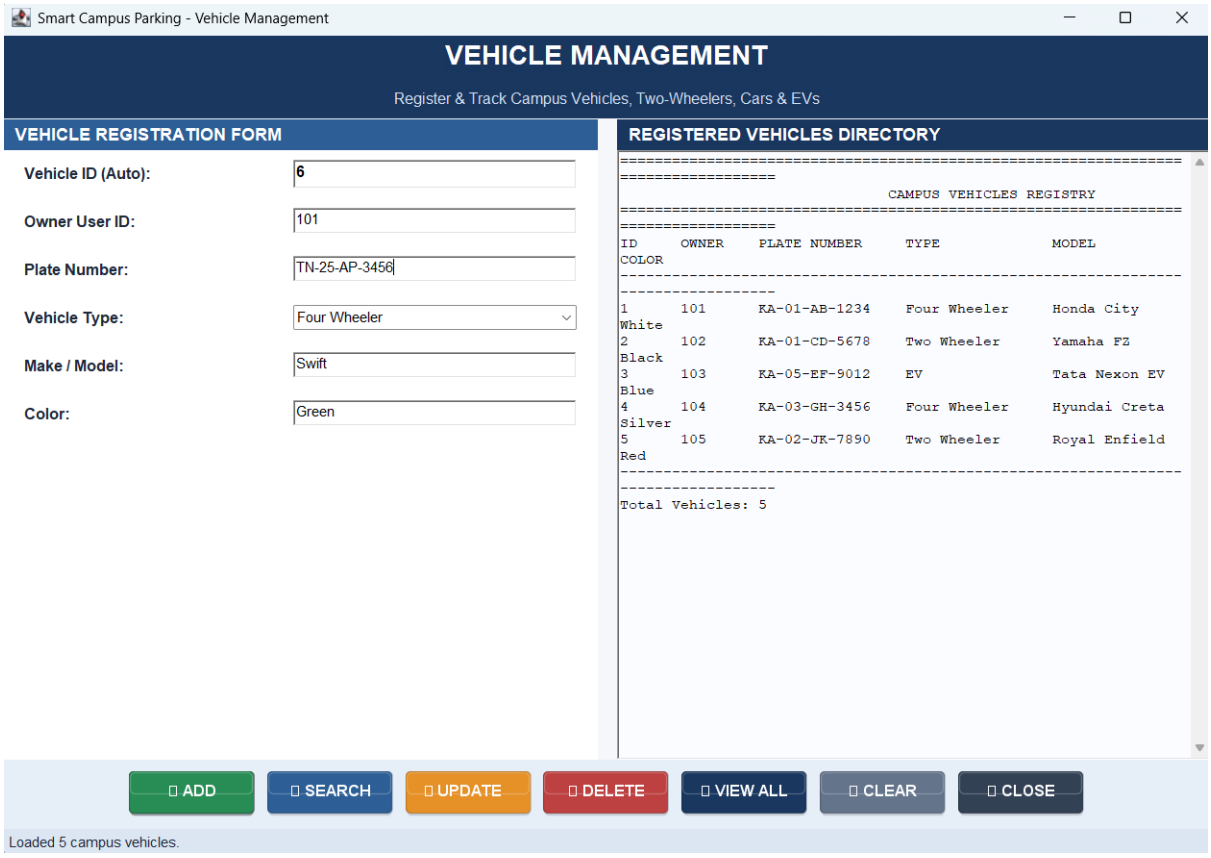
Parking Slot Management:





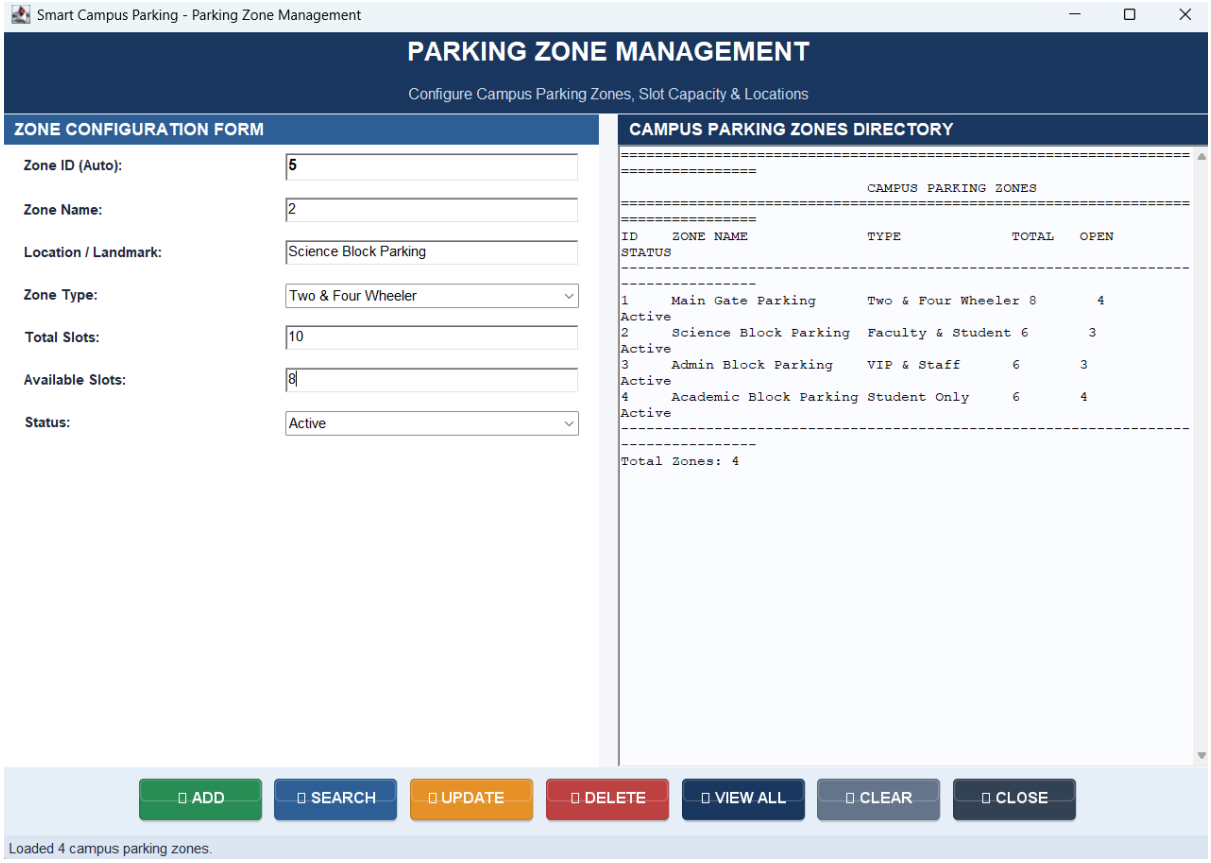
```
407        1          A-03           Four Wheeler       Available
```

Vehicle Management:



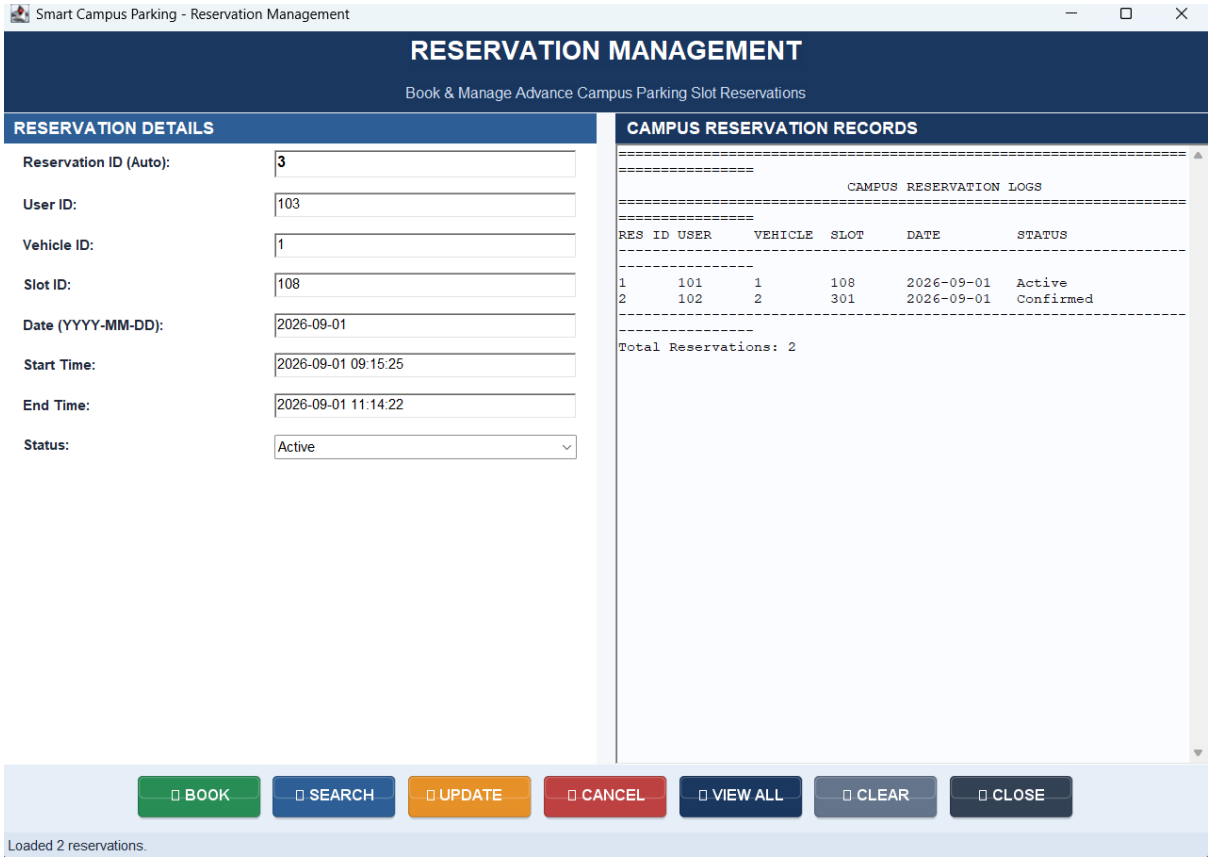

```
6       101       TN-25-AP-3456     Four Wheeler      Swift
Green
-------------------------------------------------------------------
------------------
```

Parking Zone Management:



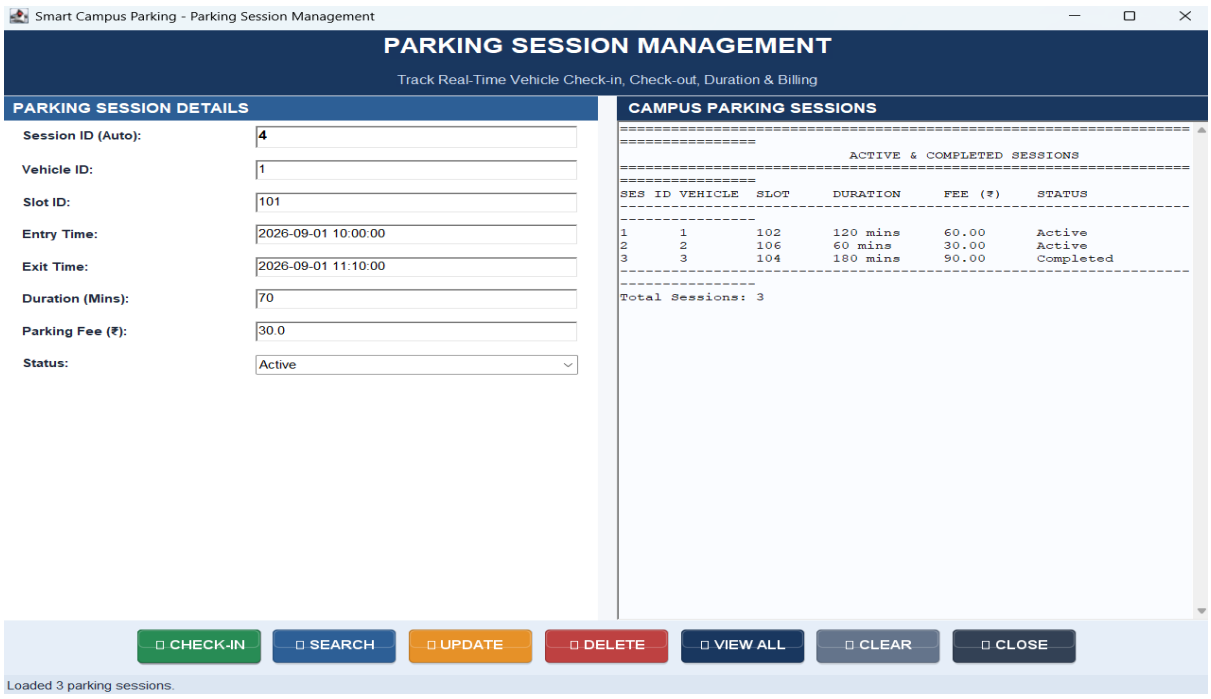

```
5     2                       Two & Four Wheeler 10      8
Active
--------------------------------------------------------------------------------
----------------
```

Reservation Management:



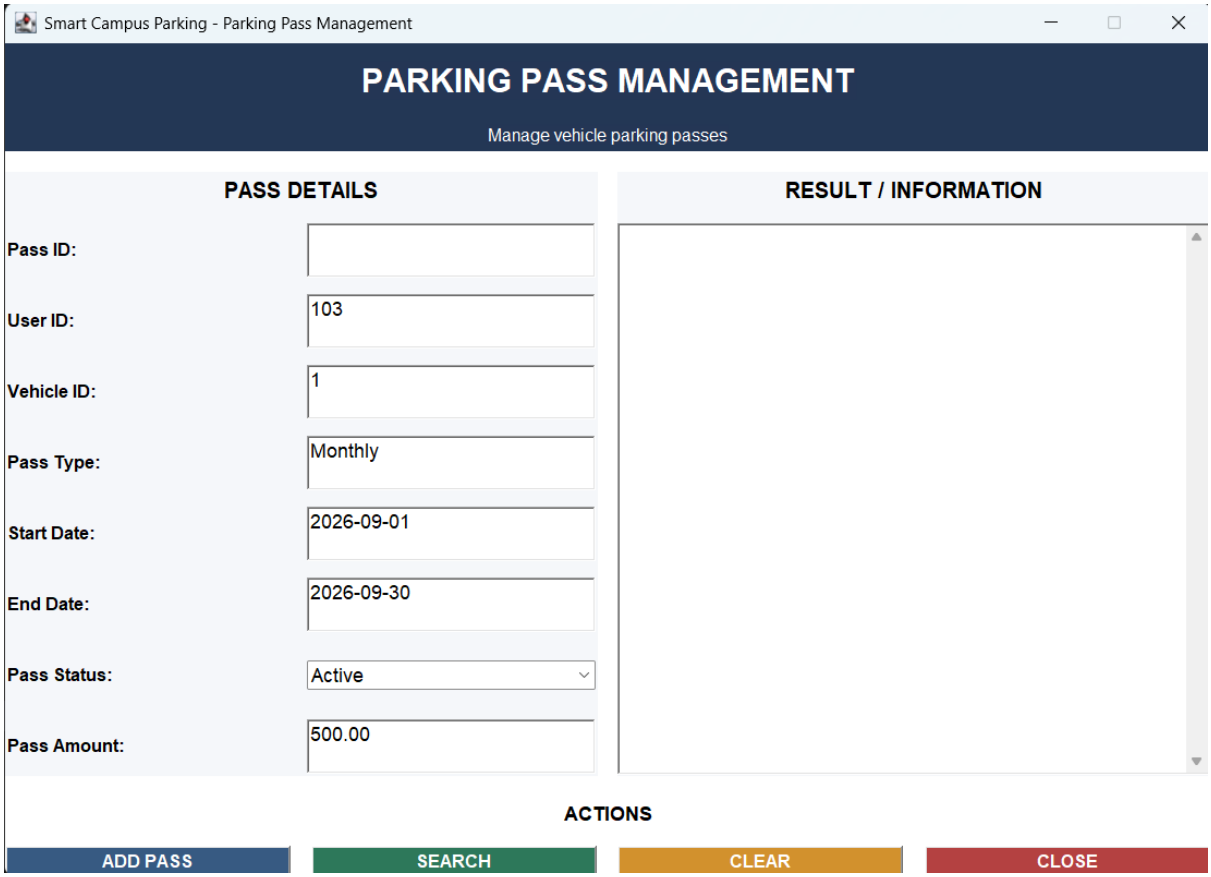
3 103 1 108 2026-09-01 Active

Parking Session Management:

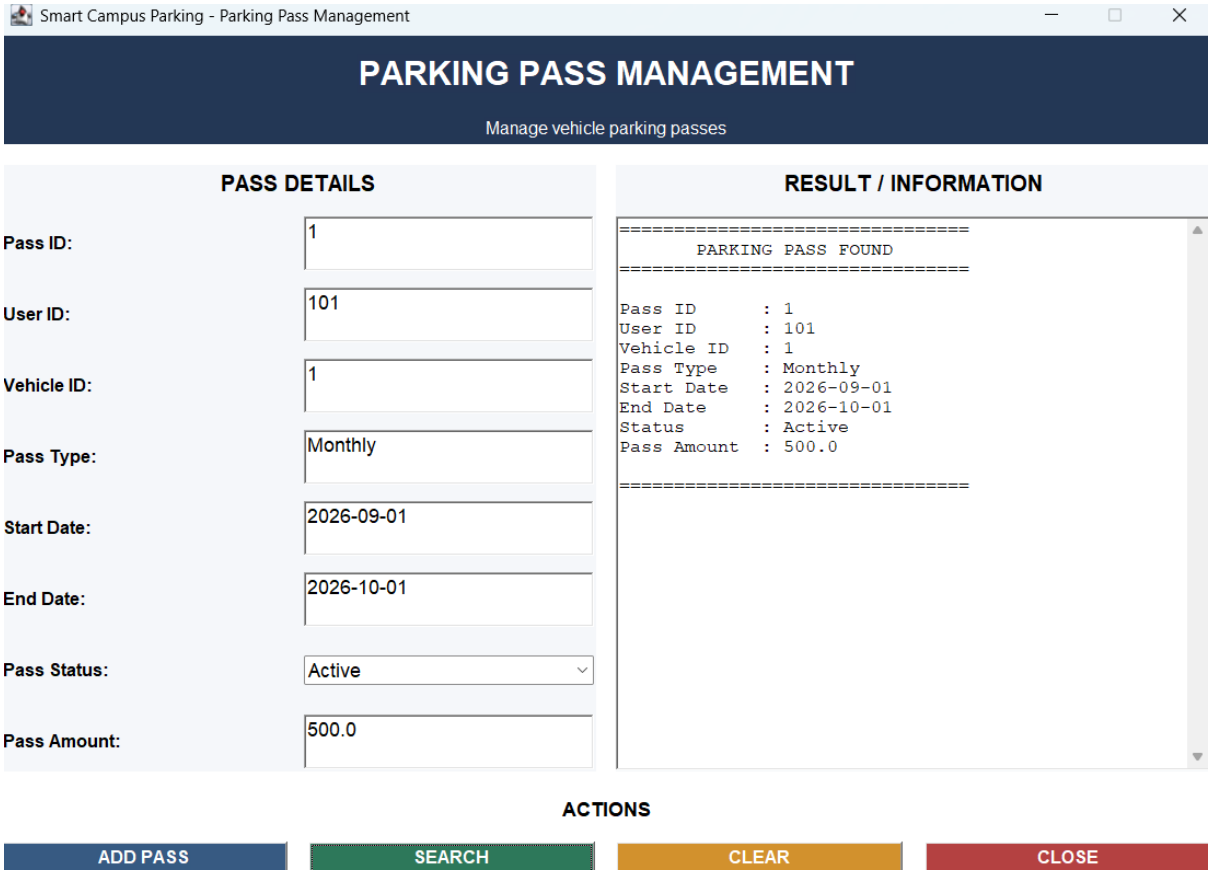


```
4       1         101       70 mins       30.00       Active
-------------------------------------------------------------------
----------------
```

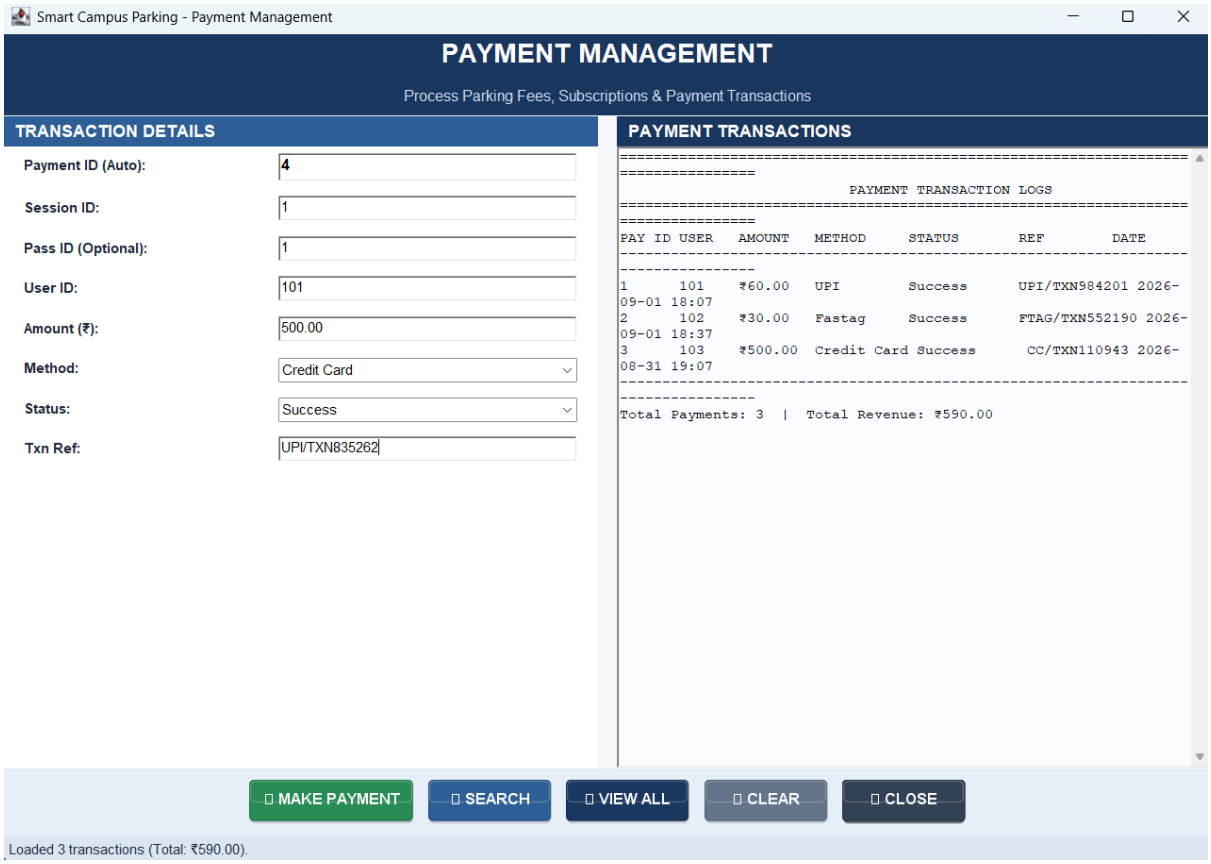
Parking Pass Management:



Search

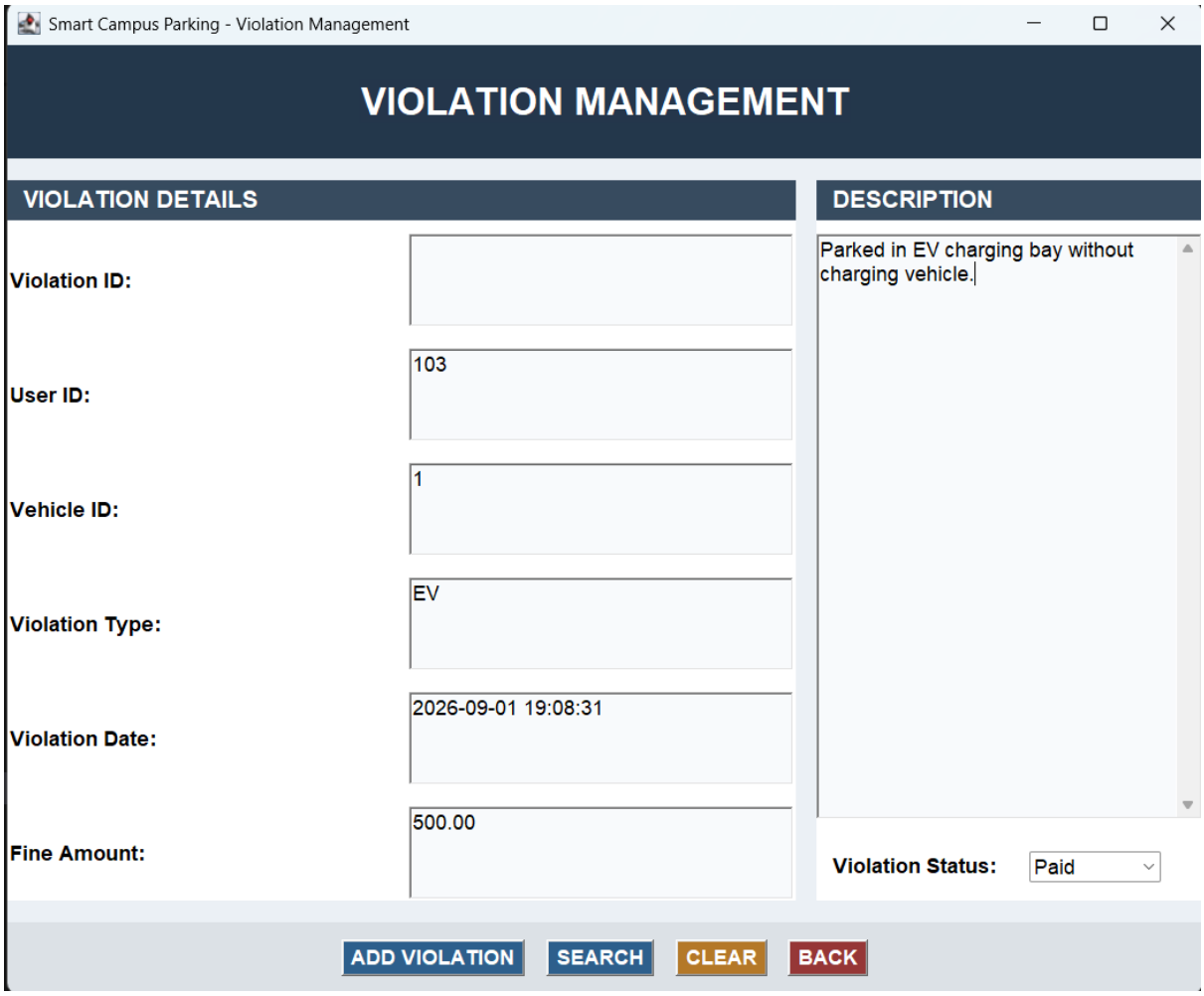

Smart Campus Parking - Parking Pass Management
PARKING PASS MANAGEMENT
Manage vehicle parking passes
PASS DETAILS
Pass ID:
1
User ID:
101
Vehicle ID:
1
Pass Type:
Monthly
Start Date:
2026-09-01
End Date:
2026-10-01
Pass Status:
Active
Pass Amount:
500.0
RESULT / INFORMATION
==========
PARKING PASS FOUND
==========
Pass ID : 1
User ID : 101
Vehicle ID : 1
Pass Type : Monthly
Start Date : 2026-09-01
End Date : 2026-10-01
Status : Active
Pass Amount : 500.0
==========
ACTIONS
ADD PASS
SEARCH
CLEAR
CLOSE

Payment:


Smart Campus Parking - Payment Management
PAYMENT MANAGEMENT
Process Parking Fees, Subscriptions & Payment Transactions
TRANSACTION DETAILS
Payment ID (Auto):
4
Session ID:
1
Pass ID (Optional):
1
User ID:
101
Amount (₹):
500.00
Method:
Credit Card
Status:
Success
Txn Ref:
UPI/TXN835262
PAYMENT TRANSACTIONS
PAYMENT TRANSACTION LOGS
PAY ID USER AMOUNT METHOD STATUS REF DATE
1 101 ₹60.00 UPI Success UPI/TXN984201 2026-09-01 18:07
2 102 ₹30.00 Fastag Success FTAG/TXN552190 2026-09-01 18:37
3 103 ₹500.00 Credit Card Success CC/TXN110943 2026-08-31 19:07
Total Payments: 3 | Total Revenue: ₹590.00
MAKE PAYMENT
SEARCH
VIEW ALL
CLEAR
CLOSE
Loaded 3 transactions (Total: ₹590.00).

```
4       101     ₹500.00  Credit Card Success      UPI/TXN835262 2026-
09-01 19:08
-------------------------------------------------------------------
---------------
```

Violation Management:



Student Name: K. Dhilip
Register Number: 192365013
Project Title: Smart Campus Parking & Traffic Management System

Individual Role/Contribution

I worked on the project as a Java developer and a system integration developer. I helped create the database connectivity, application logic, core Java program, and integration of the many modules into the overall Smart Campus Parking & Traffic Management System.

Modules/Tasks Completed

- Developed the core Java application using Eclipse IDE.
- Worked on the overall system structure and module integration.
- Implemented user and vehicle management functionalities.
- Worked on parking zone and parking slot management.
- Implemented parking reservation and parking session operations.
- Worked on database connectivity using JDBC and MySQL.
- Developed and integrated DAO classes for database operations.
- Implemented CRUD operations and input validation.
- Tested the complete application and corrected errors.

Integrated the different modules into the final working system.

Brief Description of Work

I contributed to the development of the main application and integration of the Smart Campus Parking & Traffic Management System. I worked with the Java classes responsible for users, vehicles, parking zones, parking slots, reservations, and parking sessions. I also contributed to database connectivity and DAO-based database operations.

I tested the application functionality, identified errors, corrected implementation issues, and integrated the different modules so that they could work together as one complete system.

Outcome/Learning

Through this project, I gained practical experience in Java, OOP, JDBC, MySQL, DAO architecture, CRUD operations, debugging, testing, and system integration. I also improved my problem-solving and software development skills.

Student Name: Lithish Kumar
Register Number: 192424169
Project Title: Smart Campus Parking & Traffic Management System

Individual Role/Contribution

I worked as an Application Module and Testing Developer on the project. I helped with the system's integration, validation, testing, parking-related features, and application module implementation.

Modules/Tasks Completed

- Worked on parking slot and parking zone functionalities.
- Contributed to reservation management.
- Worked on parking session functionality.
- Implemented parking violation management.
- Worked on vehicle-related operations.
- Assisted with user management functionality.
- Implemented validation for application inputs.
- Tested different system operations and scenarios.
- Identified and reported errors during development.
- Assisted in integrating all modules into the final application.

Brief Description of Work

I contributed to the development and testing of the parking management features of the Smart Campus Parking & Traffic Management System. I worked with modules related to parking zones, parking slots, reservations, parking sessions, vehicles, and parking violations.

I also participated in testing the system using different scenarios, checking whether parking operations were working correctly, and identifying errors. I assisted in integrating the different modules with the main application and database.

Outcome/Learning

Through this project, I gained practical experience in Java programming, application development, parking management logic, database operations, input validation, testing, debugging, and system integration. I also improved my teamwork, communication, and problem-solving skills.

Student Name: Mohammed Subahaaan H
Register Number: 192572165
Project Title: Smart Campus Parking & Traffic Management System

Individual Role/Contribution

I worked as a database and backend developer on the project. I helped with database-related tasks, backend component implementation, and the system's parking information management.

Modules/Tasks Completed

- Worked on the MySQL database structure and database connectivity.
- Developed and worked with DAO classes for database operations.
- Implemented parking pass management functionality.
- Worked on parking session database operations.
- Implemented payment management and payment-related database operations.
- Worked on reservation data management.
- Implemented CRUD operations for the required entities.
- Tested database operations with different inputs.
- Identified and corrected database-related errors.
- Assisted in integrating backend modules with the main application.

Brief Description of Work

I contributed mainly to the backend and database-related functionality of the Smart Campus Parking & Traffic Management System. I worked with the DAO classes responsible for communicating with the database and handling operations such as inserting, retrieving, updating, and deleting records.

I contributed to modules related to parking passes, parking sessions, reservations, and payments. I also tested database operations and helped identify and resolve errors during development. The backend components were integrated with the other modules to form the complete application.

Outcome/Learning

Through this project, I gained practical experience in Java backend development, MySQL, JDBC, DAO architecture, database operations, CRUD functionality, testing, debugging, and module integration. I also developed better understanding of how application logic communicates with a relational database.